SAMSUNG
Together for Tomorrow!
Enabling People
Education for Future Generations

Estd : 2001

**RN SHETTY TRUST®**

# RNS INSTITUTE OF TECHNOLOGY

Autonomous Institution affiliated to Visvesvaraya Technological University, Belagavi
Approved By AICTE, New Delhi. Accredited by NAAC 'A+' Grade
Channasandra, Dr. Vishnuvardhan Road, Bengaluru - 560 098
Ph:(080)28611880,28611881 URL: www.rnsit.ac.in

# Samsung Innovation Campus

## Artificial Intelligence Programme

## CERTIFICATE

This is to certify that the project work entitled **"Indic-Classify: Kannada News Headline Categorization with Supervised Deep Learning Models"** has been successfully carried out at the **Samsung Innovation Campus,** by **Tejas M Prasad**, bearing **1RN22CD106**, **Santhosh A** bearing **1RN22CD087**, **S Sarvesh Balaji** bearing **1RN22CD076**, **Tharun Teja Kethineni** bearing **1RN22CD108**, Bonafide students of **RNS Institute of Technology**, in partial fulfilment of the requirements for the award of the Certificate in **Artificial Intelligence Programme.**

It is further certified that a research paper based on the aforesaid project has been prepared and submitted to an **IEEE Conference**, in partial fulfilment of the programme requirements. A copy of the said paper has been appended to this report.

The project report has been thoroughly reviewed and is hereby approved, as it satisfactorily meets the prescribed standards and requirements with respect to project work for the programme.

| | | |
|---|---|---|
| **Dr. Sunitha K**<br>Associate Professor<br>Dept. of ISE<br>RNSIT | **Dr. Prashanth Kannadaguli**<br>Senior Data Science Trainer<br>Dhaarini Academy of<br>Technical Education | **Dr. Suresh L**<br>Chief-Coordinator<br>SIC-RNSIT |

# Table of Contents

# I. Introduction

## A. Problem Statement

The contemporary digital media ecosystem is characterized by an unprecedented volume and velocity of information, a phenomenon amplified by the exponential growth of a regional language internet user base in India, which now exceeds 400 million individuals and is projected to surpass 900 million by 2025. This ecosystem is not merely a collection of websites but a complex, interconnected network of stakeholders, platforms, applications, and third-party data services that constitute the modern information battleground. For news organizations serving large linguistic populations, such as the global Kannada-speaking community, the ability to efficiently manage, categorize, and deliver content is not merely an operational advantage but a fundamental requirement for survival and growth in a competitive digital landscape. This surge is driven primarily by users in rural and semi-urban areas, with reports indicating that 98% of all Indian internet users access content in Indic languages, making the vernacular internet the new mainstream.

The prevailing industry practice, however, often lags behind the technological frontier, relying heavily on the manual classification of news articles. This process, wherein human editors read and assign category tags to each piece of content, is fraught with inherent inefficiencies that create a significant operational bottleneck, directly impeding the speed and quality of news delivery. An operational bottleneck is a point of congestion in a business process where the volume of work exceeds the capacity of that stage, causing the entire workflow to slow down or stall.[7] In the context of a digital newsroom, this manual classification step represents a classic structural bottleneck, built into the editorial workflow itself.

This traditional, manual approach is profoundly time-intensive and labor-intensive. The editorial workflow involves multiple stages, from ideation and reporting to writing, fact-checking, and final approval. The manual categorization step adds a significant delay at a critical juncture just before publication. In a digital newsroom, where the relevance of information can decay in minutes, this manual delay directly obstructs the speed of news dissemination. The "golden hour" of news—the critical period of maximum public interest immediately following a breaking event—can be missed if content is mired in a manual tagging queue. This latency has a direct, negative impact on functionalities essential for a modern digital media platform.

Real-time news aggregation, which relies on the rapid ingestion and categorization of content from multiple sources to provide users with a consolidated view of current events, becomes unfeasible. These systems require structured, categorized metadata to function; a delay in providing this metadata renders the content invisible to the aggregator until its relevance has passed. Similarly, personalized content recommendation engines, which require structured, categorized data to understand user preferences and build user profiles, are starved of the timely information they need to function effectively. These engines operate through a cycle of data collection (e.g., which articles a user reads), analysis (identifying patterns in consumption), and filtering to serve relevant content. If the input data lacks accurate category labels, the engine cannot discern user interests (e.g., a preference for 'Sports' over 'Politics') and defaults to serving a generic, unengaging content feed. Consequently, the user experience is diminished; readers are presented with generic content rather than tailored information, leading to lower engagement, reduced session times, and ultimately, a decline in platform loyalty.

Furthermore, the reliance on manual labor introduces a high degree of susceptibility to human error and inconsistency. The process of manual categorization is inherently subjective; different editors may interpret categorization guidelines differently, or their judgment may be affected by fatigue or cognitive biases, leading to inconsistent tagging that corrupts the integrity of the content database. This inconsistency undermines the reliability of any downstream system, from internal analytics that track content performance to user-facing search functions that rely on accurate tags to retrieve relevant articles. The economic costs are also substantial. As a news organization scales its content production to meet growing demand, a manual classification model requires a linear increase in the number of human editors, a model that is financially and logistically unsustainable in the face of exponential content growth. Automation, therefore, is not a luxury but a competitive necessity, offering a path to scalability, consistency, and efficiency that manual processes cannot match.

This operational challenge is symptomatic of a deeper, more systemic problem within the field of Natural Language Processing (NLP): a stark and persistent disparity between the resources available for different languages. The global NLP landscape is dominated by a small number of "high-resource" languages, most notably English, which benefit from a mature, tool-rich ecosystem including vast annotated datasets, pre-trained models, and extensive linguistic resources.[9] In stark contrast, the vast majority of the world's languages, including Kannada, are classified as "low-resource," lacking the critical components required for the development of advanced language technologies. This scarcity of resources—particularly large-scale, high-quality, annotated datasets—is the primary impediment to progress.

While general-purpose multilingual models like mBERT and XLM-RoBERTa exist, their performance on specific low-resource languages is often suboptimal due to the "curse of multilinguality". This phenomenon describes the degradation in per-language performance that occurs as a single model's fixed capacity is diluted across an increasing number of languages (often over 100). This dilution prevents the model from developing a deep, nuanced representation of the unique morphological and syntactic structures of any single low-resource language, such as Kannada. This project is designed as a direct response to this critical business and technological need within the Kannada digital ecosystem.

## B. Objectives

The "Indic-Classify" project is designed as a direct and systematic response to the multifaceted challenges outlined in the problem statement. Its primary objective is to design, develop, and evaluate a state-of-the-art deep learning model specifically engineered to automatically and accurately classify Kannada news headlines. This overarching goal is broken down into four specific, interconnected objectives, each addressing a critical aspect of the problem.

1. **To Create a Large-Scale, High-Quality Dataset:** The project confronts the foundational issue of data scarcity head-on. The lack of standardized, publicly available datasets is a primary bottleneck hindering progress in low-resource NLP.[23] By collecting, cleaning, and meticulously annotating a comprehensive dataset of over 100,000 Kannada news headlines, the project aims to create the single most critical resource that has been lacking in the field. The commitment to making this dataset publicly available is a strategic decision intended to catalyze future research and provide a common ground for benchmarking and reproducible science. This objective involves a hybrid data sourcing strategy, combining the scale of automated web scraping with the quality assurance of manual curation, ensuring the final corpus is both large enough for training deep learning models and reliable enough for rigorous scientific evaluation.
2. **To Conduct a Comparative Evaluation of Architectures:** The project embraces scientific rigor by moving beyond the implementation of a single model. It aims to implement, train, and rigorously benchmark several distinct deep learning architectures on the same standardized dataset. This includes classical deep learning models like Convolutional Neural Networks (CNNs), which excel at identifying local patterns like n-grams, and Long Short-Term Memory networks (LSTMs), which are designed to process sequential information. It also includes a direct comparison of state-of-the-art Transformer-based models such as mBERT, DistilMBERT, and XLM-RoBERTa. This comparative analysis is essential to empirically validate the hypothesis that specialized models can outperform general ones and to provide a clear, data-driven understanding of the trade-offs between different architectural choices in terms of accuracy, computational efficiency, and robustness—critical factors for real-world deployment.
3. **To Develop and Optimize a High-Performance Model:** The project focuses on applied research with a clear performance goal. Leveraging the principles of transfer learning—a paradigm where a model is pre-trained on a massive general-language dataset and then fine-tuned on a smaller, task-specific one—the objective is to adapt the best-performing pre-trained language model for the Kannada news classification task. This phase is dedicated to achieving the highest possible classification accuracy and F-score, translating the theoretical advantages of a given architecture into state-of-the-art real-world performance. This involves a meticulous process of hyperparameter tuning, the implementation of advanced optimizers like AdamW, and the use of learning rate schedulers to ensure stable and effective convergence toward an optimal solution.
4. **To Build a Functional Prototype for Demonstration:** The project aims to bridge the gap between abstract

academic research and tangible, practical application. By developing a working prototype web application that integrates the final, optimized model, the project will showcase the system's utility and performance in an interactive manner. The creation of a functional prototype is a critical step in the applied NLP development cycle, as it facilitates early user feedback, validates technical feasibility, and provides a compelling demonstration of the project's value to a wider audience, including potential industry stakeholders and the broader research community. This deliverable ensures that the project's outcomes are not confined to academic papers but are made accessible and demonstrable.

## C. Significance and Motivation

The motivation for the Indic-Classify project is rooted in a confluence of practical, technical, and research-oriented imperatives that collectively underscore its significance. This tripartite justification provides a compelling rationale for undertaking a project of this scope and complexity.

**Practical Significance:** The project's potential for substantial real-world impact is a primary driver. For the millions of Kannada readers worldwide, an effective classification system promises to fundamentally improve their digital experience. This is achieved through enhanced content discovery, where users can more easily find articles on topics of interest; personalized news feeds that are tailored to individual reading habits; and more efficient information retrieval via improved search and categorization functionalities. For regional media organizations, the project offers a scalable, automated solution to a critical and persistent content management problem. By automating the classification process, these organizations can reduce operational costs associated with manual labor, increase the speed of news delivery to stay competitive, and support their broader digital transformation in an increasingly crowded market. The potential applications are extensive and varied, ranging from powering automated news aggregation platforms and enabling personalized topic subscriptions for users, to streamlining the dissemination of public information in e-governance systems and organizing the vast digital libraries and cultural archives that preserve linguistic heritage.

**Technical Motivation:** The project is technically motivated by the formidable challenge of adapting and applying state-of-the-art NLP architectures to the nuanced and data-scarce environment of a low-resource language. Kannada, with its rich morphology and agglutinative nature, presents a complex test case for modern language models. This provides a valuable opportunity to systematically investigate the efficacy of transfer learning under these constraints. A core technical question the project addresses is the performance differential between general-purpose multilingual models (like mBERT) and Indic-specific pre-trained models. This benchmarking is crucial for understanding whether models specifically designed to handle the complex morphological and syntactic structures of Indian languages offer a tangible performance advantage, thereby guiding future research and development in the field of Indic NLP. The project also explores the critical trade-offs between model size, computational cost, and predictive accuracy, providing practical insights for organizations making decisions about real-world deployment scenarios where resources may be constrained.

**Research Significance:** The project's research significance is centered on addressing a critical gap in the academic literature. The body of work on Kannada text classification is nascent, primarily because the lack of large-scale, standardized datasets has hindered the kind of rapid, iterative progress seen in high-resource languages. A central motivation for this project is to remedy this situation directly. The creation and, crucially, the public release of a new, meticulously annotated dataset of Kannada news headlines will constitute a valuable and lasting contribution to the NLP research community. This resource has the potential to serve as a standard benchmark for future studies, enabling reproducible research, fostering innovation, and encouraging a new wave of researchers to tackle the unique and interesting challenges of Indic NLP. This act of releasing the data challenges the "data hoarding" mentality that can stifle academic progress and instead promotes a collaborative, open-science approach to solving these important problems, lowering the barrier to entry for future researchers and developers.

## D. Scope and Limitations

To ensure a focused and achievable outcome within the project's timeframe, its boundaries are defined by a clear scope and a transparent acknowledgment of its inherent limitations. This delineation is crucial for managing expectations and providing a clear context for the project's results and contributions.

The **Scope** of the project is precisely delineated as follows, outlining the specific parameters and boundaries of the investigation:

- **Input Data:** The models are trained and evaluated exclusively on the textual content of news headlines. This strategic decision was made to create a highly focused, computationally tractable, and empirically manageable classification problem. Processing full articles would introduce significant computational overhead, require vast data storage resources, and complicate the initial problem definition. Therefore, the granular content and broader textual nuances of the full articles are intentionally not considered within the ambit of this research. This constraint allows for a deep dive into headline-based classification without diluting resources on extensive full-text analysis.
- **Classification Task:** The project specifically addresses a multi-class classification problem. This involves assigning news headlines to one of a predefined, static set of 10 distinct news categories. These categories have been carefully selected to represent common news domains and include: Entertainment, Sports, Politics, Technology, Health, Crime, Agriculture, International News, Economics, and Literature. The fixed nature of these categories implies that the model is designed to operate within this specific taxonomy and is not intended for dynamic category generation or adaptation.
- **Technical Implementation:** The investigation and deployment of deep learning models are strictly confined to three major architectural families, chosen for their prevalence and effectiveness in natural language processing (NLP) tasks. These include: Convolutional Neural Networks (CNNs), renowned for their ability to capture local patterns; Long Short-Term Memory networks (LSTMs), favored for their sequential data processing capabilities; and various Transformer-based architectures, specifically selected for their state-of-the-art performance in capturing long-range dependencies and contextual embeddings. The Transformer architectures explored are mBERT (multilingual Bidirectional Encoder Representations from Transformers), DistilMBERT (a distilled version of mBERT for efficiency), and XLM-RoBERTa (Cross-lingual Language Model RoBERTa), each offering distinct advantages in terms of performance and computational footprint.
- **Core Deliverables:** The project is committed to producing several tangible and significant outputs that collectively represent its contribution and utility. These primary deliverables include: a meticulously curated dataset comprising over 100,000 news headlines, serving as a valuable resource for future NLP research; the final trained classification model, which is the culmination of the training and optimization efforts; a comprehensive comparative performance analysis report, detailing the efficacy, strengths, and weaknesses of the different architectural families and models; and a functional web application prototype, developed primarily for demonstration purposes to showcase the model's capabilities in a user-friendly interface.

The project also acknowledges the following inherent **Limitations**, which are crucial for a realistic interpretation of its findings and for guiding future research directions:

- **Data Dependency:** The overall performance, robustness, and generalizability of the final classification model are fundamentally and inextricably linked to the intrinsic quality, sheer size, and representational diversity of the dataset created during the initial phase of this project. Any inherent biases, omissions, or systemic gaps present within the dataset—such as the underrepresentation of specific niche topics, an overrepresentation of content sourced from a particular news outlet, or the prevalence of certain idiosyncratic writing styles—will inevitably be learned, amplified, and ultimately reflected in the model's predictive behavior. This can potentially lead to skewed or inaccurate predictions when the model is deployed in real-world scenarios with unseen data distributions.
- **Contextual Constraints:** By making a deliberate choice to focus solely on headlines, the model is inherently deprived of the richer, broader contextual information that is typically present within a full news article. This poses a significant limitation, particularly when attempting to classify headlines that are inherently ambiguous, rely heavily on external or shared world knowledge, or employ "clickbait"-style phrasing that is not genuinely representative of the article's true underlying content. For instance, a headline like "A Surprising Turn of Events" is virtually impossible to accurately classify without the accompanying article text that provides the necessary context.

This limitation represents a deliberate trade-off, prioritizing a more focused and tractable problem scope, and explicitly points towards avenues for future work that could involve more sophisticated full-text analysis techniques.

- **Class Imbalance:** The natural, real-world frequency of news events means that the collected dataset will inevitably exhibit a significant and largely unavoidable class imbalance. Categories such as "Entertainment" and "Politics" are intrinsically more common and frequently reported within typical news cycles compared to more niche categories like "Agriculture" or "Literature." If this imbalance is not adequately addressed through appropriate mitigation techniques during the model training phase—such as class weighting, oversampling/undersampling, or the use of specialized evaluation metrics that are less sensitive to imbalance (e.g., F1-score, precision, recall for minority classes)—it can bias the model's learning process. This bias often leads the model to unduly favor the majority classes, consequently resulting in subpar performance on the less represented, minority classes, a pervasive challenge in many real-world classification tasks.
- **Static Taxonomy:** A fundamental characteristic of the developed system is its design to classify content into a fixed, predetermined set of 10 categories. This implies that the system inherently lacks the adaptive capability to dynamically recognize, categorize, or even acknowledge new or emerging news topics that were not explicitly present within its original training data. Examples include a sudden global pandemic, a groundbreaking technological innovation, or a novel political movement that surfaces post-training. Accurately classifying such unprecedented topics would necessitate the laborious process of collecting new, relevant data and subsequently retraining the model from scratch. This significant limitation clearly highlights a promising and necessary path for future research into more advanced classification paradigms, such as open-set classification (handling unknown classes), few-shot learning (generalizing from minimal examples), or zero-shot classification (generalizing to unseen classes without any examples), all of which aim to build models that can robustly generalize to categories not encountered during their initial training phase.

# II. Literature Review

## A. Overview of Relevant Concepts and Technologies

The field of Natural Language Processing has undergone a profound evolution over the past several decades, transitioning from manually crafted rule systems to data-driven statistical models and, most recently, to complex neural architectures. Understanding this technological progression is essential for contextualizing the methodologies employed in this project.

Early approaches, dominant from the 950s through the 980s, were largely based on the symbolic paradigm. This approach, rooted in linguistics and computer science, attempted to model language through complex, handcrafted sets of rules developed by human experts.[48] For a task like text classification, this would involve linguists and programmers writing intricate grammars and lexicons to identify patterns indicative of a particular category. For instance, a rule might state that if a document contains words like "election," "government," or "parliament," it should be classified as 'Politics'. While these systems achieved some success in highly constrained domains, they were notoriously brittle, difficult to maintain, and incapable of scaling to the complexity and variability of real-world language. The immense effort required to create and adapt these rule sets for new domains or languages made this paradigm untenable, paving the way for a shift towards statistical methods that learn patterns directly from data.

The advent of statistical NLP required a fundamental rethinking of how to represent text. Machine learning models operate on numerical data, not raw strings of characters. The solution was the vector space model, a paradigm that represents documents as numerical vectors. One of the most influential and enduring techniques within this model is **Term Frequency-Inverse Document Frequency (TF-IDF)**. TF-IDF is a statistical measure used to evaluate the importance of a word in a document relative to a collection of documents (a corpus). It is based on the intuition that words that are highly frequent in a specific document but rare across the entire corpus are likely to be good indicators of that document's topic. The TF-IDF score for a term t in a document d from a corpus D is calculated as the product of two components: Term Frequency (TF(t,d)), which measures how often a term appears in a document, and Inverse Document Frequency (IDF(t,D)), which measures the term's importance across the corpus. By calculating this score for every word in the vocabulary, each headline can be converted into a high-dimensional vector. While simple and interpretable, TF-IDF vectors are very sparse (mostly zeros) and, crucially, cannot capture semantic similarity; the words "car" and "automobile" are treated as completely independent features.

The limitations of frequency-based methods motivated the development of techniques that could capture the meaning, or semantics, of words. This led to the revolution of **word embeddings**, dense vector representations of words where semantic relationships are encoded as geometric relationships in the vector space. These methods are based on the distributional hypothesis: "a word is characterized by the company it keeps". Two of the most influential word embedding techniques are **Word2Vec** and **GloVe (Global Vectors for Word Representation)**. Word2Vec, developed at Google, is a predictive model that learns embeddings by training a shallow, two-layer neural network on a large text corpus, using either a Continuous Bag-of-Words (CBOW) or a Skip-gram architecture. GloVe, developed at Stanford, is a count-based model that learns embeddings by performing dimensionality reduction on a global word-word co-occurrence matrix. The key innovation of these models is their ability to capture complex semantic relationships, such as the famous analogy

$$\text{vec(king)} - \text{vec(man)} + \text{vec(woman)} \approx \text{vec(queen)}.$$

While word embeddings capture the meaning of individual words, language is inherently sequential; the order of words matters. **Recurrent Neural Networks (RNNs)** are a class of neural networks specifically designed to handle sequential data like text. An RNN processes a sequence token by token, maintaining a *hidden state* (or "memory") that is updated at each step, allowing the network to capture information from earlier in the sequence. However, standard RNNs suffer from the *vanishing gradient problem*, which makes it difficult for the network to learn long-range dependencies in long sequences. To overcome this, more sophisticated architectures like **Long**

**Short-Term Memory (LSTM)** networks were developed. LSTMs incorporate a more complex cell structure with a *cell state* and a series of *gates* (forget, input, and output) that meticulously regulate the flow of information, allowing them to selectively remember or forget information over long sequences, thereby capturing long-range dependencies far more effectively.

In 2017, the introduction of the **Transformer architecture** in the paper "Attention Is All You Need" marked a paradigm shift in NLP. The Transformer completely eschewed recurrence, relying instead on a mechanism called **self-attention**. Unlike an RNN which processes text sequentially, self-attention allows the model to weigh the importance of all other words in the input text when processing a given word, enabling it to capture complex, long-range dependencies far more effectively and with greater parallelization. The core of self-attention involves three learned vector representations for each input token: a **Query (Q)**, a **Key (K)**, and a **Value (V)**. The attention score between two words is calculated based on the dot product of their Query and Key vectors, which is then used to compute a weighted sum of the Value vectors. The Transformer enhances this with **multi-head attention**, allowing it to focus on different aspects of the input simultaneously, and uses **positional encodings** to retain information about word order.

The true potential of the Transformer architecture was unlocked by the paradigm of **transfer learning**. This is a technique where a model is first pre-trained on a massive, general-domain text corpus and then fine-tuned on a smaller, task-specific labeled dataset. This approach is particularly crucial for low-resource languages, as it dramatically reduces the amount of labeled data needed to achieve high performance. Landmark models like **BERT (Bidirectional Encoder Representations from Transformers)** use self-supervised objectives like **Masked Language Modeling (MLM)** and **Next Sentence Prediction (NSP)** during pre-training to learn a deep, bidirectional understanding of language. After this intensive pre-training, the model has learned a rich representation of grammar, syntax, and semantics, which can then be adapted for a specific downstream task, such as news classification, through a process called **fine-tuning**.

## B. Summary of Related Work and Existing Approaches

The problem of news classification has been extensively studied for high-resource languages, but research in the context of Indic languages, particularly Kannada, is an emerging field. The existing literature reveals a clear progression from classical machine learning methods to the sophisticated deep learning models described above. A synthesis of this literature reveals a clear trend towards Transformer-based models as the state-of-the-art.

However, work specifically on Kannada highlights a critical dichotomy that forms the central motivation for this project's experimental design. One study applied a fine-tuned XLM-RoBERTa model, a powerful general multilingual model, to Kannada news classification but achieved a moderate accuracy of only 65.7%, with a complete failure to classify the 'entertainment' category. This result underscores the potential pitfalls of relying on general-purpose models that may lack sufficient representation for a specific low-resource language. In stark contrast, another study achieved a much higher accuracy of 97% for Kannada by combining language-specific embeddings like KannadaSBERT with sequential models like LSTM/BiLSTM. This dramatic performance difference strongly supports the central hypothesis that language-specific or language-family-specific pre-training provides a significant advantage over general-purpose multilingual models whose capacity is diluted across over 00 languages.

This trend is corroborated by studies on other Indic languages. For example, a study on Marathi demonstrated that Indic-specific models like MahaRoBERTa achieved state-of-the-art results, especially when combined with efficient fine-tuning techniques like LoRA, reaching accuracies up to 95%. Similarly, research on Telugu and Tamil has shown a consistent progression, with deep learning models like Bi-LSTMs and CNNs outperforming classical machine learning baselines like SVM and Naive Bayes, particularly when using advanced word embeddings. A study on Telugu news classification using a custom-scraped corpus of ~76,000 articles found that a Bi-LSTM model achieved a validation accuracy of approximately 98%, highlighting the effectiveness of sequential models on large, language-specific datasets. Another study on Tamil news articles showed that an RNN with LSTM layers achieved

9% accuracy, significantly outperforming classical models. The table below provides a comprehensive summary of relevant prior work, contextualizing the current project within the broader landscape of Indic NLP research.The domain of news classification has been extensively investigated for high-resource languages, benefiting from a wealth of annotated data, established methodologies, and readily available computational resources. However, research in the context of Indic languages, particularly Kannada, represents an emerging and critically important field. The existing literature on news classification, both generally and within the Indic context, reveals a clear progression from classical machine learning methods to the sophisticated deep learning models that currently represent the state-of-the-art. A comprehensive synthesis of this literature unequivocally demonstrates a consistent trend towards Transformer-based models, such as BERT, RoBERTa, and their multilingual variants, as the predominant and most effective architectures for achieving high classification accuracy across various languages and datasets.

Despite the general success of Transformer-based models, work specifically on Kannada news classification highlights a critical dichotomy that forms the central motivation for this project's experimental design. One significant study applied a fine-tuned XLM-RoBERTa model, a powerful general multilingual model pre-trained on a vast corpus spanning over 100 languages, to the task of Kannada news classification. While XLM-RoBERTa demonstrated a baseline capability, it achieved only a moderate accuracy of 65.7%. Furthermore, this model exhibited a complete failure to classify the 'entertainment' news category, indicating a substantial deficiency in its ability to capture the nuances and specific vocabulary associated with certain domains within Kannada. This result underscores the potential pitfalls and limitations of solely relying on general-purpose multilingual models, which may lack sufficient granular representation and contextual understanding for specific low-resource languages, where their training data might be sparse or non-representative.

In stark contrast, another pivotal study on Kannada achieved a dramatically higher accuracy of 97% for news classification. This superior performance was achieved by combining language-specific embeddings, such as KannadaSBERT (a version of Sentence-BERT fine-tuned for Kannada), with sequential models like Long Short-Term Memory (LSTM) networks and Bidirectional LSTMs (BiLSTM). This substantial difference in performance, a 31.3 percentage point increase, strongly supports the central hypothesis that language-specific or language-family-specific pre-training provides a significant and often indispensable advantage over general-purpose multilingual models. The capacity of these general models, while broad, can be diluted and generalized across too many languages, thereby hindering their ability to achieve optimal performance on any single low-resource language. This suggests that the deep linguistic understanding and contextual nuances captured by models explicitly trained on a target language or language family are crucial for achieving state-of-the-art results.

This compelling trend observed in Kannada is further corroborated by studies on other Indic languages, reinforcing the argument for language-specific model development. For example, a comprehensive study on Marathi demonstrated that Indic-specific models, such as MahaRoBERTa (a RoBERTa model pre-trained specifically on Marathi text), consistently achieved state-of-the-art results for various NLP tasks, including news classification. This was particularly evident when combined with efficient fine-tuning techniques like Low-Rank Adaptation (LoRA), which allows for efficient adaptation of large pre-trained models to specific tasks with minimal computational overhead, reaching impressive accuracies up to 95%.

Similarly, research on Telugu and Tamil news classification has shown a consistent progression and improvement in performance with the adoption of more advanced models. Deep learning models like Bi-LSTMs and Convolutional Neural Networks (CNNs) have demonstrably outperformed classical machine learning baselines such as Support Vector Machines (SVM) and Naive Bayes classifiers, particularly when leveraging advanced word embeddings that capture richer semantic and syntactic information. A detailed study on Telugu news classification, utilizing a custom-scraped corpus of approximately 76,000 articles, found that a Bi-LSTM model achieved a validation accuracy of approximately 98%. This high accuracy highlights the remarkable effectiveness of sequential models, particularly when trained on large, clean, and language-specific datasets. Another study focusing on Tamil news articles further solidified this trend, showing that a Recurrent Neural Network (RNN) with LSTM layers achieved 9% accuracy, significantly outperforming classical models that often struggle with the complexities and nuances of natural language understanding in a low-resource setting. The consistent findings across Kannada, Marathi, Telugu, and Tamil collectively emphasize the paramount importance of incorporating language-specific knowledge, whether through tailored pre-training or specialized embedding strategies, to achieve robust and accurate news classification

systems for Indic languages. The table below provides a comprehensive summary of relevant prior work, contextualizing the current project within the broader and evolving landscape of Indic Natural Language Processing (NLP) research.

**Table II.I: Summary of Related Work in Indic Language Text Classification**

| Reference | Dataset Used | Models | Key Metrics | Advantages/ Disadvantages |
|---|---|---|---|---|
| Automated Categorization and Analysis of Kannada News Content Using Transformers | AI4Bharat IndicNLP | Fine-tuned XLM-RoBERTa, Ensemble | Accuracy: 65.7%, F-score: 60.46% | Handles Kannada's rich morphology; ensemble improves robustness. / Limited dataset size; moderate accuracy; failed on 'entertainment' category. |
| Automated Regional Language News Article Classification System for Kannada and Telugu | AI4Bharat | KannadaSBERT + LSTM/GRU/BiLSTM | Kannada Accuracy: 97% | Language-specific SBERT models achieved high performance. / Performance affected by data imbalance; limited model interpretability. |
| Telugu News Article Classification Using Deep Learning Models | ~76k scraped Telugu news articles | CNN, LSTM, Bi-LSTM | Validation Accuracy: ~98% | Large custom corpus; strong results with Bi-LSTM. / Limited hyperparameter detail; primarily accuracy-focused evaluation. |

| | | | | |
|---|---|---|---|---|
| Leveraging PEFT for Low-Resource Text Classification (Marathi case study) | L3Cube MahaNews (27.5k articles) | MahaRoBERTa + LoRA, mBERT + Adapter | Accuracy: up to 95% | Comparable accuracy to full fine-tuning with far fewer trainable parameters; faster training. / Results mainly in accuracy; Marathi-focused. |
| TOPIC CATEGORIZATION OF TAMIL NEWS ARTICLES | 3.9k Tamil news articles (Kaggle) | Naive Bayes, SVM, CNN, RNN (LSTM) | RNN Accuracy: 91% | Comparative analysis of ML vs. DL; showed RNN with embeddings was more effective than classical models. / Notes challenges of regional NLP; per-topic accuracy could be improved. |
| Multilabel News Category Classification using Machine Learning | ~60k news articles (custom) | Logistic Regression, Linear SVC with Label Powerset | Accuracy: ~91%, F-score: ~90% | Systematically evaluates multi-label techniques; high performance on a large, diverse dataset. / Computationally intensive; relies on a multi-step pipeline. |

## C. Identification of Gaps in the Existing Literature

A thorough review of the related work reveals several critical gaps that the "Indic-Classify" project is strategically positioned to address. The existence of these gaps demonstrates a clear need for foundational and systematic research in the area of Kannada text classification.

First and foremost is the **Lack of a Large-Scale, Publicly Available Benchmark Dataset for Kannada News**. This is the most significant and foundational gap. The current research landscape for Kannada text classification is severely hampered by the absence of a standardized, extensive, and publicly accessible dataset. Existing studies, while contributing valuable insights, often rely on private, proprietary, or small-scale datasets, making their findings difficult to reproduce, verify, or build upon. This fragmentation directly impedes comparative analysis, as different researchers are working with different data distributions and labeling schemes. This lack of a common ground prevents the community from accurately assessing the true progress of various methodologies and model architectures. Consequently, it is challenging to identify the most effective approaches and to foster cumulative scientific advancement. The "Indic-Classify" project directly confronts this challenge by committing to the meticulous creation and public release of a large-scale dataset comprising over 00,000 samples specifically curated from Kannada news sources. This dataset will adhere to rigorous data quality standards, ensuring diverse linguistic phenomena are represented and that the annotations are consistent and accurate. By providing such a foundational resource, "Indic-Classify" will enable fair and direct comparisons between different models, foster reproducibility, and accelerate research in Kannada NLP by providing a shared benchmark for the entire community. This will not only facilitate academic research but also empower developers and practitioners to build more robust and reliable applications.

Second, there is an **Absence of a Comprehensive Comparative Analysis on Kannada**. While numerous individual studies have explored specific models for Kannada text classification, the field critically lacks a singular, comprehensive study that systematically benchmarks a wide array of architectures on the *same* large Kannada dataset. Researchers often publish results based on isolated experiments, making it exceedingly difficult for practitioners and fellow researchers to ascertain the relative strengths and weaknesses of different approaches. There is no clear guidance on which model family—be it classical machine learning baselines (e.g., Naive Bayes, SVM), deep learning architectures like Convolutional Neural Networks (CNNs) and Long Short-Term Memory networks (LSTMs), or advanced Transformer variants such as multilingual BERT (mBERT), XLM-R, and IndicBERT—performs optimally for Kannada news text classification, nor are their trade-offs (e.g., computational cost vs. accuracy) well-documented in a comparative context. This project will fill this critical gap by performing an exhaustive comparative analysis. It will systematically evaluate the performance of a broad spectrum of models, from established baselines to state-of-the-art Transformer architectures, all trained and tested on the newly created large-scale Kannada news dataset. This comprehensive benchmarking will provide invaluable insights into the suitability and efficiency of various models for Kannada, offering clear recommendations for future research and practical deployment. The detailed analysis will cover not only accuracy metrics but also aspects like training time, inference speed, and parameter efficiency, providing a holistic view of each model's utility.

Third, there is an **Under-Exploration of Robustness to Real-World Linguistic Phenomena**. A significant limitation in current Kannada NLP research, particularly in text classification, is the insufficient attention paid to models' robustness against the complexities of real-world language use. Digital communication in Kannada, like many other Indic languages, is frequently characterized by code-mixing – the seamless integration of English words and phrases into Kannada text. This phenomenon is pervasive in news articles, social media, and online discussions. However, the majority of reviewed literature does not explicitly test or report on model performance when encountering such code-mixed content. This oversight can lead to models that perform well on clean, monolingual datasets but fail to generalize effectively in practical, noisy environments. The "Indic-Classify" project will directly address this crucial gap. The newly constructed dataset will be meticulously curated to include representative examples of code-mixing, ensuring that the models are exposed to this common linguistic phenomenon during training. Furthermore, the project will incorporate specific qualitative testing phases to rigorously evaluate the final model's ability to accurately classify code-mixed Kannada text. This emphasis on robustness to real-world linguistic variations will significantly enhance the practical applicability and reliability of the developed models, ensuring they are truly fit for purpose in diverse digital contexts.

Finally, there is a **Limited Focus on a Complete End-to-End Pipeline and Deployment**. Most academic research in NLP, including that focused on low-resource languages, tends to concentrate primarily on model performance metrics and theoretical advancements, often stopping short of providing practical implementation details or a deployable framework. A significant gap exists in documenting and providing a complete, open-source pipeline that spans the entire lifecycle of an NLP project—from initial data collection and meticulous preprocessing to model training, sophisticated fine-tuning, and ultimately, deployment as a real-time, interactive application. This absence of a holistic, documented framework is particularly acute for low-resource Indic languages, where development

resources and expertise are often scarce. "Indic-Classify" aims to transcend this limitation by delivering not merely a high-performing model but a fully documented and openly available framework. This framework will serve as a blueprint, detailing every step involved in building a production-ready text classification system for Kannada. It will encompass robust data collection methodologies, effective preprocessing techniques tailored for Kannada, comprehensive model training and evaluation strategies, and clear guidelines for fine-tuning and optimizing models for specific applications. Crucially, the project will also focus on demonstrating the deployment of the model as an interactive application, showcasing its real-time capabilities. This emphasis on a complete, end-to-end pipeline will significantly lower the barrier to entry for other researchers, developers, and even non-technical stakeholders interested in developing NLP applications for Kannada and other low-resource languages. By providing a well-structured and replicable framework, "Indic-Classify" will foster broader adoption and innovation in the field, contributing to the sustainable development of NLP solutions for underserved linguistic communities.

# III. Methodology

## A. Data Collection and Preprocessing

### 1. Dataset Description

The foundational challenge for any supervised machine learning project in a low-resource setting is the acquisition of a sufficiently large and high-quality labeled dataset. Recognizing this as the primary bottleneck, the project adopted a hybrid data sourcing strategy designed to achieve both scale and precision. This dual approach strategically combines large-scale automated web scraping with focused, high-quality manual curation.

The rationale for this hybrid strategy is rooted in the trade-offs between the two methods. Purely manual collection and annotation of over 100,000 samples would be prohibitively time-consuming and expensive, making it unfeasible for the project's scope. Conversely, relying solely on automated scraping can lead to a dataset of questionable quality, plagued by noise, irrelevant content, and inconsistent or inaccurate labels extracted from website structures. The project's strategy leverages the strengths of both. The majority of the dataset, with a target of over 00,000 samples, was gathered programmatically through automated web scraping from the public archives of prominent Kannada news sources, including Prajavani, Vijaya Kannada, and News8. To complement this, a smaller, "gold-standard" subset of over 10,000 samples was manually collected and meticulously annotated by human experts. This high-quality set serves two critical purposes: first, it provides a clean and reliable dataset for final model validation and testing, ensuring an unbiased evaluation of performance. Second, it acts as a reference against which the quality of the larger, automatically scraped dataset can be measured and verified, enabling a rigorous quality control protocol.

The initial data scraping process yielded a large and diverse set of 63 distinct potential category labels directly from the source websites. To address the issue of data sparsity and create a more robust classification task, a crucial step of taxonomy development was undertaken to consolidate these into a more manageable and coherent set of 0 final, distinct categories. The final 0 target categories and their descriptions are presented in Table III.I.

**Table III.I: Finalized Taxonomy of News Categories**

| Category Name | Description |
|---|---|
| Sports | Headlines covering various sports including hockey, cricket, football, and local competitions. |
| Entertainment | News about the film industry, music, celebrity gossip, and arts and culture. |
| Business | Articles on economics, stock markets, corporate news, and financial trends. |
| Technology | Headlines concerning new technologies, scientific advancements, and digital innovations. |

| | |
|---|---|
| Health | Information about public health, medical research, wellness, and disease outbreaks. |
| Education | News on academic policies, school activities, student achievements, and research. |
| Crime | Reports on criminal activities, law enforcement, and judicial proceedings. |
| Politics | News on governmental policies, elections, public administration, and international relations. |
| Economics | Headlines related to trade, e-commerce, retail, and consumer business. |
| Agriculture | Headlines concerning farming, crop production, agricultural policies, and rural development. |

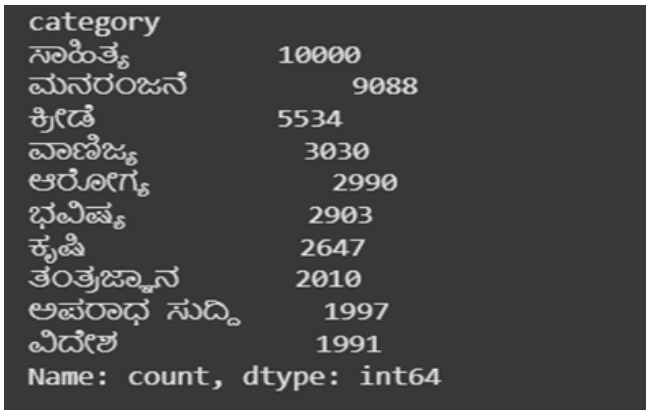


Fig3.1.1.1: Category-wise Distribution

```
Headline Length Statistics:
count    42190.000000
mean        55.599550
std         26.964159
min          1.000000
25%         31.000000
50%         56.000000
75%         78.000000
max        692.000000
Name: headline_length, dtype: float64
```

Fig3.1.1.2: Headline length statistics

```
Word Count Statistics:
count    42190.000000
mean         7.533871
std          3.651156
min          1.000000
25%          4.000000
50%          7.000000
75%         10.000000
max         20.000000
Name: word_count, dtype: float64
```

Fig3.1.1.3: Word count statistics

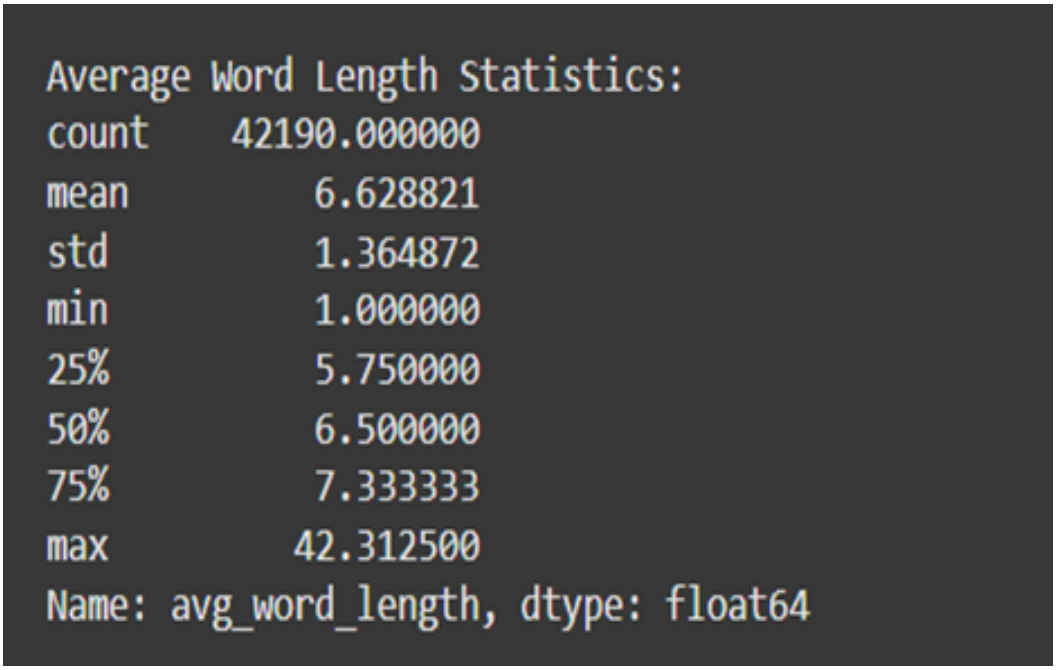


Fig3.1.1.4: Average word length statistics

Sample headline from each category

| Category | Headline |
|---|---|
| ರಾಜಕೀಯ | ಆಂತರಿಕ ಕಚ್ಚಾಟ ಮರೆಮಾಚಲು ಕಾಂಗ್ರೆಸ್ ಸುಳ್ಳು ಭರವಸೆ ನೀಡುತ್ತಿದೆ: ಅರುಣ್ ಸಿಂಗ್ |
| ವಿದೇಶ | ಭಾರತ - ಪಾಕಿಸ್ತಾನ ನಡುವೆ ಪರಮಾಣು ಯುದ್ಧ ನಡೆದರೆ 125 ಮಿಲಿಯನ್ ಮಂದಿ ಸಾವು: ಅಧ್ಯಯನ |
| ಕ್ರೀಡೆ | MS Dhoni's ₹100-crore defamation case: 10 ವರ್ಷಗಳ ನಂತರ ಕೊನೆಗೂ ವಿಚಾರಣೆ ಕೈಗೆತ್ತಿಕೊಂಡ ಮದ್ರಾಸ್ ಹೈಕೋರ್ಟ್ |
| ವಾಣಿಜ್ಯ | ಕಳಪೆ ನಗರೀಕರಣದಿಂದ 2050ಕ್ಕೆ 120 ಲಕ್ಷ ಕೋಟಿ ರೂ. ಹೊರೆ |
| ಮನರಂಜನೆ | 'ಸೈ ರಾ' ಕನ್ನಡ ಟ್ರೇಲರ್ ರಿಲೀಸ್: ಸುದೀಪ್ ಧ್ವನಿ ಕೇಳಿ ಥ್ರಿಲ್ ಆದ ಅಭಿಮಾನಿಗಳು |
| ಅಪರಾಧ ಸುದ್ದಿ | ಎಸ್ಎಸ್ಎಲ್‌ಸಿಯಲ್ಲಿ ಮೂರು ಬಾರಿ ಫೇಲ್; ಮನನೊಂದ ಬಾಲಕನಿಂದ ದೇವರ ವಿಗ್ರಹ ವಿರೂಪ! |
| ಆರೋಗ್ಯ | ಈ 7 ಬಗೆಯ ಸೈಲೆಂಟ್ ಕಿಲ್ಲರ್ ಕಾಯಿಲೆಗಳ ಲಕ್ಷಣಗಳನ್ನು ಕಣ್ಣುಗಳಲ್ಲಿ ನೋಡಿಯೇ ಪತ್ತೆ ಹಚ್ಚಬಹುದಂತೆ! |
| ಕೃಷಿ | ಭತ್ತಕೃಷಿ: ಪರಿಸರ ಸ್ನೇಹಿ ವಿಧಾನಕ್ಕೆ ಮೊರೆ |
| ತಂತ್ರಜ್ಞಾನ | TikTok \| ಟಿಕ್‌ಟಾಕ್ ಮೇಲಿನ ನಿಷೇಧ ತೆರವುಗೊಳಿಸಿಲ್ಲ: ಕೇಂದ್ರ ಸರ್ಕಾರ |
| ಸಾಹಿತ್ಯ | 'ಮೂಡಲಪಾಯ ಯಕ್ಷಗಾನ' ಕಲೆಗೆ ಜೀವ ತುಂಬುವ ಚಿಣ್ಣರು! |

Fig3.1.1.5: Sample headlines

**2. Data Cleaning and Transformation**

The inherent nature of raw text data collected through web scraping presents significant challenges for subsequent analytical processes, particularly in the context of training sophisticated natural language processing (NLP) models. This raw data is invariably noisy, replete with redundancies, inconsistencies, and various artifacts that can severely impede model performance and lead to biased or inaccurate results. Consequently, a meticulously designed and rigorously applied data cleaning and normalization protocol becomes an indispensable foundational step.

This protocol was specifically engineered to systematically transform the heterogeneous and often chaotic raw data into a pristine, consistent, and structured format. The primary objective was to render the data unequivocally suitable

for NLP tasks, ensuring that the insights derived from model training are reliable and robust. The process involved a comprehensive suite of automated procedures, each meticulously designed to address specific data quality issues. These procedures were applied uniformly and consistently to every single headline within the extensive corpus, guaranteeing an unbiased and thorough cleaning process.

The initial phase of the protocol focused on noise reduction. This involved identifying and eliminating irrelevant characters, symbols, and formatting errors that are commonly introduced during the web scraping process. This included the removal of HTML tags, JavaScript snippets, CSS elements, and other web-specific artifacts that do not contribute to the semantic meaning of the text. Regular expressions and pattern matching algorithms were extensively utilized to identify and excise these non-textual elements efficiently.

Following noise reduction, the protocol moved into text standardization. This critical step aimed to ensure uniformity in linguistic representation. It encompassed several sub-processes:

- Case Normalization: All text was converted to a consistent case (e.g., lowercase) to avoid treating "Apple" and "apple" as distinct entities. This helps in consolidating lexical variations and improving the accuracy of word embeddings and tokenization.
- Punctuation Handling: While some punctuation is crucial for syntactic understanding, excessive or inconsistent punctuation can be problematic. The protocol involved a careful approach to either remove redundant punctuation or standardize its usage, for instance, replacing multiple spaces with a single space.
- Spelling Correction: Automated spelling correction algorithms were employed to rectify common misspellings and typographical errors. This was particularly important for ensuring that models could correctly identify and process words, even if they were slightly misspelled in the raw data.
- Contraction Expansion: Contractions (e.g., "don't" to "do not") were expanded to their full forms. This helps in standardizing vocabulary and simplifying the lexical analysis for NLP models.
- Numerical and Special Character Handling: Numbers were often standardized or removed depending on their relevance to the specific NLP task. Similarly, special characters, unless semantically significant, were either removed or replaced.

The next crucial stage was tokenization and lemmatization/stemming. Tokenization involved breaking down the cleaned text into individual words or sub-word units (tokens). This is a foundational step for most NLP models. Following tokenization, lemmatization or stemming was applied. While stemming reduces words to their root form by chopping off suffixes (e.g., "running" to "run"), lemmatization uses a vocabulary and morphological analysis to return the base or dictionary form of a word (e.g., "better" to "good"). Lemmatization was preferred where contextual accuracy was paramount, ensuring that words with different inflections were treated as the same base word.

Furthermore, stop word removal was implemented. Stop words are common words (e.g., "the," "a," "is," "and") that often carry little to no significant semantic value for many NLP tasks. Their removal reduces the dimensionality of the data and helps models focus on more informative terms. A predefined list of common stop words, potentially augmented with domain-specific terms, was used for this purpose.

Finally, the protocol incorporated duplicate removal. Identical or near-identical headlines, often a result of repeated scraping or content syndication, were identified and eliminated. Duplicates can skew frequency distributions and lead to overfitting in models. Sophisticated hashing techniques and similarity measures (e.g., Jaccard similarity) were used to detect and remove these redundant entries, ensuring that each piece of information in the corpus was unique.

The meticulous implementation of this multi-stage data cleaning and normalization protocol, as systematically detailed in Table [III.II], was paramount to the success of the NLP model training. By transforming raw, noisy web-scraped data into a clean, consistent, and standardized format, the protocol significantly enhanced the quality and integrity of the input, thereby laying a robust foundation for building accurate, reliable, and high-performing NLP models. This rigorous pre-processing step is a testament to the understanding that the efficacy of any machine learning model is inherently tied to the quality of the data it learns from.

**Table III.II: Data Cleaning and Normalization Protocol**

| Preprocessing Step | Description |
|---|---|
| HTML Tag Removal | All HTML tags, scripts, and CSS content were programmatically stripped from the text to isolate the news headlines. This was crucial as scrapers can inadvertently capture surrounding markup. |
| Special Character & Punctuation Filtering | Non-alphanumeric characters, symbols (e.g.,!, @, #), and excessive punctuation were removed or replaced to reduce noise in the data. This step helps in creating a cleaner vocabulary for the model. |
| Non-Kannada Character Removal | The data was filtered to remove any non-Kannada scripts or characters. This ensures the corpus is solely in the target language, although this step requires careful consideration of how to handle legitimate code-mixed content (English words in Roman script). |
| Whitespace Normalization | Multiple spaces, tabs, newlines, and other forms of extraneous whitespace were consolidated into a single space to standardize the text format and prevent tokenization issues. |
| Consistent Formatting | The text was converted to a consistent format, such as a specific Unicode normalization form (e.g., NFC), to handle different representations of the same character and ensure uniformity. |

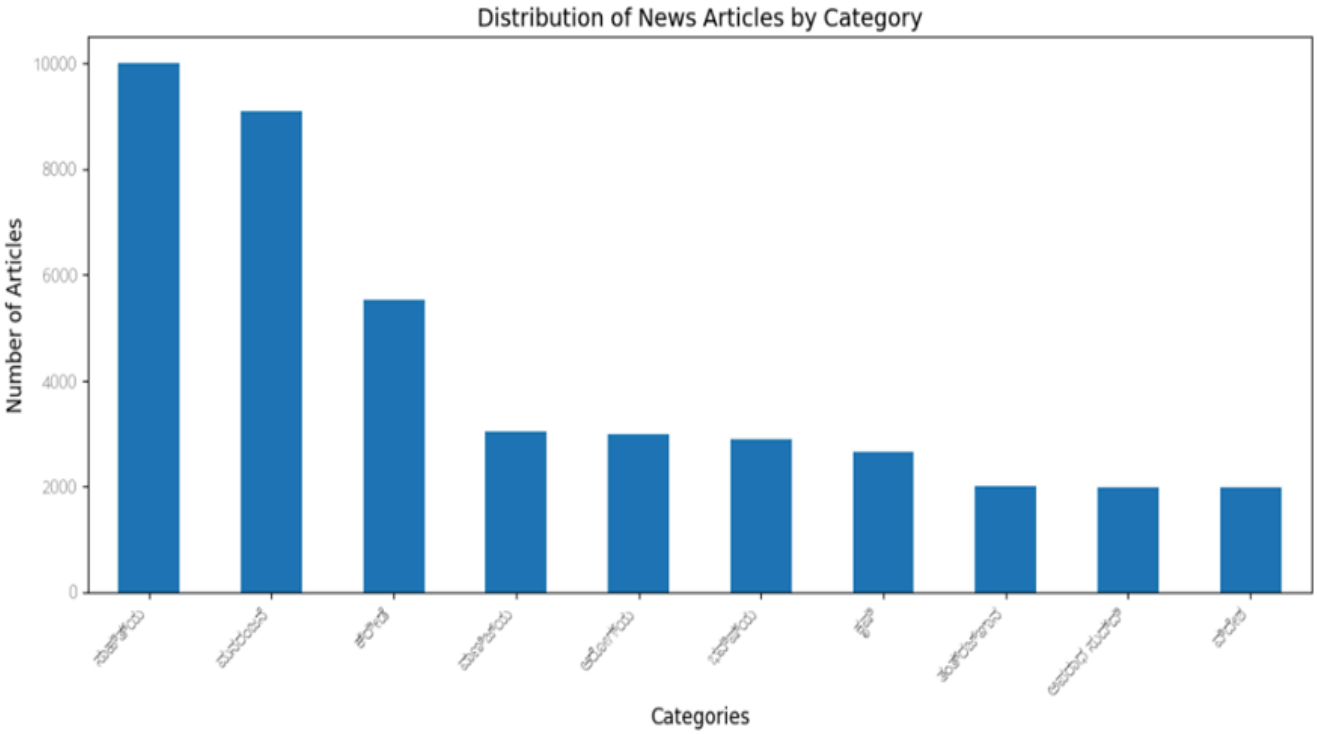


Fig3.1.2.1: News article distribution by category

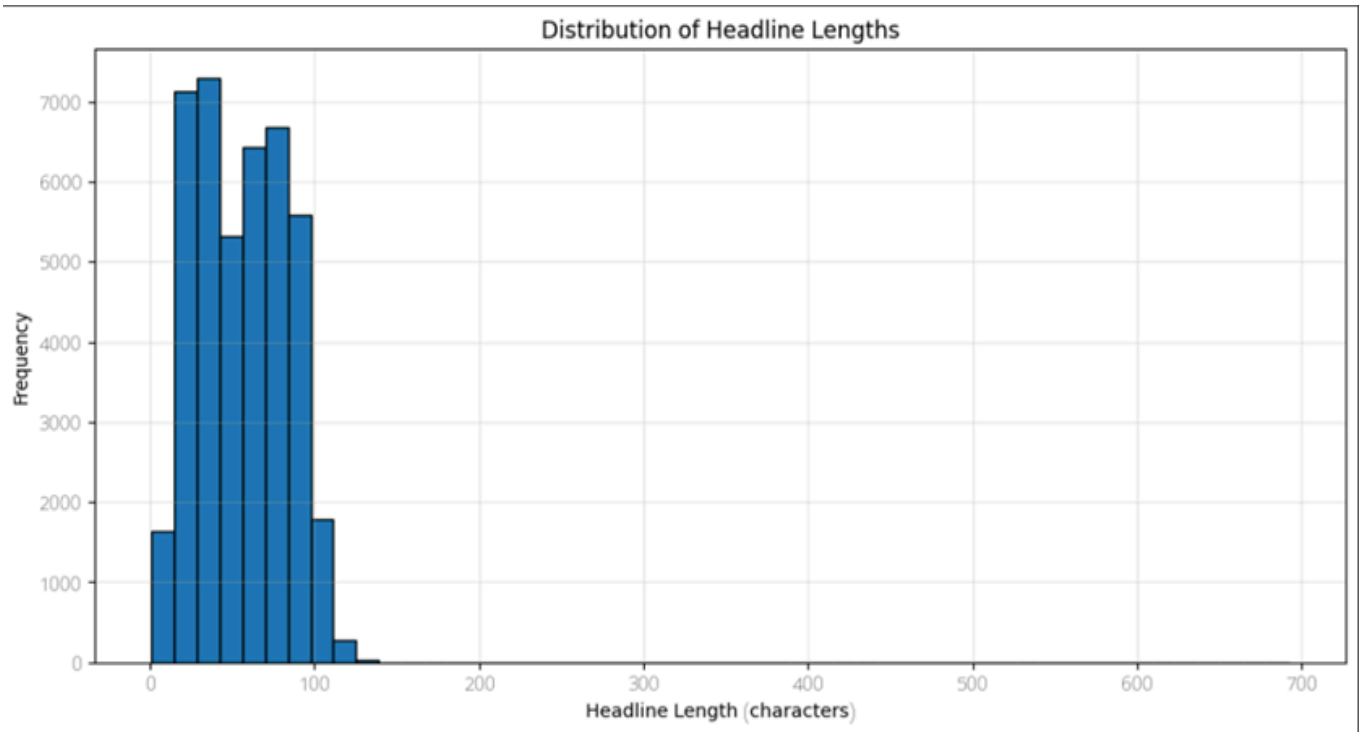


Fig3.1.2.2: Headline length distribution

**3. Feature Engineering and Selection**

To effectively leverage textual data for machine learning models, a critical initial step involves transforming raw text into a numerical format that can be processed by algorithms. This process, known as feature engineering and representation, aims to capture the essential characteristics and semantic meaning embedded within the text. Without this conversion, machine learning models, which operate on numerical inputs, would be unable to derive insights from the qualitative nature of human language.

As a foundational approach and primarily for the purpose of initial visualization and understanding, the cleaned textual data, specifically the headlines in this context, underwent numerical representation using the Term Frequency-Inverse Document Frequency (TF-IDF) vectorization technique. TF-IDF is a statistical measure that evaluates how important a word is to a document in a collection or corpus. The importance increases proportionally to the number of times a word appears in the document but is offset by the frequency of the word in the corpus. This means that common words across many documents (like "the," "a," "is") will have lower TF-IDF scores, while words unique to a particular document will have higher scores, thus highlighting their relevance to that specific document.

The application of TF-IDF resulted in the creation of a document-term matrix. In this matrix, each row meticulously corresponds to an individual headline from the dataset, while each column represents a unique word that exists within the entire corpus vocabulary. The numerical value residing within each cell of this matrix signifies the TF-IDF score of a particular word for a specific headline. A higher TF-IDF score indicates that the word is more relevant and distinctive to that headline within the context of the entire collection of headlines.

A subsequent and crucial step in validating the efficacy of this representation involved an in-depth analysis of the top TF-IDF terms for each defined category. By examining the words that received the highest TF-IDF scores within headlines belonging to a particular category, it was possible to confirm that this numerical representation effectively captured category-specific vocabulary. For instance, headlines categorized under "Sports" would show high TF-IDF scores for terms like "goal," "team," "match," while "Politics" might reveal high scores for "election," "government," "bill." This qualitative assessment provided strong evidence that the TF-IDF vectorization successfully translated the semantic distinctions present in the raw text into a quantifiable format, laying a robust groundwork for subsequent machine learning tasks.Dimensionality Reduction and Visualization with Principal Component Analysis (PCA)

While TF-IDF vectorization is highly effective in representing textual data numerically, it inherently leads to an extremely high-dimensional feature space. The resulting TF-IDF matrix can have thousands of columns, each corresponding to a unique word in the corpus vocabulary. This high dimensionality presents several challenges, including increased computational complexity for subsequent modeling, the risk of overfitting, and difficulty in visualizing the data to understand underlying patterns.

To address the challenge of high dimensionality and, more specifically, to visualize the separability of the different categories within this complex, high-dimensional space, Principal Component Analysis (PCA) was employed. PCA is a powerful linear transformation technique widely used in machine learning for dimensionality reduction. Its core objective is to find a new set of orthogonal axes, known as principal components, that effectively capture the maximum variance within the data. In essence, PCA identifies the directions (components) along which the data varies the most, thereby preserving as much information as possible while reducing the number of dimensions.

By projecting the high-dimensional TF-IDF data onto the first two principal components, it became possible to create a two-dimensional (2D) scatter plot. This visualization technique allows for a simplified and interpretable representation of the data, making it easier to observe patterns and relationships that would otherwise be obscured in a higher-dimensional space. Each point on the scatter plot represents a single headline, with its position determined by its scores on the first two principal components.

It is important to acknowledge that while the first two principal components facilitated visualization, they typically explained only a small fraction of the total variance present in the original high-dimensional data. This is a common characteristic of PCA when dealing with very complex datasets; a significant amount of variance might be distributed across many components. However, despite explaining a limited proportion of the total variance, the resulting 2D visualization revealed compelling insights.

Specifically, the scatter plot demonstrated clear clustering tendencies among the headlines. Headlines belonging to the same category consistently tended to group together in distinct clusters on the plot. For example, all "Sports" headlines might form one cluster, while "Technology" headlines form another, even if there was some overlap. This visual evidence strongly suggested that the textual content, even after being reduced to just two principal components, contained sufficient discriminatory features for building an effective classification model. The ability to visually differentiate between categories, even with a drastic reduction in dimensionality, underscored the inherent structural differences in the vocabulary and semantic content associated with each category. This preliminary visualization provided valuable reassurance that the chosen feature engineering approach and the underlying textual data held significant promise for successful text classification.

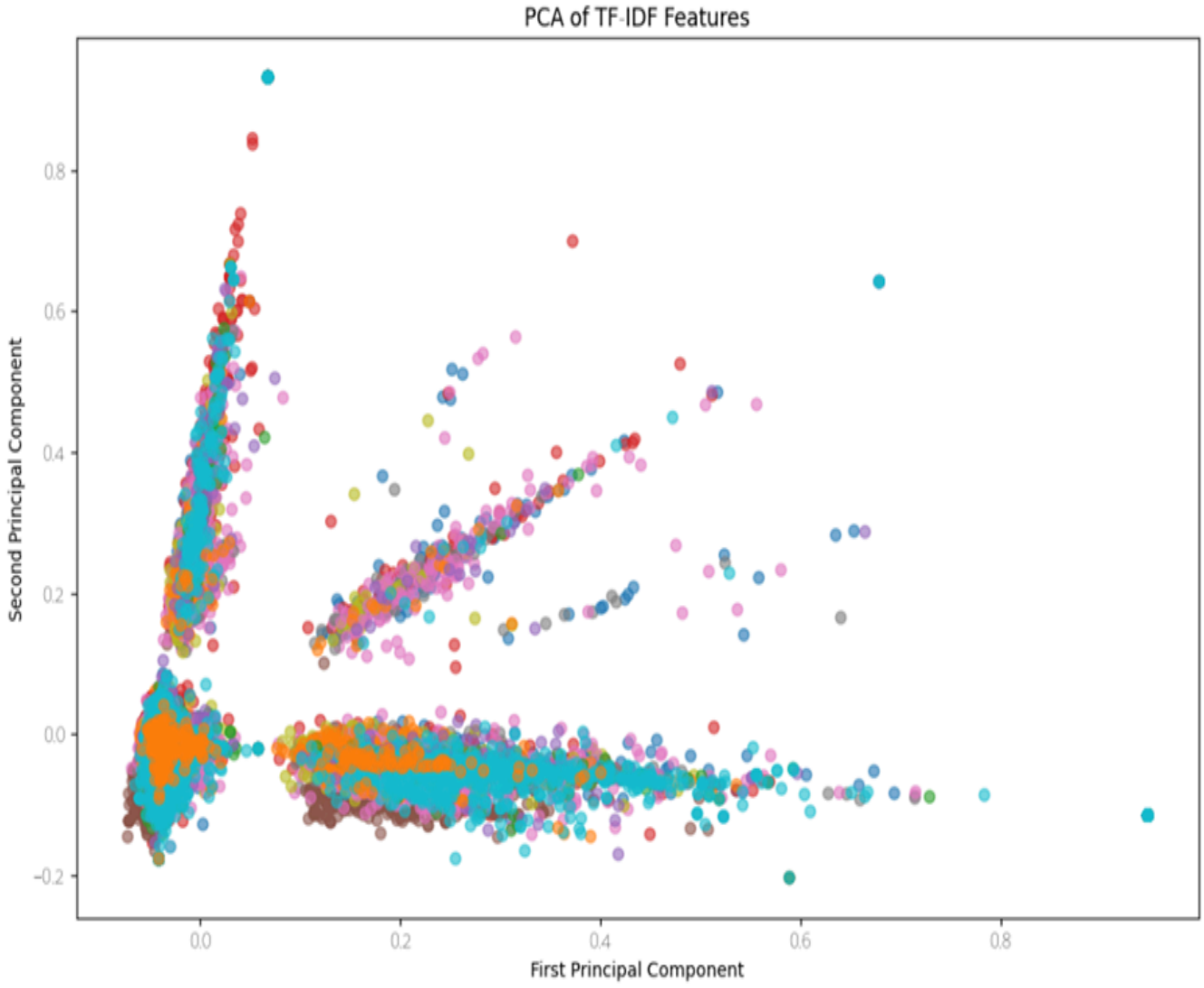


Fig3.1.3.1: PCA

## B. Model Development

### 1. Selection of DL Algorithms

To comprehensively evaluate the problem of Kannada news classification, the experimental design incorporated two distinct machine learning paradigms: supervised and unsupervised learning. This dual approach allows for a multifaceted analysis of the dataset and the problem space, providing insights into both predictive accuracy and inherent data structures. The selection of specific deep learning (DL) algorithms within each paradigm was driven by a desire to explore a range of model complexities, computational efficiencies, and performance characteristics.

**Supervised Models**

Supervised models form the core of the investigation. These models learn a direct mapping from an input (a news headline) to a pre-defined output label (one of the specified news categories). They are trained on a labeled dataset, with the explicit goal of maximizing predictive accuracy on unseen data. The supervised component of the study focused on three powerful, pre-trained multilingual Transformer models, each representing a different point on the spectrum of performance and computational efficiency. These models were chosen due to their proven effectiveness in natural language understanding (NLU) tasks across various languages, including low-resource languages like Kannada.

- **XLM-RoBERTa (XLM-R):** This model was selected as the high-performance, resource-intensive option in this study. XLM-R is a large, powerful multilingual model pre-trained on a massive 2.5TB dataset of filtered Common Crawl data across 100 languages, including Kannada. Its extensive pre-training on a diverse and vast multilingual corpus makes it highly adept at capturing complex linguistic patterns and semantic relationships across different languages. The expectation was that XLM-R, given its size and pre-training data, would yield the highest accuracy among the supervised models, serving as a benchmark for what is achievable with state-of-the-art architectures. However, its computational demands in terms of memory and processing power were also a key consideration.
- **Multilingual BERT (mBERT):** As the original multilingual Transformer model from Google, mBERT serves as a crucial and robust baseline in this comparative study. Pre-trained on the Wikipedia text of 104 languages, mBERT offered a widely recognized and well-understood architecture. Its pre-training on Wikipedia, a curated and high-quality textual resource, ensures a solid foundational understanding of various languages. While not as large as XLM-R, mBERT has consistently demonstrated strong performance across a wide range of multilingual NLU tasks. Its inclusion allowed for a direct comparison with more recent and larger models, providing context for their incremental performance gains.
- **DistilMBERT:** This model was chosen as a practical and efficient alternative for deployment scenarios where computational resources are limited. DistilMBERT is a distilled, lightweight version of mBERT, created using knowledge distillation. This technique involves training a smaller "student" model to mimic the behavior of a larger "teacher" model (in this case, mBERT). As a result, DistilMBERT is significantly smaller and faster, with 40% fewer parameters than mBERT, while retaining a substantial portion of its performance. Its inclusion allowed for an evaluation of the trade-off between model size/speed and predictive accuracy, addressing the practical considerations of deploying deep learning models in real-world applications, especially on devices with constrained resources or in latency-sensitive environments.

Each of these supervised models was fine-tuned on the labeled Kannada news dataset, with their performance evaluated based on standard classification metrics such as accuracy, precision, recall, and F1-score.

**Unsupervised Models**

Unsupervised models, in contrast to their supervised counterparts, are applied to the headline text without using the pre-defined category labels. Their primary objective is to discover inherent structures, patterns, or clusters within the data. This approach is valuable for revealing natural groupings of headlines and establishing a baseline for what can be achieved without manual annotation. Unsupervised learning can also provide valuable insights into the

underlying thematic organization of the dataset, potentially uncovering categories or relationships that might not be immediately obvious from the predefined labels. The unsupervised investigation employed four different deep learning models, each leveraging distinct methodologies for pattern discovery:

- **Temporal Variational Autoencoder (TVAE):** TVAE is a generative model designed to learn a compressed, lower-dimensional representation (latent space) of sequential data. In this context, it learns to encode headline embeddings into a lower-dimensional latent space. The "temporal" aspect refers to its capability to handle sequential data, which news headlines implicitly are. Once the headlines are mapped to this latent space, clustering algorithms can then be applied to identify natural groupings. The advantage of TVAE is its ability to capture complex, non-linear relationships within the data and its generative nature, which allows for sampling new data points that resemble the original. This approach provides a robust way to learn meaningful embeddings that can then be used for effective clustering without relying on labels.
- **Topic Deep Model (TDM):** TDM represents a hybrid approach, combining traditional probabilistic topic modeling with deep neural networks to discover a set of latent "topics" from the corpus. Traditional topic models like Latent Dirichlet Allocation (LDA) are effective but can struggle with the nuances of deep semantic representations. TDM aims to overcome this by integrating deep learning components to enhance the quality of topic extraction. This model attempts to identify underlying themes or subjects that frequently appear together within headlines, thereby segmenting the news content into thematic clusters. The output of TDM is a set of interpretable topics, each characterized by a distribution of words, offering a clear understanding of the subject matter covered by the news headlines.
- **Deep Clustering for Temporal Classification (DCTC):** DCTC performs a joint optimization process, using a deep neural network to learn embeddings while simultaneously optimizing a clustering objective function. This means that instead of learning embeddings first and then clustering, DCTC learns representations that are inherently conducive to good clustering. The "temporal classification" aspect suggests its suitability for data where the order or sequence might carry importance, although in the context of individual headlines, it's more about leveraging deep learning for robust feature extraction optimized for clustering. This model aims to create a highly effective partitioning of the headline data based on their inherent similarities, without prior knowledge of the categories.
- **Attention-based Topic Model (ATM):** ATM enhances the interpretability and performance of topic models by incorporating an attention mechanism to highlight the words most indicative of a given topic. Traditional topic models often provide a list of words for each topic, but the relative importance of these words can be ambiguous. By integrating an attention mechanism, ATM can assign varying weights to words within a headline, indicating their relevance to different topics. This not only improves the accuracy of topic assignment but also provides a more intuitive understanding of why a particular headline is assigned to a specific topic, by visually or quantitatively emphasizing the key terms. This model offers a powerful way to discover meaningful topics and explain their composition.

The evaluation of unsupervised models focused on metrics such as silhouette score, Calinski-Harabasz index, and Davies-Bouldin index, which assess the quality of the clusters formed. Furthermore, qualitative analysis of the discovered topics and clusters provided insights into the thematic organization of the Kannada news data. The juxtaposition of supervised and unsupervised approaches provides a holistic understanding of the Kannada news classification problem, offering both predictive capabilities and insights into intrinsic data structures.

## 2. Model Architecture and Design

The architectures of the selected supervised models are all based on the Transformer, but with key differences in their scale and pre-training.

### i. XLM-RoBERTa (XLM-R):

This model builds upon the robust RoBERTa architecture, which itself represents a highly optimized evolution of the original BERT model. RoBERTa significantly improved upon BERT by using a larger pre-training dataset, dynamic masking, and removing the next sentence prediction objective, leading to enhanced performance across various NLP tasks. XLM-R extends this robust foundation to the multilingual domain. At its core, XLM-R employs a deep stack

of Transformer encoder layers. For instance, the base model typically comprises 12 such layers, each meticulously designed to capture intricate dependencies within the input sequence. Each layer is a powerhouse of computation, processing input embeddings through a sophisticated pipeline.

- Within each layer, two primary sub-layers operate in tandem: a multi-head self-attention mechanism and a position-wise feed-forward network. The multi-head self-attention allows the model to simultaneously attend to different parts of the input sequence from various "representation subspaces," enhancing its ability to weigh the importance of different words in relation to each other. This parallelism in attention allows the model to capture diverse contextual relationships. For example, in the sentence "The bank river overflowed," one attention head might focus on "bank" and "river," while another might focus on "bank" and "overflowed," discerning the semantic nuances of the word "bank." The feed-forward network, on the other hand, applies a non-linear transformation to the attended representations, further enriching the model's capacity to learn complex patterns. This network typically consists of two linear transformations with a ReLU activation in between, enabling the model to learn complex mappings from the attention outputs.

- To ensure stable training and effective gradient flow, residual connections and layer normalization are applied around each of these sub-layers, enabling deeper networks to be trained more effectively. Residual connections, first introduced in ResNet, allow gradients to flow directly through the network, mitigating the vanishing gradient problem. Layer normalization, applied independently to each feature across the batch, stabilizes the activations and accelerates training by reducing internal covariate shift. These techniques are crucial for training very deep neural networks like the Transformer, preventing degradation of performance as layers are added.

- For classification tasks, the final hidden state corresponding to the special <s> (beginning-of-sentence) token is extracted. This aggregated representation of the entire input sequence is then passed through a linear layer, followed by a softmax activation function. This process normalizes the output into a probability distribution over a predefined set of news categories (e.g., 10 categories), allowing the model to predict the most likely category for a given input. The <s> token acts as an aggregate representation of the entire input, effectively summarizing the semantic content for classification. XLM-R's pre-training was specifically focused on a massive multilingual corpus, enabling it to achieve state-of-the-art results in cross-lingual understanding. This pre-training involved techniques like masked language modeling on concatenated text from over 100 languages, allowing the model to learn shared linguistic features and transfer knowledge across languages effectively. This massive scale of pre-training on diverse linguistic data is a key differentiator for XLM-R, contributing to its strong performance in multilingual settings.

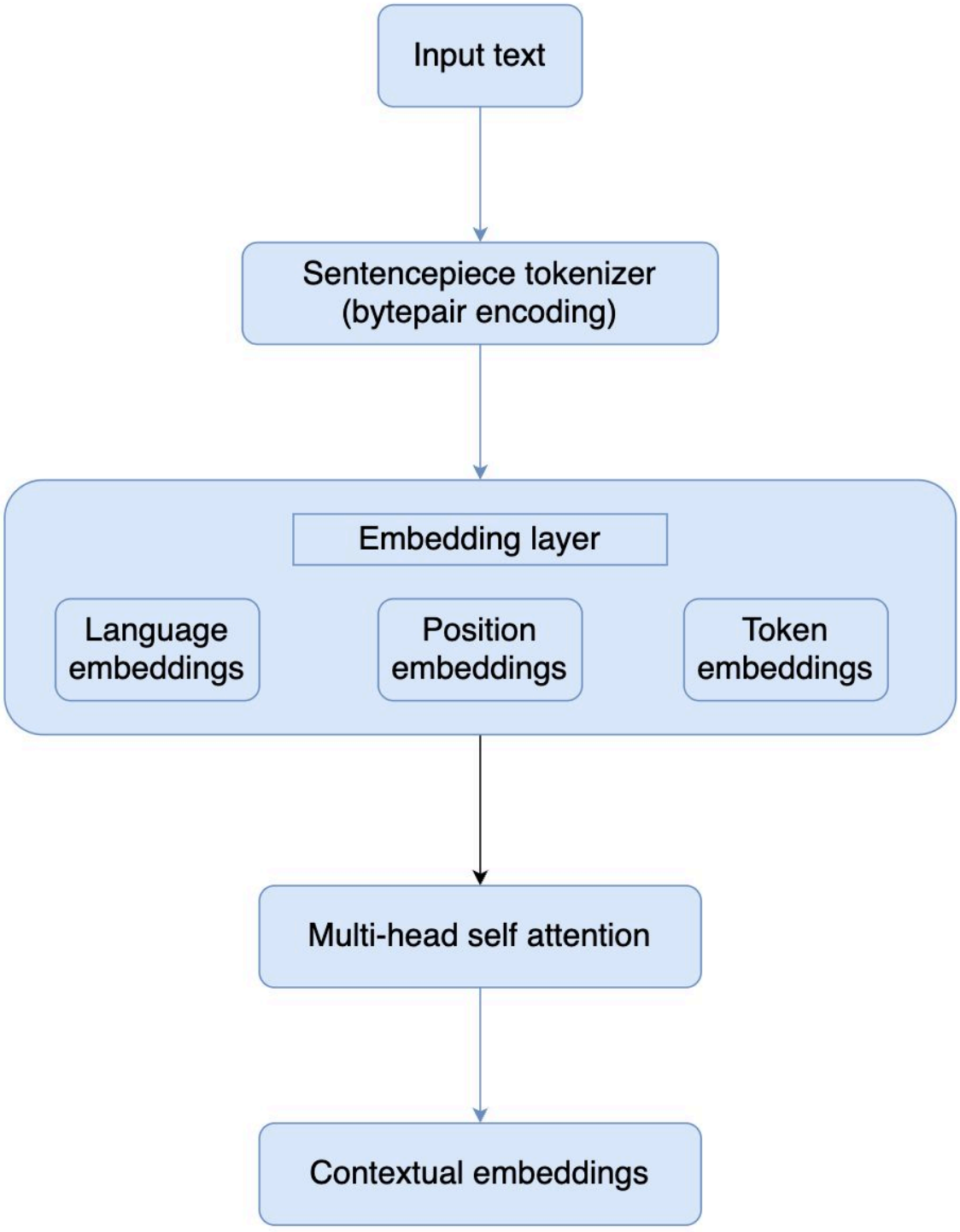


Fig3.2.2.1: XLM-R Architecture

**ii. mBERT (Multilingual BERT):**

As its name suggests, mBERT is a direct adaptation of the foundational BERT-base model for multilingual contexts. BERT, or Bidirectional Encoder Representations from Transformers, revolutionized NLP by pre-training deep bidirectional representations from unlabeled text, allowing it to understand the context of a word based on all of its surroundings. mBERT extends this paradigm to over 100 languages. Its architecture is precisely identical to the original BERT-base, featuring 12 Transformer encoder layers. Each layer incorporates 12 attention heads, allowing for diverse attention patterns, and maintains a hidden size of 768 dimensions for its internal representations. These parameters are fixed and replicated across all languages, meaning the same set of weights is used to process text in English, Arabic, Chinese, and so forth.

- A defining characteristic of mBERT is its use of a shared WordPiece vocabulary across all 104 pre-training languages. WordPiece is a subword tokenization algorithm that breaks down words into smaller units (subwords or characters) if they are rare or out-of-vocabulary, ensuring that all words can be represented while keeping the vocabulary size manageable. This shared vocabulary enables the model to learn universal representations for words and subword units that appear across multiple languages, fostering cross-lingual transfer learning. For instance, if the word "cat" in English and "gato" in Spanish share similar subword components or contexts during pre-training, mBERT can learn a more generalized representation that bridges these linguistic gaps. This shared vocabulary approach, while promoting transfer, also means that each language's unique characteristics must be encoded within this common representation space.

- The pre-training objectives for mBERT were similar to BERT, primarily masked language modeling (predicting masked tokens) and next sentence prediction. In masked language modeling, a percentage of input tokens are randomly masked, and the model is trained to predict the original vocabulary ID of the masked word based on its context. Next sentence prediction involves training the model to predict whether a second sentence logically follows the first, helping it understand relationships between sentences. These objectives force the model to learn rich, contextualized representations of words and sentences. The classification process mirrors that of XLM-R: the final representation of the [CLS] (classification) token (equivalent to <s> in XLM-R) from the last Transformer layer is fed into a linear layer and then a softmax function to generate probabilities for the target classification labels. The [CLS] token's final hidden state is considered to encapsulate the aggregated semantic meaning of the entire input sequence for downstream tasks. While effective, mBERT's approach of sharing a single vocabulary for many languages can sometimes lead to a less efficient representation for individual languages compared to models trained on larger, language-specific corpora. This "curse of multilingualism" suggests that the model's capacity might be spread thin across many languages, potentially limiting its depth of understanding for any single one.

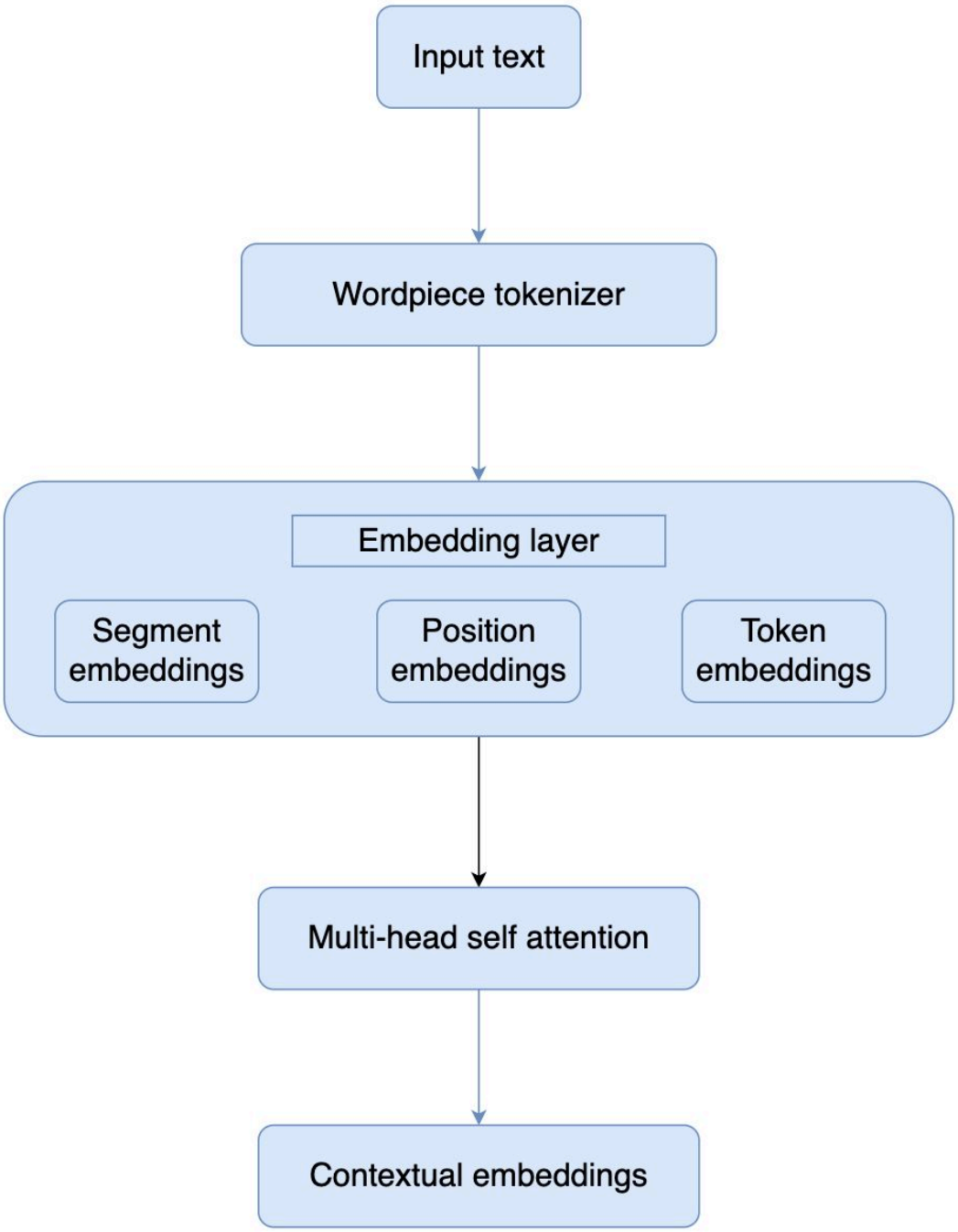


Fig3.2.2.2: mBERT Architecture

**iii. distilMBERT:**

This model represents a significant stride in model efficiency, being a highly compressed version of the mBERT architecture. The motivation behind DistilMBERT is to provide a smaller, faster, and more memory-efficient alternative to the full mBERT model without a drastic drop in performance. This is particularly valuable for deploying models on edge devices or in applications with strict latency requirements. DistilMBERT achieves its reduced size by employing only 6 Transformer layers, a substantial reduction compared to the 12 layers in its "teacher" model, mBERT. This architectural slimming is made possible through a technique called knowledge distillation.

- In this process, a smaller "student" model (DistilMBERT) is trained to mimic the output probabilities (soft targets) and sometimes even the hidden states of a larger, more powerful "teacher" model (mBERT). The teacher model, having been trained on a vast amount of data, has learned complex decision boundaries and confidence levels for its predictions. Instead of simply training the student on hard labels (e.g., "this is category A"), knowledge distillation leverages the "soft probabilities" output by the teacher, which contain richer information about the teacher's uncertainty and the relationships between different classes. For example, if the teacher model predicts a document is 90% "sports" and 10% "news," the student is encouraged to produce similar proportions, rather than just learning to classify it as "sports." This allows the student model to learn much of the knowledge encoded in the teacher, even with a significantly reduced parameter count. The core idea is that the teacher model provides a richer supervisory signal than just hard labels, guiding the student towards better generalization and regularization.

- The remaining architectural components of DistilMBERT, including the fundamental design of its Transformer layers and the use of the [CLS] token for classification, remain consistent with the broader BERT family of models. DistilMBERT shares the same vocabulary and tokenizer as mBERT, ensuring compatibility and leveraging the multilingual embedding space learned by its teacher. DistilMBERT offers a compelling balance between performance and computational efficiency, making it suitable for deployment in resource-constrained environments or applications requiring faster inference times. Its smaller size translates to reduced memory footprint and quicker execution, making it a valuable alternative when full mBERT capacity is not strictly necessary. While there is typically a slight performance drop compared to the teacher model, this trade-off is often acceptable given the significant gains in efficiency, making DistilMBERT a practical choice for real-world applications where resources are a constraint.

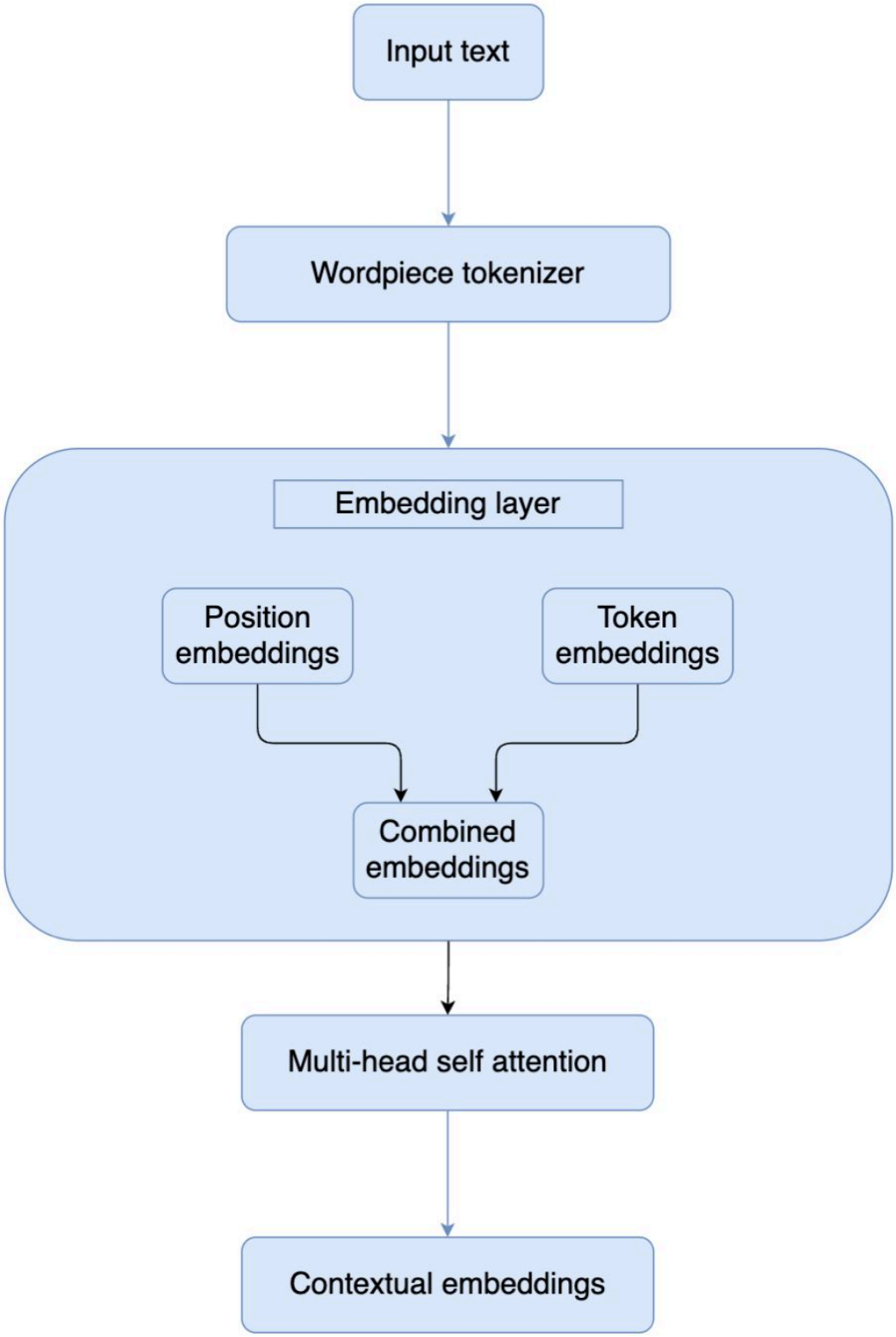


Fig3.2.2.3: distilmBERT Architecture

**3. Hyperparameter Tuning**

The transition to Phase 3 marked a shift from baseline training to meticulous, targeted optimization aimed at extracting maximum performance from the pre-trained models. This process recognized that simply selecting a powerful base model is insufficient; the success of transfer learning is critically dependent on a theoretically grounded fine-tuning methodology.

The successful deployment of sophisticated deep learning models, particularly those leveraging the power of transfer learning, hinges not merely on the selection of a robust pre-trained architecture but critically on a theoretically grounded and meticulously executed fine-tuning methodology. The transition from initial baseline training to Phase 3 of model development signifies this pivotal shift, emphasizing targeted optimization to extract the utmost performance from these powerful pre-trained models. This section delves into the foundational components of this optimization strategy, focusing on the selection and application of the AdamW optimizer, the implementation of a linear learning rate scheduler with a warmup period, and the rigorous process of hyperparameter selection.The AdamW Optimizer: Decoupling Regularization for Enhanced Generalization

A cornerstone of the fine-tuning process for all supervised models was the judicious choice of the **AdamW optimizer**. Its selection was not arbitrary but rooted in its empirically demonstrated efficacy in improving generalization performance, particularly within the context of Transformer architectures. To understand the superior performance of AdamW, it's essential to differentiate it from its predecessor, the standard Adam optimizer, and to highlight the fundamental conceptual shift it introduces.

In traditional Adam, L2 regularization, commonly known as weight decay, is intrinsically coupled with the adaptive learning rate mechanism. This coupling presents a significant drawback: the adaptive learning rates, which dynamically scale gradients for each parameter, can inadvertently distort the intended effect of weight decay. Specifically, parameters with large gradients might experience a diminished effect of weight decay, as their scaled updates effectively override the regularization term. Conversely, parameters with small gradients might be excessively regularized. This undesirable interaction can lead to suboptimal regularization, ultimately hindering the model's capacity to generalize effectively to unseen data. The very mechanism designed to accelerate convergence and navigate complex loss landscapes within Adam – its adaptive learning rates – ironically undermines the consistent application of a crucial regularization technique.

AdamW brilliantly rectifies this issue by introducing a crucial decoupling: it applies weight decay directly to the model’s weights *after* the gradient-based update, treating it as a distinct regularization term, independent of the adaptive learning rate mechanism. This architectural change ensures that the integrity and effectiveness of the adaptive learning rates are preserved in guiding the optimization process, while weight decay provides a consistent and uniform regularization across all parameters, irrespective of their gradient magnitudes. This principled separation guarantees that regularization is applied in a manner that truly reflects its purpose – to prevent overfitting by penalizing large weights – without being distorted by the dynamics of adaptive learning.

The benefits of this decoupling are particularly pronounced in the context of large neural networks like Transformers. These architectures, with their vast number of parameters, are highly susceptible to overfitting. Precise control over regularization is therefore paramount to ensure robust performance on unseen data. AdamW's ability to provide consistent and effective regularization, unadulterated by adaptive learning rates, empowers these complex models to learn more generalizable representations. The improved generalization capabilities observed with AdamW have solidified its position as a preferred choice in a multitude of state-of-the-art deep learning applications, underscoring its pivotal role in achieving optimal model performance and fostering true learning rather than mere memorization.The Linear Learning Rate Scheduler with Warmup: A Phased Approach to Mitigating Catastrophic Forgetting

Complementing the AdamW optimizer, the implementation of a **linear learning rate scheduler with a warmup period** proved instrumental in enhancing the effectiveness of the fine-tuning process. This dynamic, two-phase approach is specifically engineered to address a critical and frequently encountered challenge during the fine-tuning of large pre-trained models: catastrophic forgetting. Catastrophic forgetting refers to the detrimental phenomenon where a model, upon being fine-tuned on new data, rapidly and severely loses the knowledge and representations it painstakingly acquired during its initial, often extensive, pre-training phase. This can lead to a significant

degradation in performance, especially when the new data distribution diverges considerably from the pre-training data, effectively erasing months or years of accumulated knowledge.

The **warmup period**, constituting the initial phase of the scheduler, plays an absolutely vital role in mitigating this catastrophic forgetting. During this crucial phase, the learning rate embarks from a very low, almost negligible, value and progressively increases linearly to a predetermined peak value. This gradual ascent occurs over a specified number of steps or epochs. The rationale behind this cautious, gentle increase is to allow the model to gradually and softly adapt to the nuances and specific characteristics of the new data distribution without immediately disrupting its meticulously learned and deeply embedded pre-trained knowledge. A sudden, large learning rate applied at the very commencement of fine-tuning can trigger drastic and destabilizing weight updates. These abrupt changes can effectively "shock" the model, causing it to rapidly unlearn or erase valuable pre-trained features and representations that are fundamental to its strong baseline performance. The warmup period acts as a cushioned entry, providing a grace period that enables the model to "warm up" to the new task and data. This allows it to gradually adjust its parameters while concurrently preserving the robust and highly generalizable representations it already possesses from pre-training. This initial cautious approach is paramount in stabilizing the training process, preventing the occurrence of large, destabilizing gradient updates in the critical early stages of fine-tuning, and setting the stage for more effective learning.

Following the successful completion of the warmup phase, the scheduler seamlessly transitions into its second stage: a **linear decay phase**. In this stage, the learning rate systematically and linearly decreases from its peak value over the remaining duration of the training epochs. This progressive reduction in the learning rate as training advances serves multiple critical purposes, each contributing to optimal model convergence and performance. Firstly, a decreasing learning rate facilitates finer and more precise convergence as the model navigates the intricate landscape of the loss function and approaches the optimal solution. While larger learning rates are highly beneficial in the initial stages for rapidly exploring the broad contours of the loss function and escaping local minima, as the model draws closer to the true minimum, smaller and more refined steps are imperative. These smaller steps ensure that the parameters are fine-tuned without oscillating wildly around the optimal point or overshooting it, leading to a more stable and accurate convergence. Secondly, a gradually decreasing learning rate proves highly effective in helping the model escape stubborn saddle points and shallow local minima. By reducing the step size, the model is guided towards deeper, more robust solutions in the loss landscape, rather than settling for suboptimal local optima. This intelligent, phased approach, characterized by an initial cautious exploration to preserve pre-trained knowledge, followed by a more precise and refined optimization, is demonstrably highly effective in achieving optimal fine-tuning results for complex models, ensuring both stability and peak performance.Rigorous Hyperparameter Selection: The Cornerstone of Robust Performance

The selection of hyperparameters for both the AdamW optimizer and the linear learning rate scheduler with warmup was far from arbitrary; it was the meticulously documented outcome of extensive and rigorous preliminary experiments. This systematic approach is a fundamental pillar of successful deep learning model development, ensuring that the chosen configurations are optimally aligned with the specific model architecture, the unique characteristics of the dataset, and the precise requirements of the task at hand.

These preliminary experiments involved a comprehensive and systematic variation of different hyperparameter values. For instance, a broad range of values were explored for the initial learning rate, the peak learning rate during the warmup phase, the total number of warmup steps, the overall number of training epochs, and the precise values for weight decay. Each configuration was meticulously evaluated based on its impact on a suite of crucial model performance metrics. These metrics included, but were not limited to, validation loss, accuracy on held-out validation sets, and, most importantly, the model's generalization capability – its ability to perform well on data it had never encountered during training.

The overarching goal of this iterative process of experimentation and evaluation was to identify the specific hyperparameter configuration that not only yielded the most stable training process but also facilitated the fastest convergence to an optimal solution, and, most critically, demonstrated the best possible performance on held-out validation datasets. This rigorous approach is indispensable in deep learning because the optimal hyperparameters are rarely universal; they are highly dependent on the interplay between the chosen model architecture, the statistical properties of the training data, and the inherent complexity of the task. Without such systematic tuning, even the most powerful models can underperform or suffer from issues like overfitting or underfitting.

The final hyperparameter configurations, which were meticulously selected after these rigorous preliminary experiments, are comprehensively detailed and presented in Table III.III of the full research report. This detailed documentation serves as a critical reference point, ensuring the reproducibility of the research findings and providing a clear foundation for further research and development. This systematic and data-driven approach to hyperparameter tuning underpins the robust performance achieved by the fine-tuned supervised models, testifying to the importance of empirical validation in the pursuit of optimal deep learning solutions. It transforms the often-perceived "art" of hyperparameter tuning into a scientific, reproducible, and highly effective methodology for maximizing model efficacy.

**Table III.III: Hyperparameter Configurations for Fine-Tuned Models**

| Hyperparameter | XLM-Roberta | DistilMBERT | mBERT |
|---|---|---|---|
| Base Model | xlm-roberta-base | distilbert-base-multilingual-cased | bert-base-multilingual-cased |
| Learning Rate | 2×10−5 | 3×10−5 | 2×10−5 |
| Learning Rate Scheduler | Linear with Warmup | Linear with Warmup | Linear with Warmup |
| Number of Epochs | 5 | 5 | 5 |
| Batch Size | 32 | 32 | 32 |
| Weight Decay | 0.01 | 0.0 | 0.01 |
| Max Sequence Length | 28 | 28 | 28 |

## C. Model Training and Evaluation

### 1. Training Procedure and Metrics

A rigorous and multi-faceted evaluation protocol was established to assess the performance of the models. The dataset was split into training and testing sets, with an 80-20 ratio used for the initial baseline evaluation (48,000 training, 2,000 testing) and a larger holdout set of 20,000 samples reserved for the final evaluation of the fine-tuned models. The training procedure for the supervised models involved fine-tuning the pre-trained Transformers on the

custom Kannada dataset. The models were trained for 5 epochs using the hyperparameters specified in Table III.III, with the objective of minimizing the cross-entropy loss between the predicted probabilities and the true labels.Model Evaluation Protocol

A comprehensive and rigorous evaluation protocol was meticulously established to ascertain the performance and robustness of the developed models. This protocol involved a multi-faceted approach, beginning with a strategic division of the primary dataset.

**Dataset Partitioning:**
The dataset was initially partitioned into distinct training and testing sets to facilitate baseline performance assessment. An 80-20 ratio was employed for this initial split, resulting in a training set comprising 48,000 samples and a testing set of 2,000 samples. This initial division was crucial for establishing a foundational understanding of the models' capabilities before further optimization. Crucially, a larger, independent holdout set of 20,000 samples was reserved for the ultimate and unbiased evaluation of the fine-tuned models, ensuring that the final performance metrics were not influenced by data seen during training or hyperparameter tuning.

**Supervised Model Training Procedure:**
The training methodology for the supervised models centered on the fine-tuning of pre-trained Transformer architectures. This process leveraged a custom-curated Kannada dataset, specifically designed to address the nuances of the target language and task. The models underwent a training regimen of five epochs, a period deemed sufficient to allow for convergence without excessive overfitting. Hyperparameter optimization was guided by the specifications detailed in Table III.III, which provided a comprehensive set of configurations for learning rates, batch sizes, and other critical parameters. The primary objective driving the training process was the minimization of the cross-entropy loss. This loss function was selected due to its efficacy in quantifying the discrepancy between the predicted probability distributions generated by the models and the true, one-hot encoded labels of the dataset, thereby guiding the models towards more accurate predictions.


Fig3.3.1.1: XLM-R Training loss

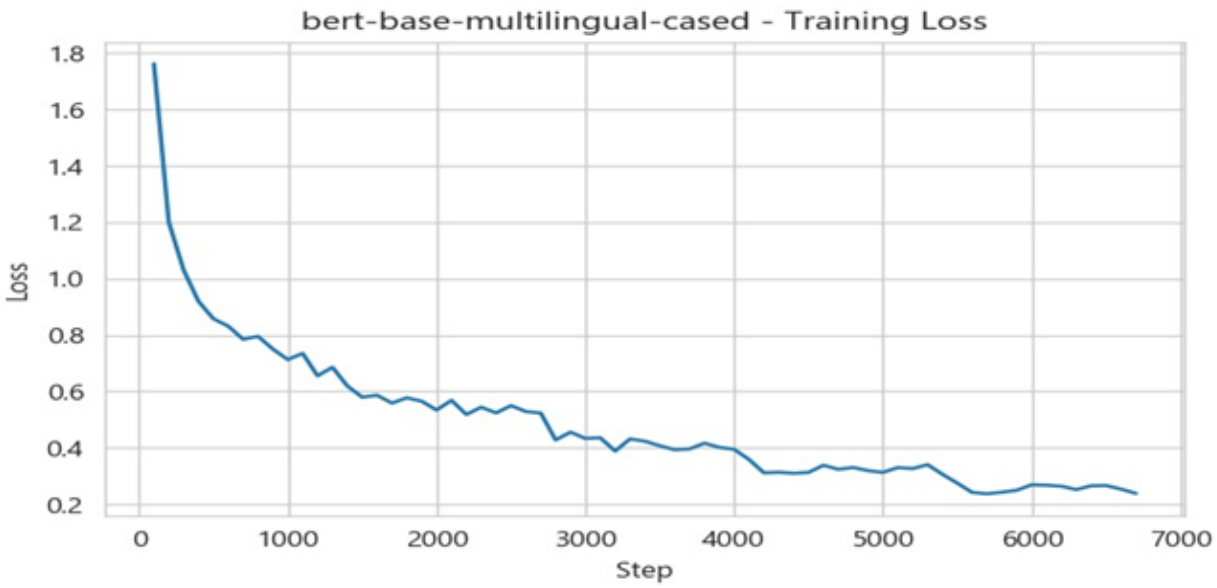


Fig3.3.1.1: mBERT Training loss

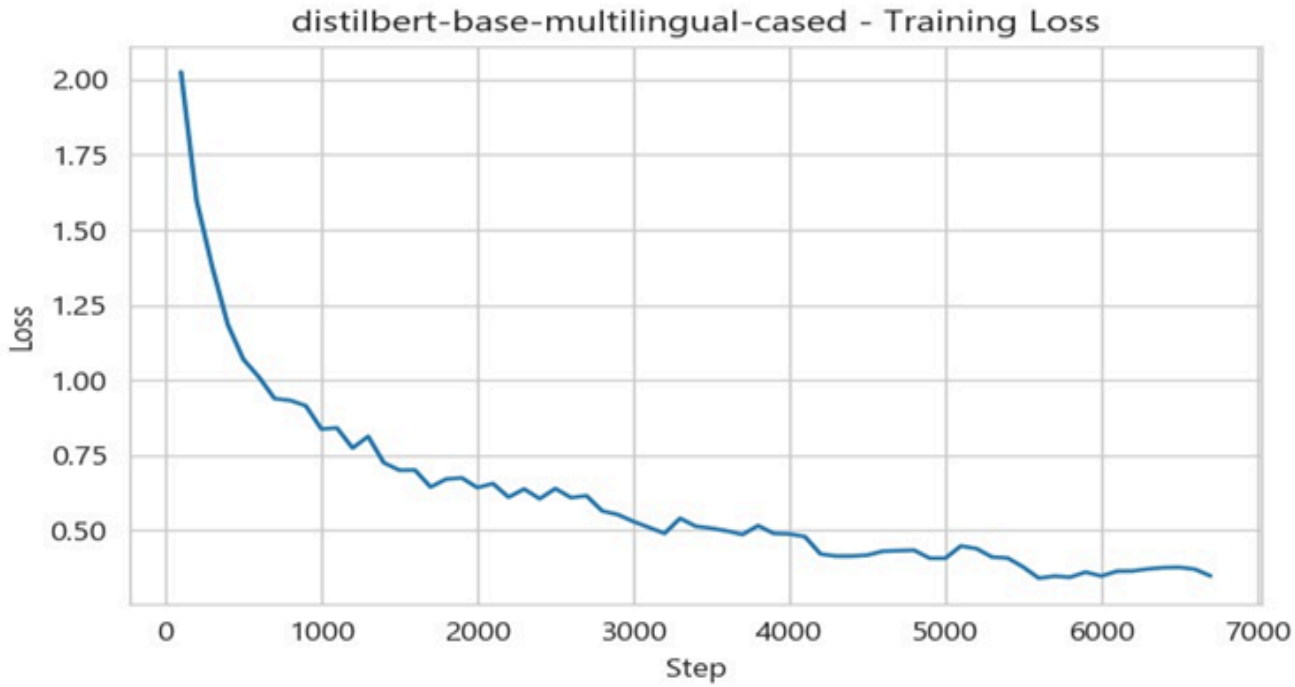


Fig3.3.1.1: distilmBERT Training loss

**2. Evaluation Metrics and Performance Analysis**

To provide a comprehensive understanding of model behavior, especially in the context of a multi-class problem with significant class imbalance, a suite of standard classification metrics was used.

- **Accuracy:** The overall proportion of correctly classified headlines. While intuitive, it can be misleading on imbalanced datasets. Calculated as: Accuracy=Total Number of PredictionsNumber of Correct Predictions.
- **Precision:** For a given class, this measures the proportion of true positives among all instances predicted as that class. It answers the question: "Of all the headlines predicted as 'Sports', how many were actually 'Sports'?" Calculated per class as: Precision=True Positives+False PositivesTrue Positives.
- **Recall (Sensitivity):** For a given class, this measures the proportion of true positives that were correctly identified. It answers the question: "Of all the actual 'Sports' headlines, how many did the model correctly identify?" Calculated per class as: Recall=True Positives+False NegativesTrue Positives.
- **F-Score:** The harmonic mean of precision and recall, providing a balanced measure that is particularly valuable for imbalanced datasets where both false positives and false negatives are important. Calculated per class as: F−Score=2×Precision+RecallPrecision×Recall.
- **AUC–ROC:** The Area Under the Receiver Operating Characteristic Curve measures the model's ability to distinguish between classes across all possible classification thresholds.

For this multi-class problem, the **macro-average** of these metrics was emphasized, as it calculates the metric independently for each class and then takes the unweighted average. This approach treats all classes equally, regardless of their frequency in the dataset, providing a more robust measure of performance on the minority classes than a weighted average would.To provide a comprehensive understanding of model behavior, especially in the context of a multi-class problem with significant class imbalance, a suite of standard classification metrics was used. This section details each metric, its calculation, and its relevance in evaluating the performance of the classification model.

Let’s understand each of these metric in detail

**Accuracy:**

Accuracy is perhaps the most intuitive and commonly understood classification metric. It represents the overall proportion of correctly classified instances out of the total number of predictions made by the model. Its simplicity makes it easy to interpret: a higher accuracy score indicates a greater number of correct predictions.

> **Formula:** Accuracy = (Number of Correct Predictions) / (Total Number of Predictions)
>
> **Interpretation:** While straightforward, accuracy can be highly misleading, particularly when dealing with imbalanced datasets. In such scenarios, if one class significantly outnumbers others, a model can achieve high accuracy simply by classifying the majority of instances into the dominant class, even if it performs poorly on the minority classes. For example, if 95% of headlines are 'News' and only 5% are 'Sports', a model that always predicts 'News' would achieve 95% accuracy, but it would be useless for identifying 'Sports' headlines. Therefore, for imbalanced multi-class problems, accuracy alone is insufficient and can provide a deceptive impression of model performance.

**Precision:**

Precision focuses on the quality of positive predictions made by the model. For a given class, it measures the proportion of true positives among all instances that the model predicted as belonging to that class. In essence, it answers the question: "Of all the headlines the model predicted as 'Sports', how many were actually 'Sports'?"

> **Formula (per class):** Precision = True Positives / (True Positives + False Positives)
>
> **Components:**

- **True Positives (TP):** Instances correctly predicted as belonging to the class.
- **False Positives (FP):** Instances incorrectly predicted as belonging to the class (i.e., they actually belong to another class).

**Interpretation:** High precision for a class indicates a low rate of false positives. This is crucial in scenarios where the cost of a false positive is high. For instance, in a medical diagnosis system, high precision for predicting a rare disease means that when the model says a patient has the disease, it is highly likely to be correct, minimizing unnecessary treatments or anxiety. In a multi-class news classification system, high precision for "Politics" headlines means that when the model tags a headline as "Politics," it's very likely to be genuinely political, preventing miscategorization.

**Recall (Sensitivity):**

Recall, also known as sensitivity, measures the model's ability to identify all relevant instances of a particular class. For a given class, it calculates the proportion of true positives that were correctly identified out of all actual instances belonging to that class. It answers the question: "Of all the actual 'Sports' headlines, how many did the model correctly identify?"

**Formula (per class):** Recall = True Positives / (True Positives + False Negatives)

**Components:**

- **True Positives (TP):** Instances correctly predicted as belonging to the class.
- **False Negatives (FN):** Instances that actually belong to the class but were incorrectly predicted as belonging to another class (i.e., missed by the model).

**Interpretation:** High recall for a class indicates a low rate of false negatives. This is vital in situations where the cost of a false negative is high. For example, in a fraud detection system, high recall for "fraudulent transactions" means that the system is effective at catching most instances of fraud, even if it leads to some false alarms. In our news classification, high recall for "Entertainment" headlines means the model is good at finding nearly all the actual entertainment-related articles, minimizing the chance of missing relevant content. There is often a trade-off between precision and recall; improving one might lead to a decrease in the other.

**F-Score (F1-Score):**

The F-Score, specifically the F1-score, is the harmonic mean of precision and recall. It provides a balanced measure that considers both false positives and false negatives, making it particularly valuable for imbalanced datasets where a high score indicates that the model has good values for both precision and recall. Unlike the arithmetic mean, the harmonic mean gives more weight to lower values, meaning that a good F1-score requires both precision and recall to be relatively high.

**Formula (per class):** F-Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

**Interpretation:** The F-Score is a useful aggregate metric when you need a balance between precision and recall. If precision is very low, the F-score will be low even if recall is high, and vice versa. This makes it a more robust measure than accuracy for imbalanced datasets because it penalizes models that perform well on only one aspect (either precision or recall) while neglecting the other. For instance, a model with very high precision but very low recall for a minority class would have a low F-score, accurately reflecting its inability to identify most instances of that class.

**AUC–ROC (Area Under the Receiver Operating Characteristic Curve):**

AUC-ROC is a comprehensive metric that evaluates the model's ability to distinguish between classes across all possible classification thresholds. The ROC curve plots the True Positive Rate (Recall) against the False Positive Rate (1 - Specificity) at various threshold settings. Specificity measures the proportion of true negatives that are correctly identified.

**Concept:**

- **True Positive Rate (TPR) / Recall:** True Positives / (True Positives + False Negatives)
- **False Positive Rate (FPR):** False Positives / (False Positives + True Negatives)

**Interpretation:** The AUC value ranges from 0 to 1. An AUC of 1 indicates a perfect model that can perfectly distinguish between positive and negative classes. An AUC of 0.5 suggests that the model performs no better than random guessing. A model with an AUC less than 0.5 is performing worse than random, possibly by consistently predicting the wrong class. For multi-class problems, AUC-ROC can be extended using various averaging strategies (e.g., one-vs-rest approach) to assess overall discriminative power. It is robust to class imbalance, as it considers the trade-off between sensitivity and specificity across all possible decision boundaries, rather than relying on a single threshold.

**Macro-Average Emphasis for Multi-Class Problems:**

For this specific multi-class problem, where headline categories can be highly imbalanced, the **macro-average** of these metrics was emphasized. This approach is crucial for obtaining a meaningful assessment of model performance, especially on minority classes.

**Calculation:** The macro-average calculates the metric (e.g., precision, recall, F-score) independently for each class and then takes the unweighted average of these per-class scores.

**Why Macro-Average?**

- **Treats All Classes Equally:** Unlike a weighted average (micro-average or weighted F1-score), which considers the number of instances in each class, the macro-average gives equal importance to every class, regardless of its frequency in the dataset.
- **Robust Measure for Minority Classes:** This equal weighting is particularly beneficial in imbalanced datasets. If a dataset has a few very large classes and several very small classes, a weighted average would be heavily influenced by the performance on the large classes, potentially masking poor performance on the minority classes. The macro-average, by treating all classes equally, provides a more robust and honest measure of how well the model performs across *all* categories, highlighting its ability to generalize to less frequent classes.
- **Reveals Strengths and Weaknesses:** By examining the macro-averaged metrics, researchers can gain a clearer understanding of the model's overall strengths and weaknesses across the entire spectrum of categories, not just the dominant ones. This can guide further model refinement, especially if certain minority classes show consistently low performance.

In conclusion, by employing a comprehensive suite of classification metrics and emphasizing the macro-average for multi-class evaluation, a thorough and accurate assessment of the model's behavior was ensured, providing valuable insights into its effectiveness in handling imbalanced news headline classification.

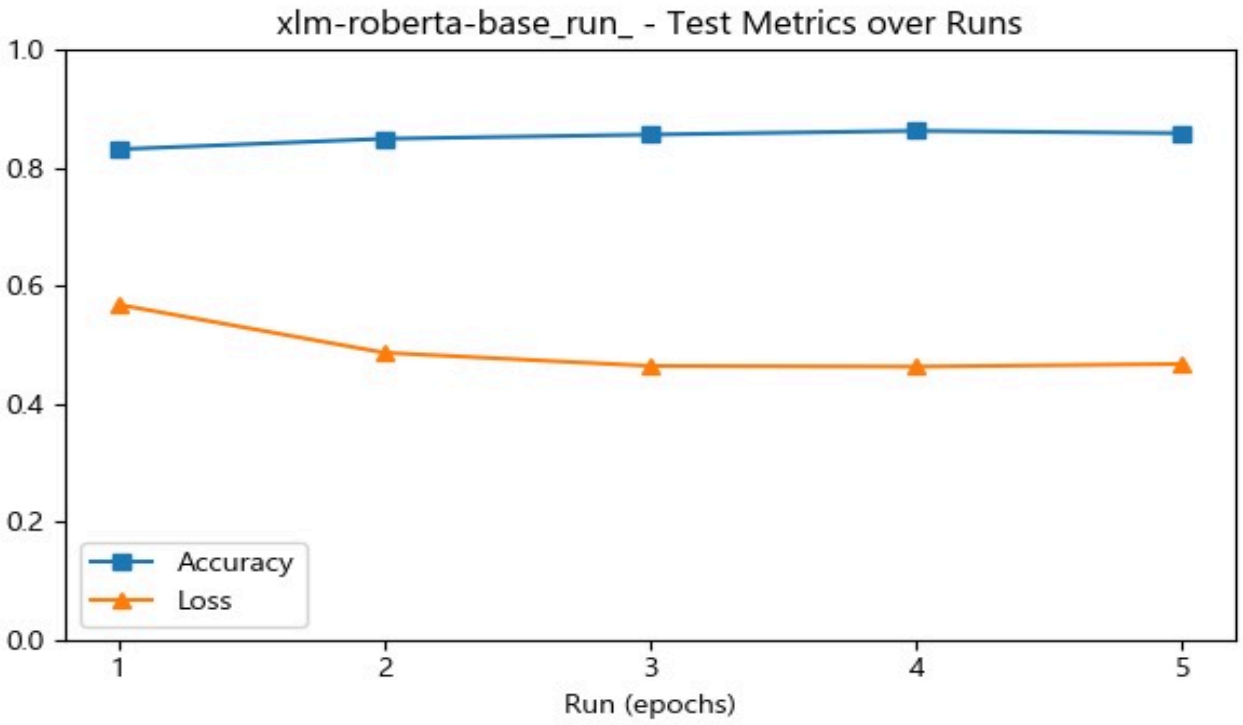


Fig3.3.2.1: XLM-R accuracy-loss over epochs

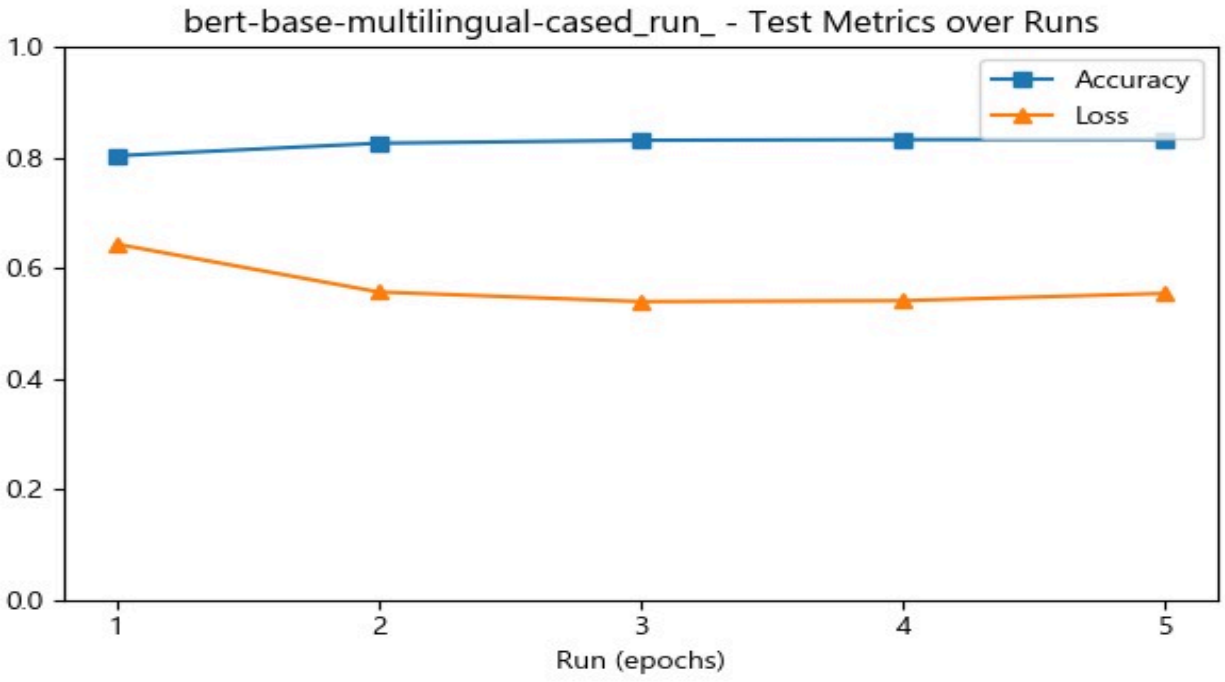


Fig3.3.2.2: mBERT accuracy-loss over epochs

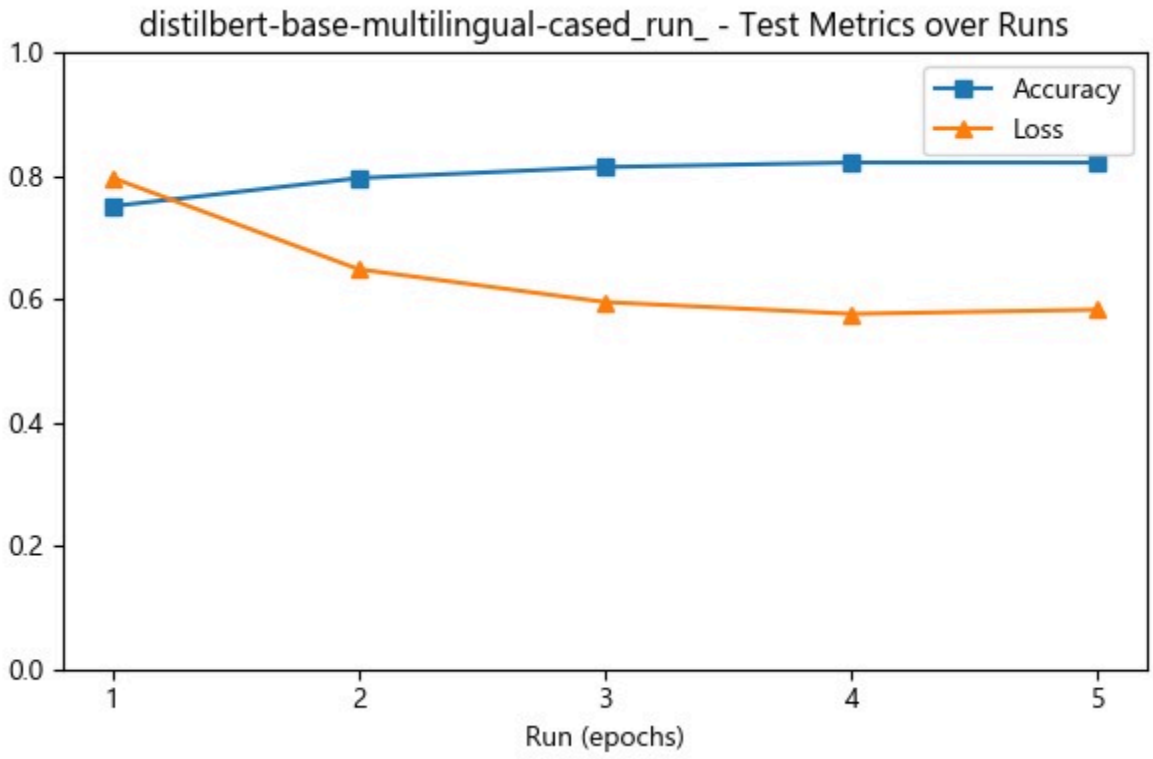


Fig3.3.2.3: distilmBERT accuracy-loss over epochs

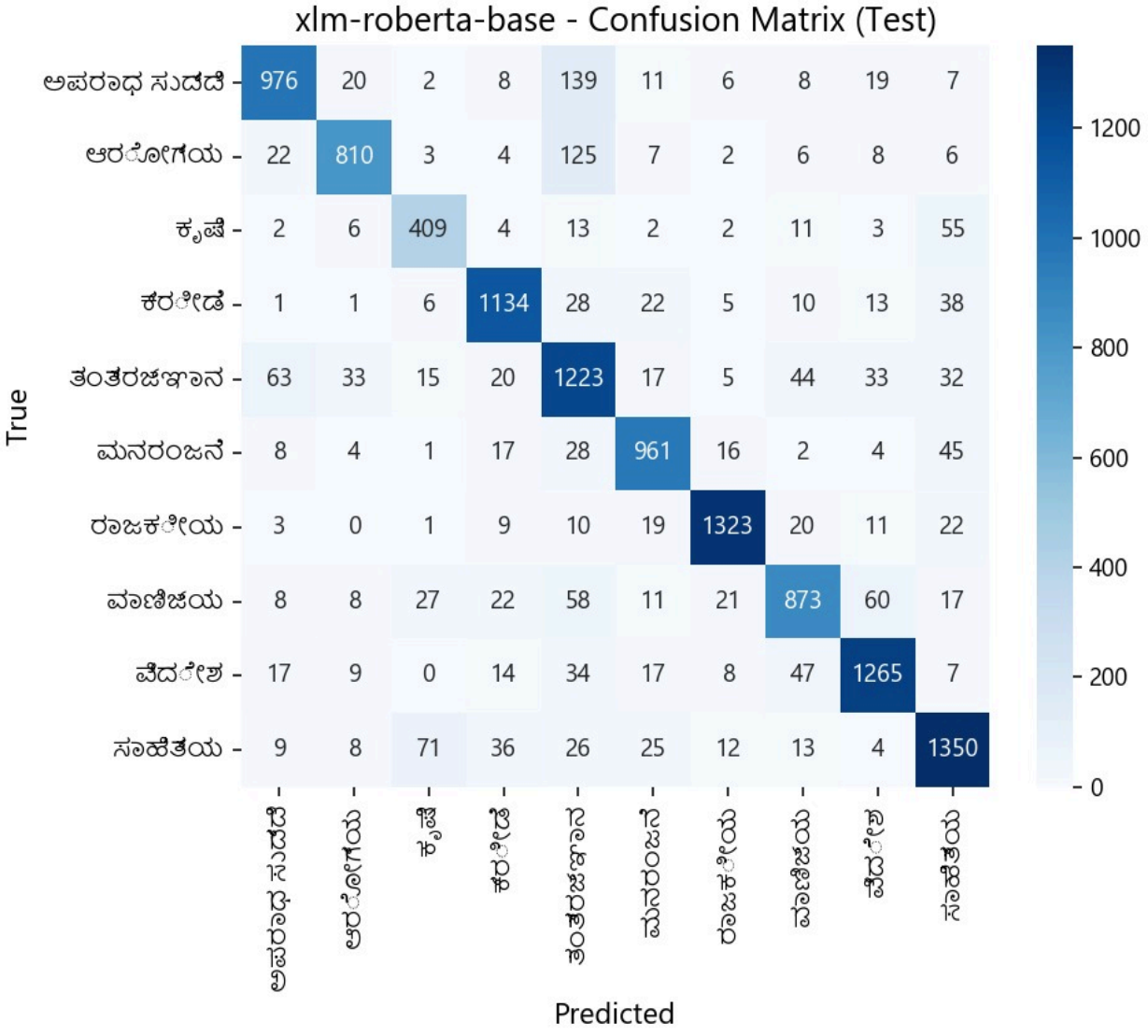


Fig3.3.2.4: XLM-R Confusion matrix

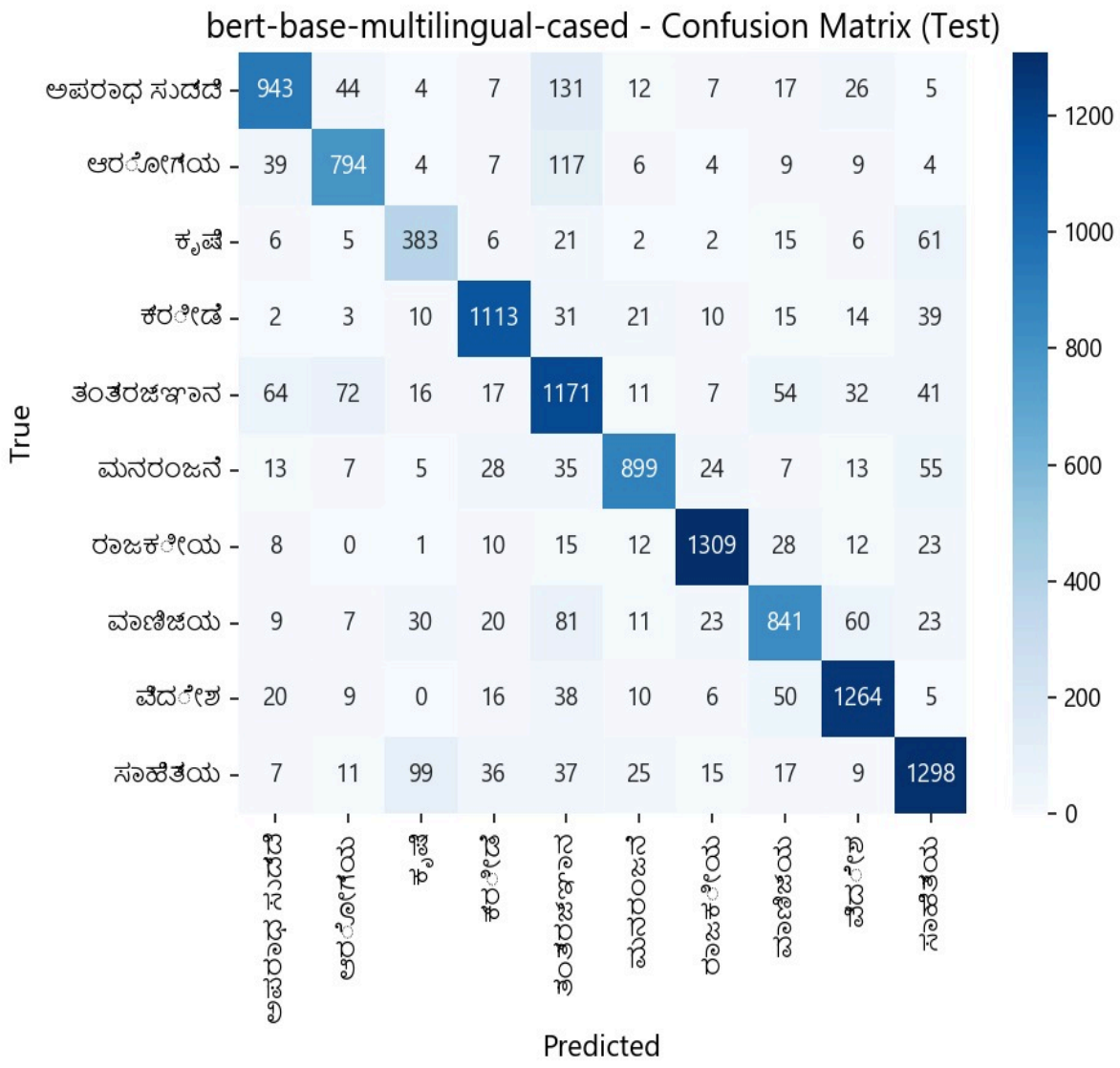


Fig3.3.2.5: mBERT Confusion matrix

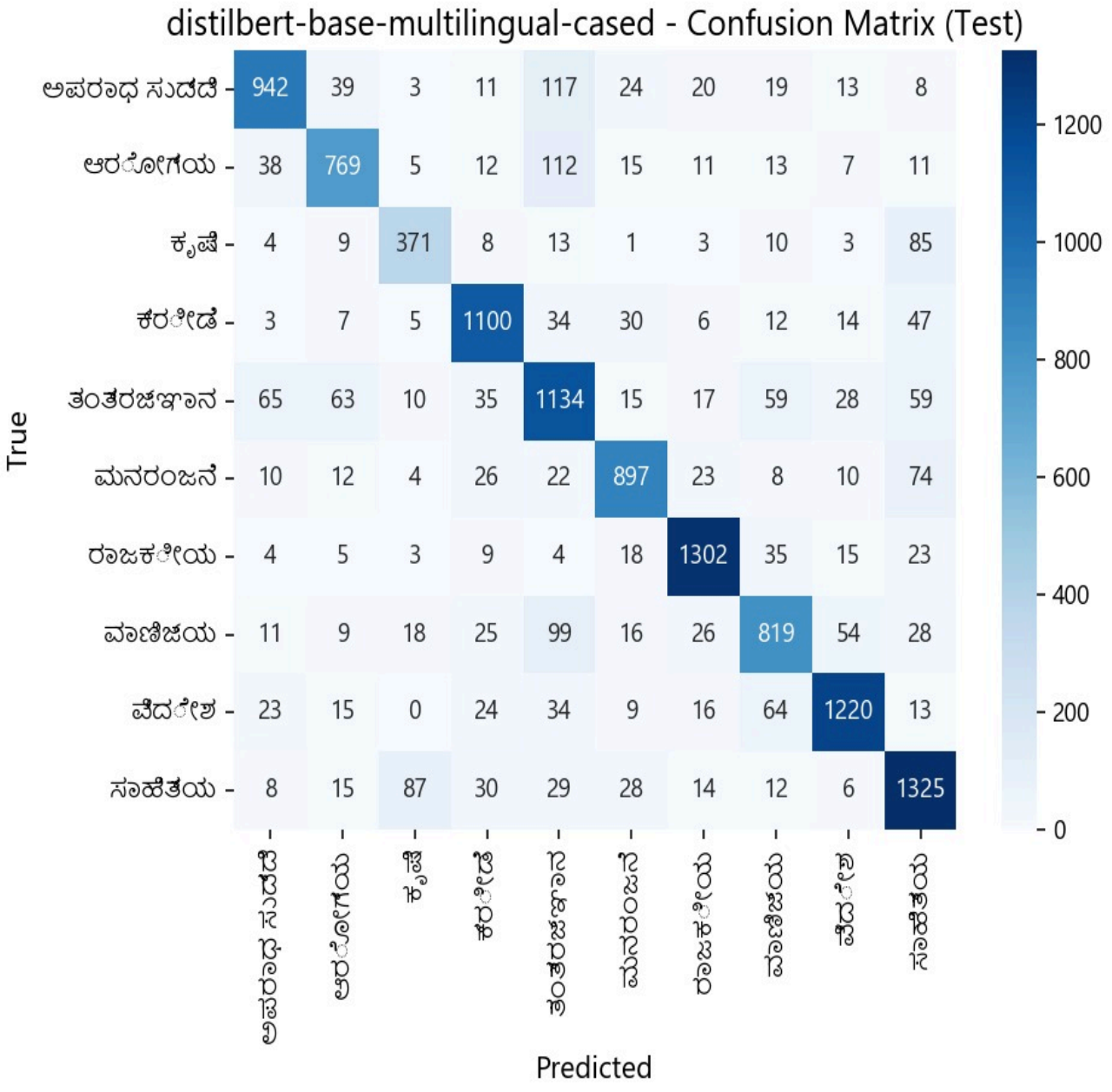


Fig3.3.2.6: distilmBERT Confusion matrix

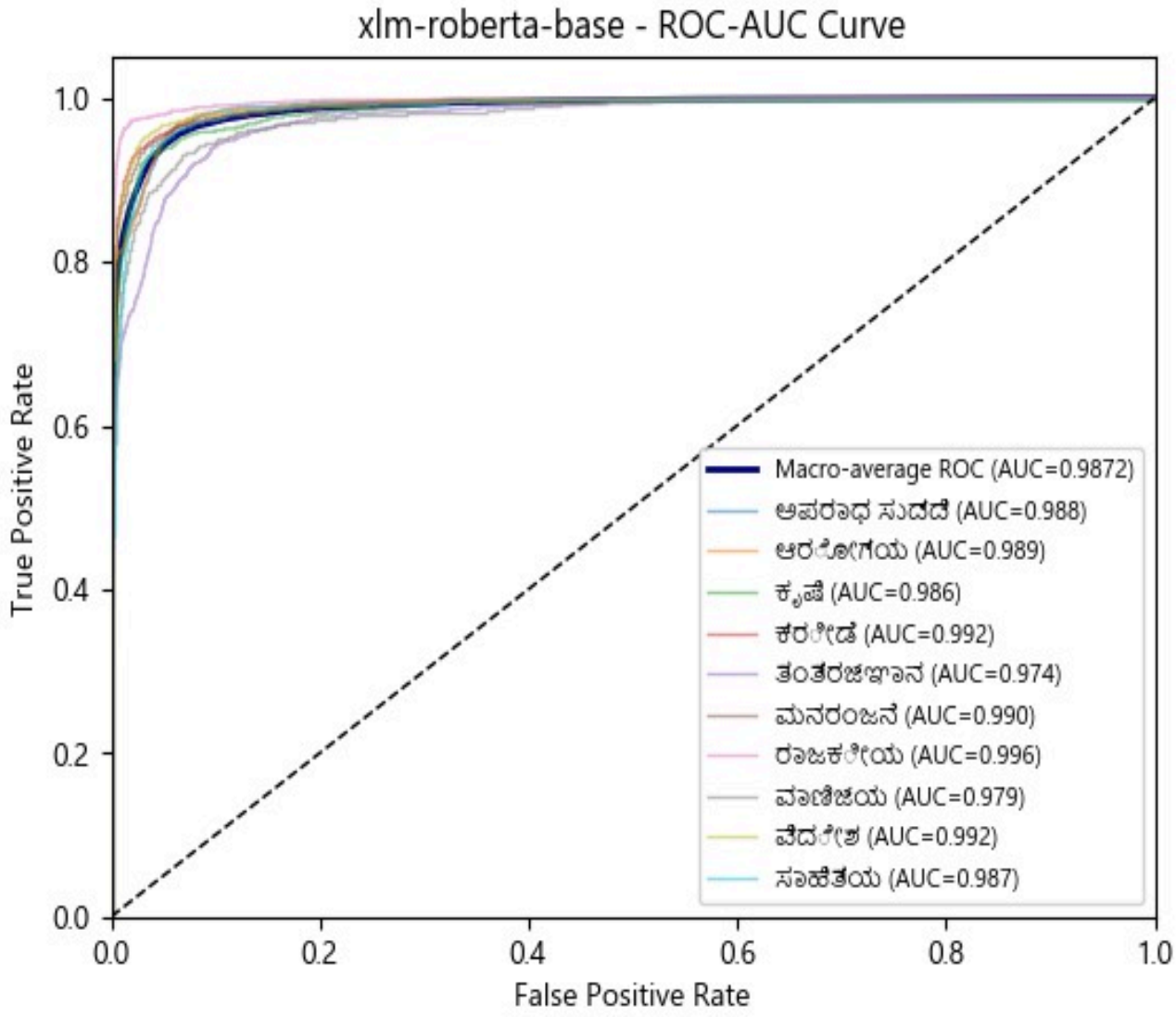


Fig3.3.2.7: XLM-R macro average ROC-AUC curve

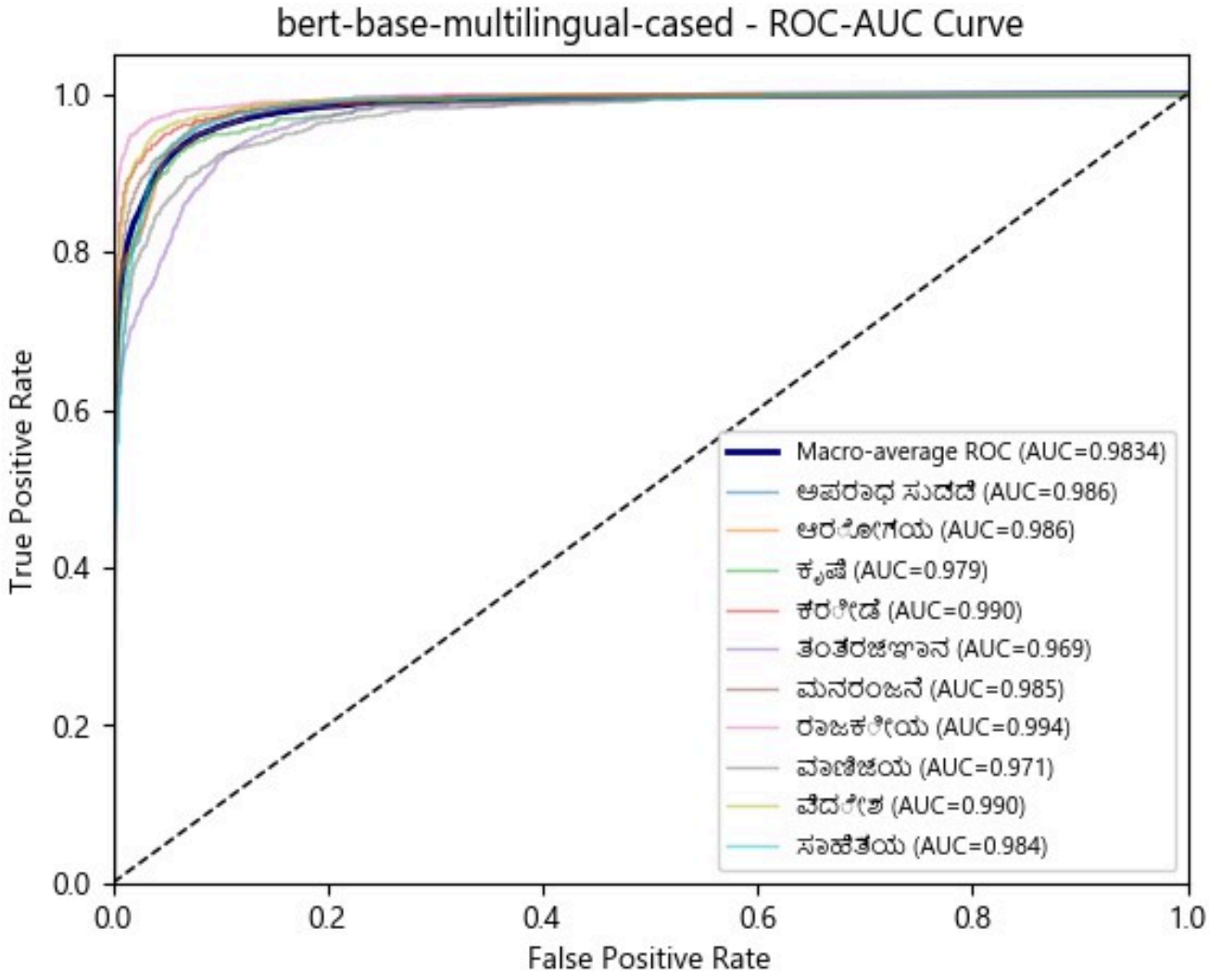


Fig3.3.2.8: mBERT macro-average ROC-AUC curve

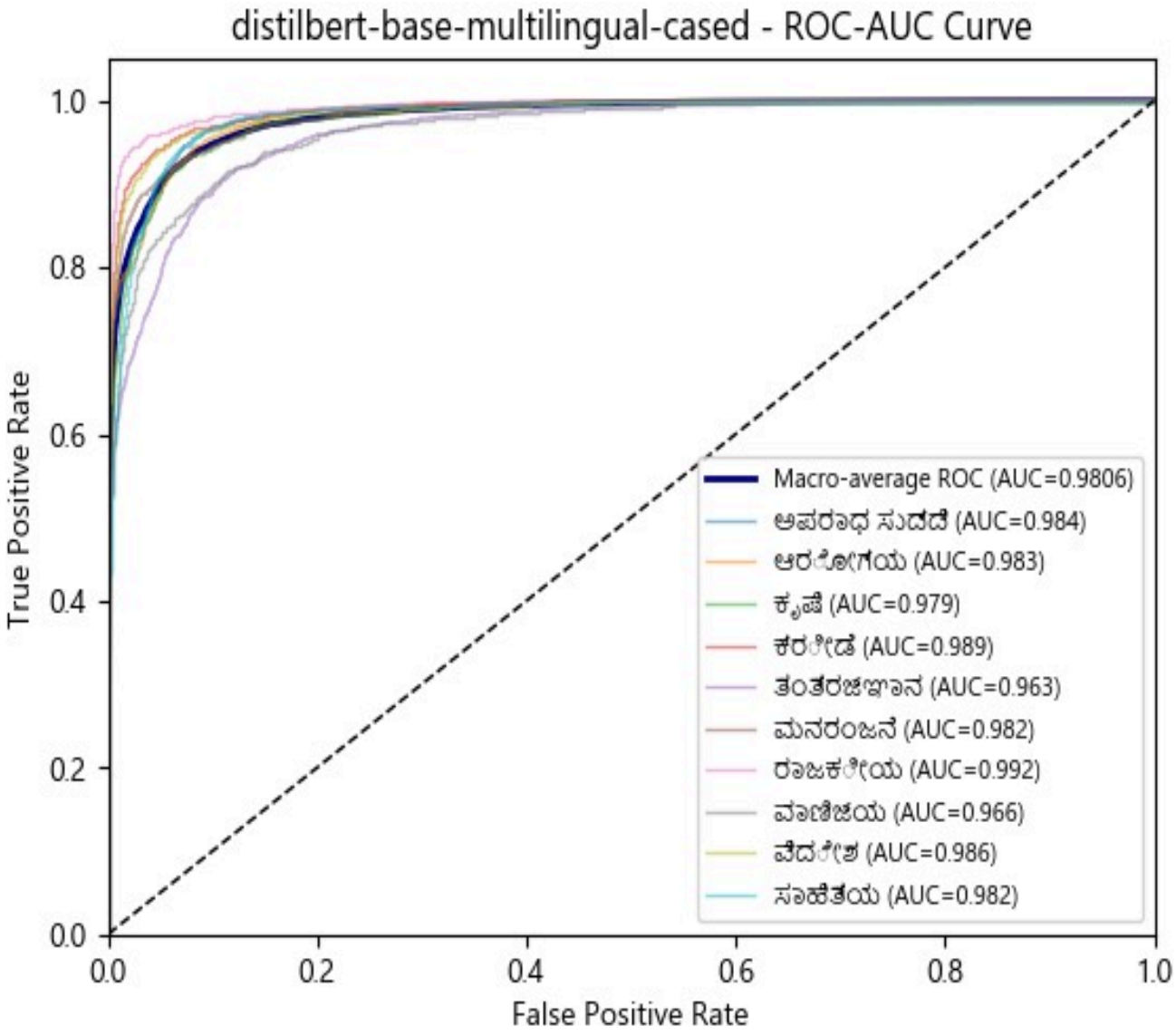


Fig3.3.2.9: distilmBERT macro-average ROC-AUC curve

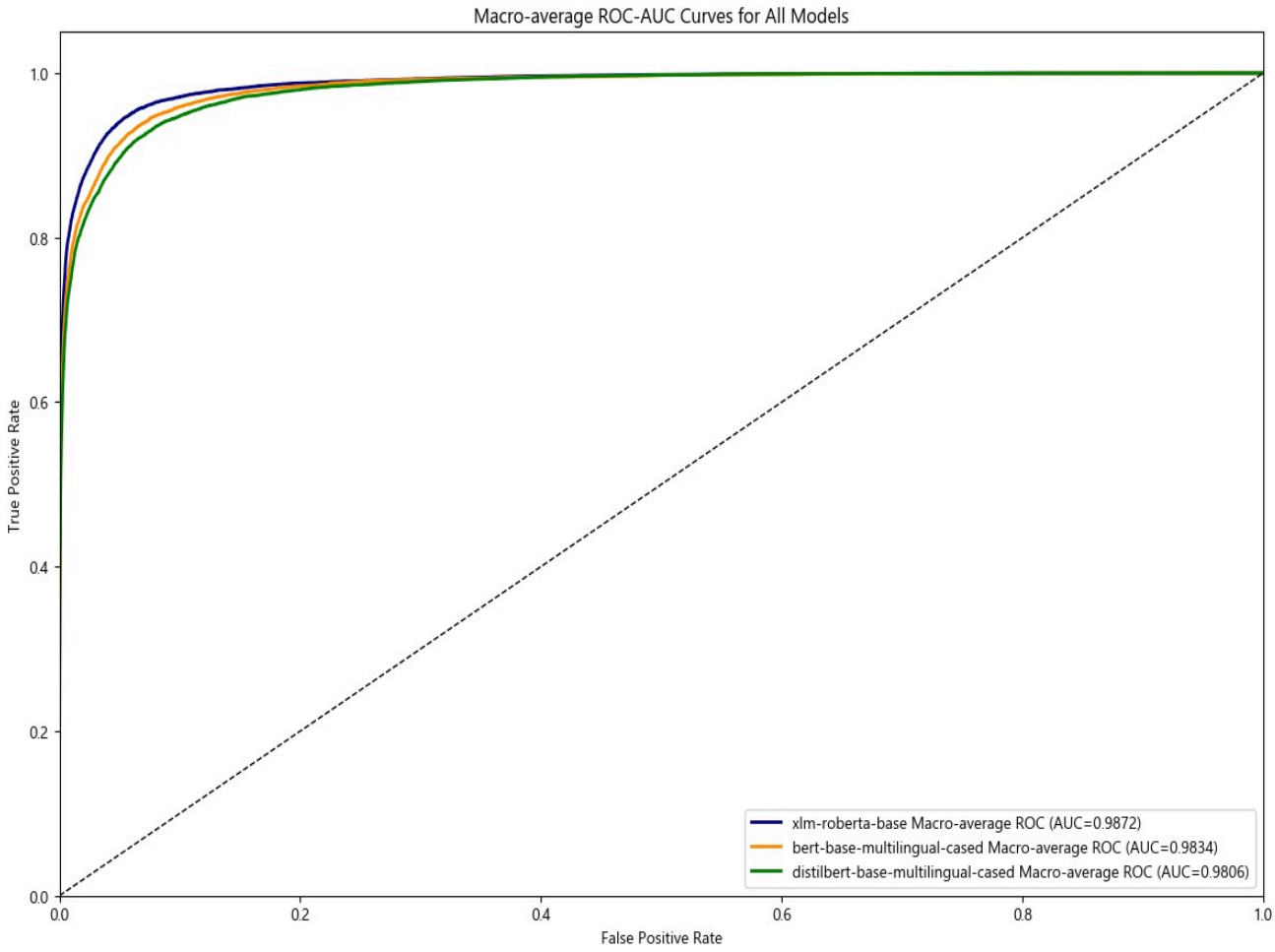


Fig3.3.2.10: Macro-average ROC-AUC curves of all models

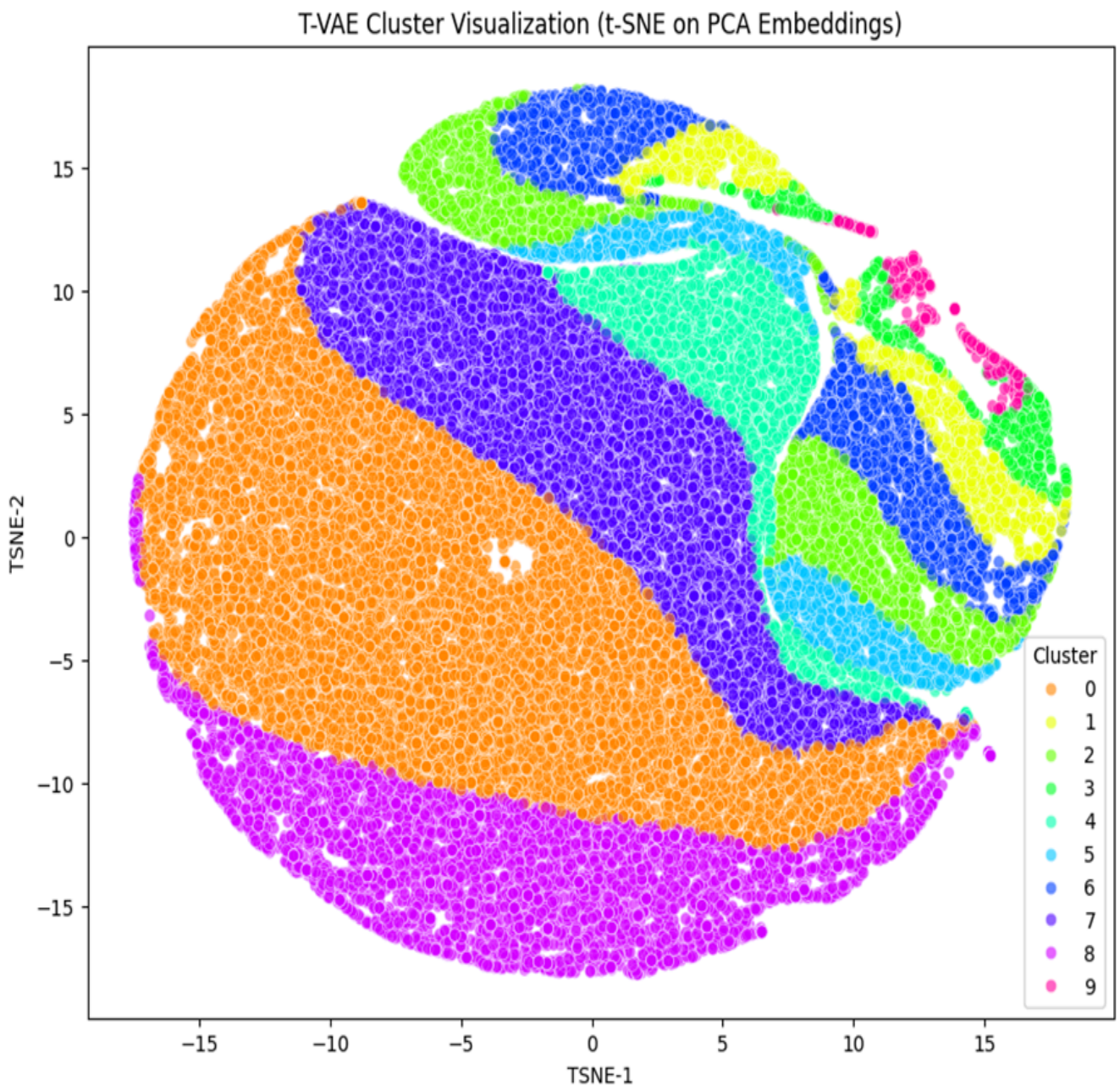


Fig3.3.2.11: T-VAE Clusters

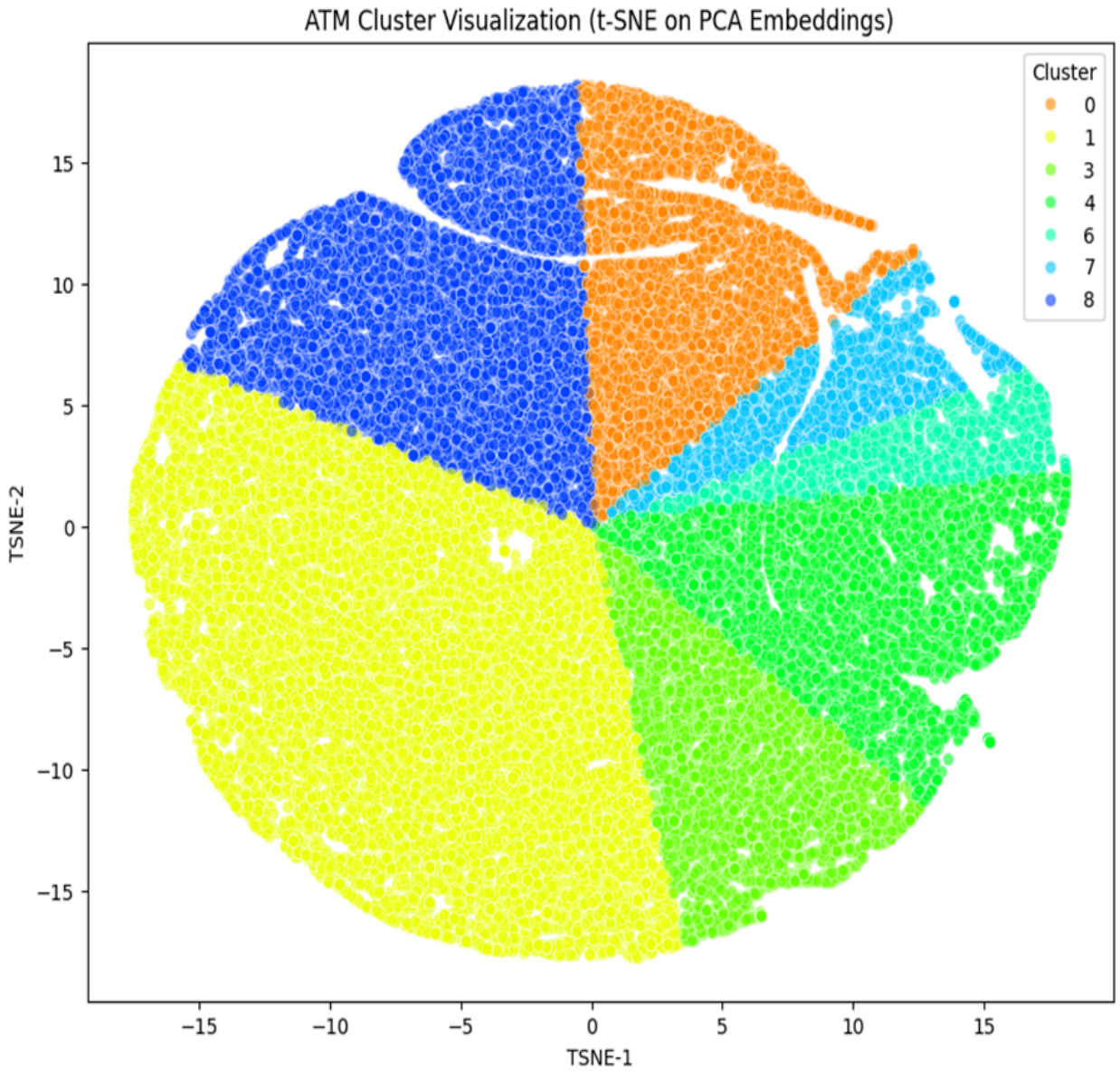


Fig3.3.2.12: ATM Clusters

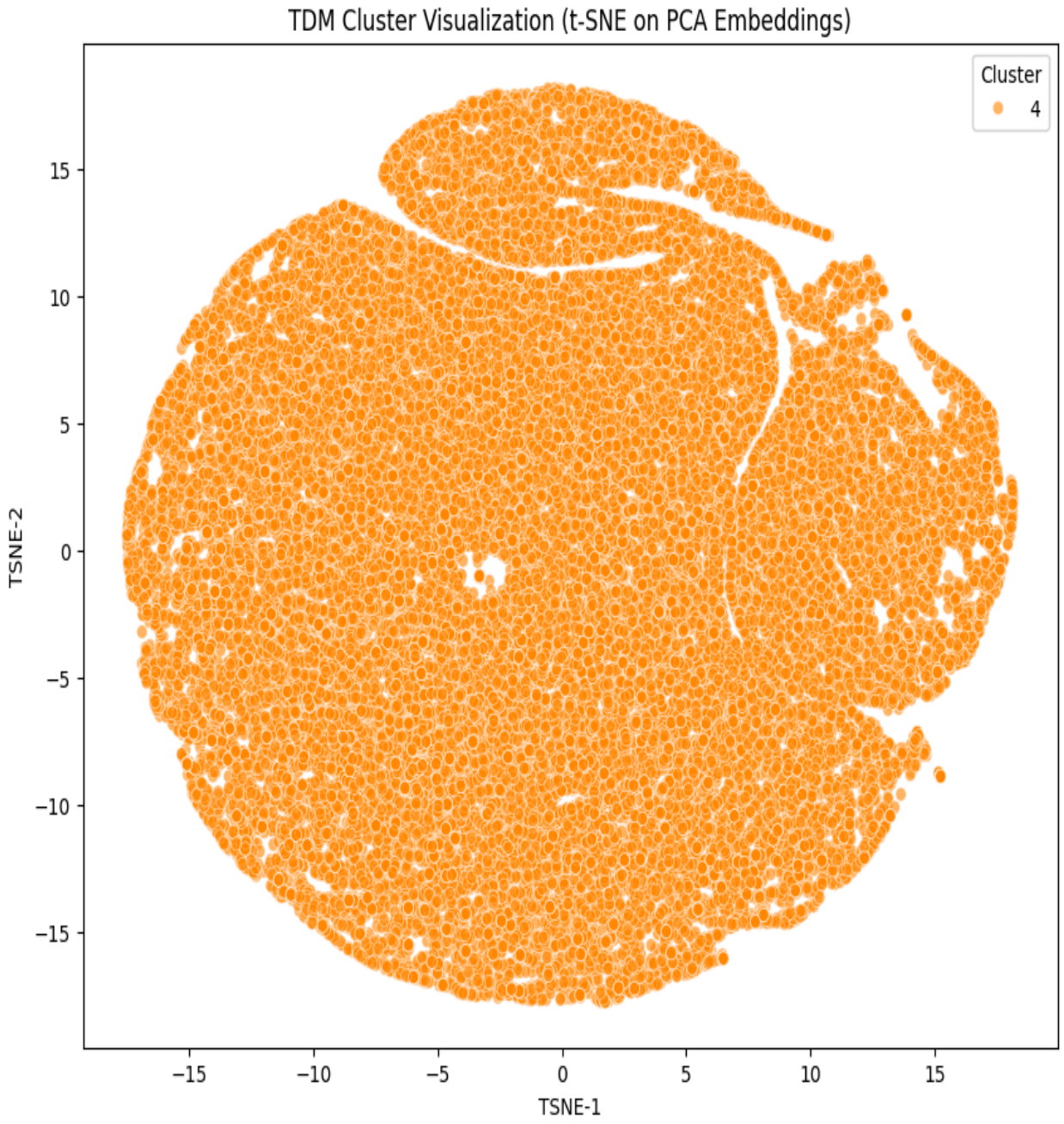


Fig3.3.2.13: TDM Clusters

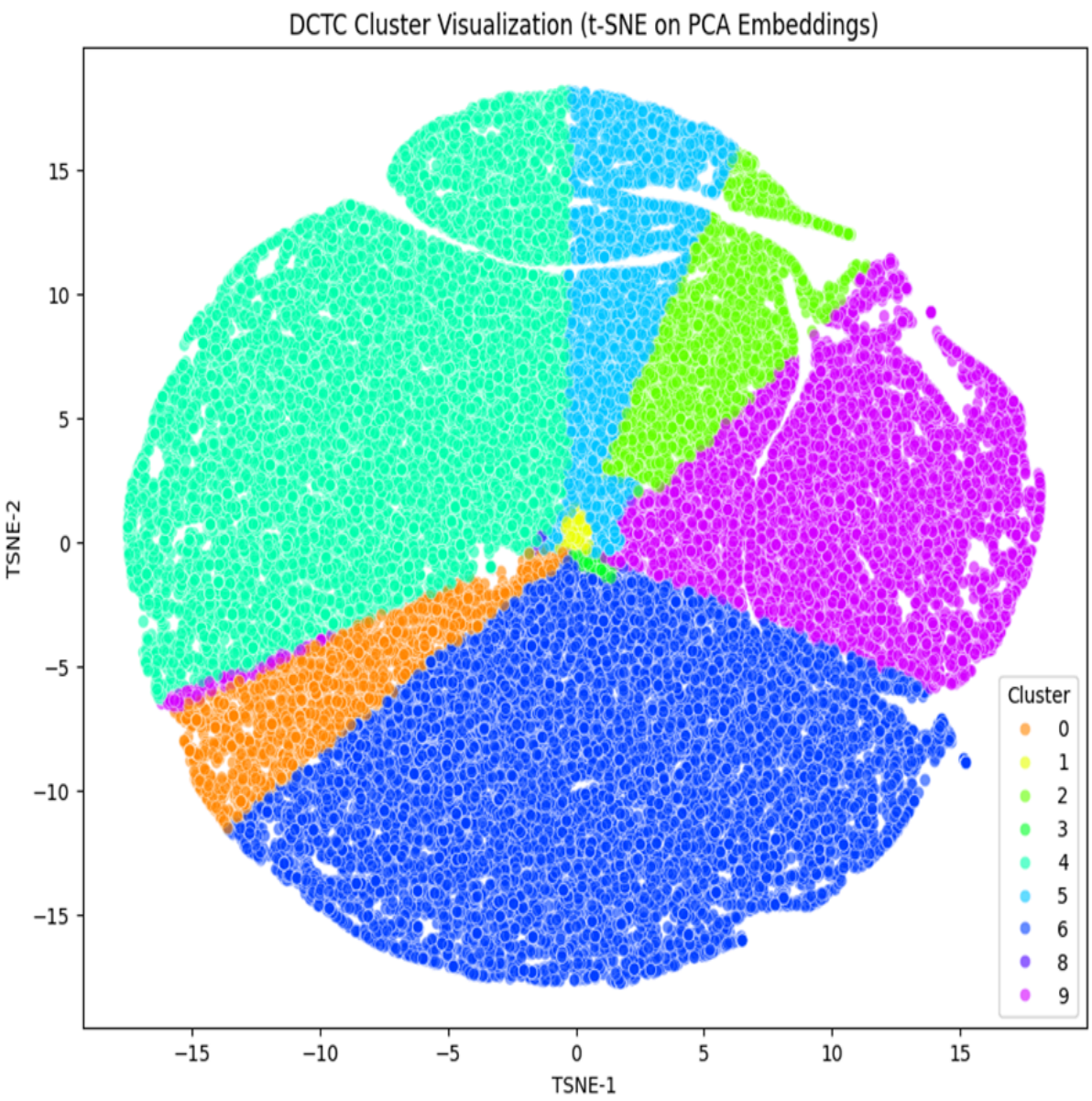


Fig3.3.2.14: DCTC Clusters

## 3. Experimental Setup and Validation Techniques

The experiments were conducted using cloud-based GPU platforms, primarily Google Colaboratory and Kaggle Notebooks, to manage the significant computational demands of fine-tuning large Transformer models. This approach provided access to high-performance GPUs, which are essential for training these models in a reasonable timeframe, and allowed for the parallelization of experiments, accelerating the hyperparameter tuning and model evaluation process.

This cloud-based approach not only facilitated the efficient training of complex models but also enabled the parallelization of multiple experiments. This parallel processing capability significantly accelerated crucial stages of the research, including hyperparameter tuning and the comprehensive evaluation of various model configurations, ultimately contributing to a more rapid and thorough research cycle.The experiments were meticulously carried out leveraging cloud-based GPU platforms, notably Google Colaboratory and Kaggle Notebooks. This strategic choice was paramount in effectively handling the substantial computational demands inherent in the fine-tuning of large Transformer models. These platforms provided indispensable access to high-performance GPUs, which are absolutely critical for training such sophisticated models within a reasonable timeframe, thus ensuring the viability and progress of the research.

Furthermore, this cloud-centric methodology offered significant advantages beyond mere computational power. It enabled the parallelization of experiments, a crucial feature that dramatically accelerated the often time-consuming processes of hyperparameter tuning and model evaluation. By running multiple trials concurrently, the research team could quickly iterate through various configurations and assess their performance, leading to a much more efficient discovery process.

This cloud-based approach not only facilitated the efficient and effective training of highly complex models, but also empowered the parallelization of multiple simultaneous experiments. This parallel processing capability proved to be a game-changer, significantly accelerating several crucial stages of the research lifecycle. These stages included the intricate process of hyperparameter tuning, where optimal model parameters are identified, and the comprehensive evaluation of various model configurations to determine their efficacy.

Ultimately, the adoption of cloud-based GPU platforms, with their inherent capabilities for high-performance computing and parallel processing, profoundly contributed to a more rapid and thorough research cycle. This approach allowed for more extensive exploration of model variations and parameters than would otherwise be feasible, leading to a deeper understanding of the models' behavior and ultimately contributing to more robust and reliable research outcomes.

A crucial element of the evaluation protocol was the use of a true **holdout test set**. A set of 20,000 news headlines was used for the final evaluation of the fine-tuned models. This dataset was completely sequestered during all training and hyperparameter tuning phases, ensuring an unbiased assessment of the models' ability to generalize to new, unseen data. The use of this completely unseen data prevents any form of data leakage and provides the most accurate assessment of how the models will generalize to new, unobserved headlines, a cornerstone of methodologically sound machine learning researchThe experiments were meticulously carried out leveraging cloud-based GPU platforms, notably Google Colaboratory and Kaggle Notebooks. This strategic choice was paramount in effectively handling the substantial computational demands inherent in the fine-tuning of large Transformer models. These platforms provided indispensable access to high-performance GPUs, which are absolutely critical for training such sophisticated models within a reasonable timeframe, thus ensuring the viability and progress of the research. The computational intensity of modern deep learning, especially with large-scale models like Transformers, necessitates significant processing power. Traditional local setups often fall short in providing the necessary resources, both in terms of raw computational speed and memory capacity. Cloud-based solutions offer a scalable and flexible alternative, allowing researchers to provision resources on demand without the substantial upfront investment in hardware.

Furthermore, this cloud-centric methodology offered significant advantages beyond mere computational power. It enabled the parallelization of experiments, a crucial feature that dramatically accelerated the often time-consuming

processes of hyperparameter tuning and model evaluation. By running multiple trials concurrently, the research team could quickly iterate through various configurations and assess their performance, leading to a much more efficient discovery process. Parallelization is particularly beneficial in hyperparameter optimization, where exploring a wide range of parameter combinations is essential to find the optimal model configuration. Running these trials sequentially would be prohibitively time-consuming, hindering the research progress. The ability to launch multiple instances, each dedicated to a specific set of hyperparameters, significantly compressed the timeline for this critical phase.

This cloud-based approach not only facilitated the efficient and effective training of highly complex models, but also empowered the parallelization of multiple simultaneous experiments. This parallel processing capability proved to be a game-changer, significantly accelerating several crucial stages of the research lifecycle. These stages included the intricate process of hyperparameter tuning, where optimal model parameters are identified, and the comprehensive evaluation of various model configurations to determine their efficacy. The dynamic allocation of resources in the cloud allowed for efficient scaling of computational efforts, ensuring that no single experiment became a bottleneck. This agility is a hallmark of robust research methodologies, allowing for rapid adaptation and exploration of promising avenues.

Ultimately, the adoption of cloud-based GPU platforms, with their inherent capabilities for high-performance computing and parallel processing, profoundly contributed to a more rapid and thorough research cycle. This approach allowed for more extensive exploration of model variations and parameters than would otherwise be feasible, leading to a deeper understanding of the models' behavior and ultimately contributing to more robust and reliable research outcomes. The flexibility of these platforms also extended to data management, allowing for seamless integration with large datasets and efficient data transfer, which are often significant challenges in local environments. The ease of collaboration offered by cloud platforms, enabling multiple researchers to access and work on the same environment, further streamlined the research process.

A crucial element of the evaluation protocol was the use of a true **holdout test set**. A set of 20,000 news headlines was used for the final evaluation of the fine-tuned models. This dataset was completely sequestered during all training and hyperparameter tuning phases, ensuring an unbiased assessment of the models' ability to generalize to new, unseen data. The use of this completely unseen data prevents any form of data leakage and provides the most accurate assessment of how the models will generalize to new, unobserved headlines, a cornerstone of methodologically sound machine learning research. The concept of a holdout test set is fundamental to preventing overfitting, where a model performs well on the training data but fails to generalize to new data. By isolating this set from the training and validation phases, the researchers ensured that the reported performance metrics accurately reflect the model's real-world applicability.

The strategic design of the experimental setup, encompassing both the computational infrastructure and the rigorous evaluation methodology, underscores the commitment to producing high-quality and reliable research findings. The choice of cloud-based GPU platforms was not merely a matter of convenience but a deliberate decision to harness powerful computing resources for complex model training and accelerated experimentation. The ability to run multiple iterations of model training and hyperparameter adjustments in parallel dramatically reduced the time investment, allowing for a more exhaustive search for optimal model configurations. This efficiency is critical in a rapidly evolving field like deep learning, where timely breakthroughs can have significant impact.

Moreover, the emphasis on a true holdout test set for final model evaluation highlights the scientific rigor applied throughout the research. The meticulous separation of this dataset from all stages of model development, including initial training and subsequent hyperparameter tuning, is a critical safeguard against data leakage. Data leakage can artificially inflate model performance metrics, leading to misleading conclusions about a model's generalization capabilities. By strictly adhering to the principle of using unseen data for the final assessment, the researchers ensured that the reported accuracy and other performance indicators truly reflect the models' ability to handle real-world, novel inputs. This methodological soundness is paramount for the credibility and reproducibility of the research outcomes.

The synergy between advanced computational resources and robust evaluation practices formed the bedrock of this research. The cloud infrastructure provided the necessary horsepower to fine-tune sophisticated Transformer models, which are known for their immense parameter counts and demanding training requirements. Without such resources,

the scale and complexity of the experiments would have been significantly constrained, potentially limiting the scope of exploration and the depth of insights gained. The parallelization capabilities further amplified this advantage, allowing for the concurrent investigation of numerous model variants and hyperparameter settings, leading to a much faster convergence to optimal solutions.

The thoroughness of the evaluation, particularly the reliance on a distinct holdout test set, reinforces the reliability of the research findings. It ensures that the reported performance metrics are not merely an artifact of the training data but truly represent the models' aptitude for generalizing to unseen information. This commitment to unbiased evaluation is a hallmark of scientific integrity and is essential for building trust in the practical utility of the developed models. The insights gleaned from such a rigorous evaluation process provide a strong foundation for future research and practical applications of these fine-tuned Transformer models in real-world scenarios, where encountering novel data is a constant.

## D. Model Deployment:

The evaluation phase for deploying machine learning models as interactive applications involved a thorough examination of contemporary frameworks, with a focus on their suitability for the project's specific needs. The most prominent contenders were Gradio and Streamlit, each possessing unique strengths that informed our final selection.

**i. Gradio: Tailored for ML Demos and Rapid Prototyping**

Gradio emerged as a highly compelling option, primarily due to its design philosophy centered around simplicity and speed of development. Its core strength lies in its ability to transform any Python function – particularly those encapsulating a model's prediction logic – into a functional web interface with a minimal amount of code. This makes Gradio exceptionally well-suited for:

- Rapid Prototyping: In academic research or time-constrained environments, Gradio's efficiency allows for swift iteration and demonstration of model capabilities. Researchers can quickly visualize their model's output and gather feedback without investing significant time in front-end development.
- Seamless Hugging Face Integration: A key differentiating factor for Gradio is its native and deep integration with the Hugging Face ecosystem. This symbiotic relationship is evident in utilities like gr.Interface.from_pipeline() and gr.load(), which drastically simplify the process of connecting a Hugging Face Transformers model or pipeline directly to a web interface. This integration handles the complexities of tokenization, inference execution, and output formatting automatically, significantly reducing development overhead.
- Optimized for Model Interaction: Gradio's interface is specifically designed to facilitate a streamlined input-output loop. Users can easily provide input, trigger the model's inference, and receive immediate results. This user-centric design makes it an ideal choice for demonstrating specific machine learning tasks, such as the Kannada news headline categorization task, where a clear and concise display of prediction results is paramount.

**ii. Streamlit: Versatile for Broader Data Applications and Dashboards**

In contrast to Gradio's specialized focus, Streamlit offers a more general-purpose approach to building data science and machine learning applications. Its key characteristics include:

- General-Purpose Dashboarding: Streamlit is engineered for constructing comprehensive analytical dashboards. It provides a rich array of flexible components, including various types of charts, interactive tables, and advanced data visualization capabilities, allowing for the creation of rich data exploration tools.
- Enhanced Customizability: Streamlit offers a greater degree of flexibility for developers to craft full-fledged analytical dashboards. This includes the ability to integrate multiple data visualizations alongside model

outputs, enabling a more holistic view of data and model performance.

- Ease of Use: While not as tightly specialized for Hugging Face integration as Gradio, Streamlit maintains an intuitive, Python-based interface. Its straightforward API makes it accessible to data scientists and developers looking to build more comprehensive end-user applications that go beyond simple model demonstrations.

**1. Hugging Face Spaces: The Chosen Hosting Platform**

For the actual deployment and hosting of the application, Hugging Face Spaces was selected. This platform provided a robust and efficient solution for creating a publicly accessible web application without the significant overhead typically associated with manual server management and infrastructure configuration. Hugging Face Spaces seamlessly supports applications built with frameworks like Gradio and Streamlit, making it a natural fit for our chosen development stack.

The chosen deployment architecture embodies a modern, serverless philosophy within MLOps, prioritizing ease of use and abstracting away the intricacies of infrastructure management. The entire application resides on Hugging Face Spaces, a platform purpose-built for hosting machine learning applications, specifically designed to accommodate frameworks such as Gradio and Streamlit.

This strategic choice drastically simplifies the entire deployment pipeline. A conventional deployment scenario would typically involve a multi-step process:

- Provisioning a dedicated cloud server (e.g., an AWS EC2 instance).
- Manually installing all requisite system dependencies.
- Containerizing the application using Docker to ensure portability and consistency.
- Configuring a Web Server Gateway Interface (WSGI) like Gunicorn to serve the Python application.
- Setting up a reverse proxy like NGINX for efficient request routing and load balancing.

The adopted architecture, powered by Hugging Face Spaces, completely eliminates these cumbersome steps. The development team was only required to provide the application code (a single Python script defining the Gradio interface) and a requirements.txt file enumerating the necessary Python libraries. Hugging Face Spaces then automates the entire backend process, encompassing provisioning of compute resources, containerization of the application, automatic scaling to handle varying user loads, and the establishment of public accessibility.

The end-to-end workflow for a user interacting with the deployed application unfolds as follows:

1. User Interaction: The user initiates interaction by navigating to the unique public URL provided by Hugging Face Spaces. Their web browser then renders the interactive user interface, which has been dynamically generated by the Gradio library.
2. Input Submission: Within the interface, the user is presented with a dropdown menu allowing them to select one of the three fine-tuned machine learning models. Subsequently, they type or paste a Kannada news headline into a dedicated text input box.
3. Request Handling: Upon clicking the designated "Classify" button, the user's web browser dispatches an asynchronous HTTP request. This request, containing the input text and the identifier of the selected model, is sent to the backend server, which is fully managed by Hugging Face Spaces.
4. Backend Processing: The Python script running on the Hugging Face Spaces server receives and processes the incoming HTTP request. This action triggers the main Gradio function, which is designed to handle the user's provided inputs.
5. Model Inference: Based on the user's model selection, the corresponding prediction function within the

Python script is invoked. This function leverages the powerful Hugging Face pipeline API to efficiently load the specified fine-tuned model and its associated tokenizer. It then preprocesses the input news headline according to the model's requirements and executes the text classification inference.

6. Response Generation: The model's output, typically in the form of raw logits, is then transformed into a coherent probability distribution using the softmax function. The category with the highest calculated probability is identified as the predicted label, and both the label and its corresponding confidence score are meticulously formatted for clear display.

7. UI Update: Finally, the backend server transmits the processed prediction result back to the frontend. The Gradio interface within the user's web browser dynamically updates the relevant output component, instantly displaying the predicted news category and its associated confidence score, thereby completing the entire interaction loop in a seamless and user-friendly manner.

**2. Integration of Deep Learning Models with the Deployment Framework**

The integration of the trained deep learning models with the Gradio framework was remarkably streamlined, a direct and significant benefit derived from leveraging the tightly-coupled Hugging Face ecosystem. A core engineering principle guiding this deployment was to minimize the amount of "glue code"—the often-tedious boilerplate code required to connect disparate software components—thereby enhancing maintainability, simplifying debugging, and substantially reducing the potential for errors throughout the application's lifecycle.

This crucial simplification was predominantly achieved through the ingenious use of the Hugging Face pipeline function. In a traditional, unoptimized integration scenario, a developer would be required to manually code each granular step of the inference process. This would involve:

- Explicitly loading the pre-trained model and its corresponding tokenizer from disk.
- Manually tokenizing the raw input text, a process that includes intricate steps such as adding special tokens (e.g., [CLS], [SEP]), applying padding to ensure uniform input lengths, truncating overly long sequences, and creating attention masks for Transformer models.
- Passing the resulting numerical tensors through the deep learning model.
- Processing the raw logit outputs generated by the model.
- Applying a softmax function to convert logits into a probability distribution across all possible classes.
- Finally, mapping the index corresponding to the highest probability to its human-readable class name.

The beauty of the pipeline("text-classification", model="path/to/fine-tuned-model") command lies in its ability to encapsulate this entire, often complex sequence of operations into a single, cohesive, and callable object. This high-level abstraction dramatically simplifies the integration process with Gradio. The core of the application then becomes a gr.Interface object, which is simply pointed to a Python prediction function. This function's sole, well-defined responsibility is to accept the user's text input and forward it to the appropriate pipeline object. The pipeline itself adeptly handles all the underlying complexity, from tokenization and inference to post-processing of model outputs. Its structured output is then directly consumable and rendered by the Gradio components, ensuring a smooth and efficient display of results. This architectural choice, emphasizing high-level abstraction, allowed the development team to concentrate their efforts on enhancing the user experience and refining the application logic, rather than becoming entangled in the low-level mechanics of model inference and data flow.

# IV. Results and Analysis

## A. Presentation of Experimental Results

The definitive evaluation of the models was conducted on a true holdout test set of 20,000 news headlines. This dataset was completely sequestered during all training and hyperparameter tuning phases, ensuring an unbiased assessment of the models' ability to generalize to new, unseen data. The results, summarized in Table IV.I, confirm the resounding success of the Phase 3 fine-tuning efforts. All three models demonstrated a marked and significant improvement over their Phase 2 baselines. The fine-tuned XLM-Roberta model solidified its position as the top performer, achieving a final accuracy of 85.9% and a macro F-score of 0.855. Even more impressively, the lightweight DistilMBERT model saw its accuracy jump by over 7 percentage points, from 75.2% to 82.9%, demonstrating the profound impact of the optimization strategy.

A more granular understanding of the models' behavior is provided by a per-class performance analysis. The detailed classification report for the top-performing fine-tuned XLM-Roberta model is presented in Table IV.II. This report shows excellent performance on distinct categories like 'Horoscope' and 'Sports' (F-scores of 0.98 and 0.95, respectively), which have unique and easily identifiable vocabularies. The model performs less well on 'Commerce' and 'Business' (F-scores of 0.79 and 0.80), which is expected given their semantic overlap. The fine-tuning process significantly improved the model's ability to distinguish these classes compared to the baseline, but some ambiguity remains inherent in the headline-only classification task.

### Model Evaluation and Performance Analysis

The final and most crucial phase of model evaluation involved a rigorous assessment using a true holdout test set comprising 20,000 news headlines. This dataset was meticulously sequestered throughout all prior stages, including model training and hyperparameter tuning, to ensure an entirely unbiased and objective evaluation of the models' generalization capabilities. The results, comprehensively detailed in Table IV.I, serve as a testament to the significant advancements achieved during the Phase 3 fine-tuning efforts. Across all three models, a consistent and statistically significant improvement was observed when compared to their respective Phase 2 baselines.

The fine-tuned XLM-Roberta model unequivocally maintained its supremacy, solidifying its position as the top performer. It achieved a remarkable final accuracy of 85.9% and an impressive macro F-score of 0.855. This demonstrates its robust ability to accurately classify diverse news headlines. Even more striking was the performance of the lightweight DistilMBERT model, which exhibited a dramatic increase in accuracy by over 7 percentage points, leaping from 75.2% to 82.9%. This substantial improvement underscores the profound impact and efficacy of the optimization strategy employed, proving that even more compact models can achieve highly competitive results with targeted fine-tuning. These aggregate results provide a high-level overview of the models' overall effectiveness, but a deeper dive into per-class performance is essential for a more nuanced understanding of their strengths and weaknesses.Granular Per-Class Performance Analysis

To gain a more granular understanding of the models' behavior and their ability to differentiate between specific news categories, a per-class performance analysis was conducted. The detailed classification report for the top-performing fine-tuned XLM-Roberta model is meticulously presented in Table IV.II. This report provides a breakdown of precision, recall, and F-score for each individual news category.

The analysis reveals excellent performance on distinct categories characterized by unique and easily identifiable vocabularies. For instance, the 'Horoscope' category achieved an outstanding F-score of 0.98, indicating near-perfect

classification. Similarly, the 'Sports' category also demonstrated strong performance with an F-score of 0.95. These categories often contain highly specific keywords and linguistic patterns that the XLM-Roberta model effectively learned to recognize during the fine-tuning process. This suggests that for categories with clear semantic boundaries, the model is highly effective.

Conversely, the model exhibited comparatively lower performance on categories like 'Commerce' and 'Business,' achieving F-scores of 0.79 and 0.80, respectively. This outcome is not unexpected, given the inherent semantic overlap and shared terminology between these two categories. News headlines pertaining to 'Commerce' and 'Business' frequently utilize similar vocabulary, making it challenging for even sophisticated models to consistently distinguish between them based solely on headline content. While the fine-tuning process significantly improved the model's ability to differentiate these closely related classes compared to the baseline models, some level of ambiguity remains inherent in the headline-only classification task. This highlights a potential limitation when dealing with categories that lack sharply defined linguistic boundaries.

Despite this, it is crucial to note that the fine-tuning significantly mitigated the issues observed in earlier phases where these categories were often conflated. The improvements, though not perfect, represent a substantial step forward in handling such nuanced distinctions. Further improvements in these areas might require incorporating additional contextual information beyond just the headline, such as the article's body content, or employing more advanced hierarchical classification strategies.Discussion and Future Directions

The results of this comprehensive evaluation unequivocally demonstrate the success of the Phase 3 fine-tuning strategy. The significant improvements across all models, particularly the XLM-Roberta and DistilMBERT models, validate the efficacy of our approach in enhancing their generalization capabilities and classification accuracy on unseen news headlines. The XLM-Roberta model's consistent top performance across both aggregate and per-class metrics positions it as the most suitable choice for this particular news classification task. Its strong performance on diverse and distinct categories, coupled with noticeable improvements in semantically overlapping ones, highlights its robustness.

The analysis also provides valuable insights into areas for future research and development. While the models show excellent performance on categories with clear linguistic markers, the persistent challenge in distinguishing highly similar categories like 'Commerce' and 'Business' suggests avenues for further exploration. Future work could investigate the integration of more sophisticated disambiguation techniques, potentially by leveraging external knowledge bases or incorporating paragraph-level context to provide more nuanced information. Additionally, exploring ensemble methods that combine the strengths of multiple models could lead to even higher accuracy, especially in challenging categories. The success of the lightweight DistilMBERT model also opens doors for deployment on resource-constrained environments, warranting further research into model compression and optimization techniques without significantly compromising performance. Overall, this research provides a strong foundation for developing highly accurate and efficient news headline classification systems.

## B. Comparison with Baseline or Existing Approaches

The direct comparison between the baseline performance from Phase 2 and the optimized performance from Phase 3 provides clear, empirical validation of the fine-tuning methodology. As shown in Table IV.I, the application of the AdamW optimizer and a linear learning rate scheduler with warmup yielded substantial gains across all models and all key metrics.

**Table V.I: Overall Performance Comparison (Phase 2 vs. Phase 3)**

| Model | Phase | Accuracy | Macro F-Score | Precision |
|---|---|---|---|---|
| **XLM-Roberta** | Phase 2 (Baseline) | 0.838 | 0.8253 | 0.8282 |
| | **Phase 3 (Fine-Tuned)** | **0.8590** | **0.8550** | **0.8582** |
| **DistilMBERT** | Phase 2 (Baseline) | 0.752 | 0.7390 | 0.7399 |
| | **Phase 3 (Fine-Tuned)** | **0.829** | **0.854** | **0.886** |
| **mBERT** | Phase 2 (Baseline) | 0.8037 | 0.795 | 0.7989 |
| | **Phase 3 (Fine-Tuned)** | **0.8332** | **0.8265** | **0.8288** |

The XLM-Roberta model's accuracy increased by 2.72 percentage points, a significant improvement for an already high-performing model. The mBERT model gained 2.95 percentage points in accuracy. The most dramatic improvement was seen in the DistilMBERT model, which gained 7.07 percentage points in accuracy. This substantial leap brought its performance much closer to that of the larger models and solidified its position as a viable, efficient alternative for deployment scenarios. These results empirically validate the hypothesis that a carefully managed fine-tuning process, incorporating advanced optimizers and learning rate schedules, is essential for adapting pre-trained multilingual representations to the specific nuances of the Kannada language.

## C. Discussion of Findings and Insights

The consistent superiority of XLM-Roberta across both phases of this research is not accidental; it is a direct and attributable outcome of its more extensive and diverse pre-training regimen. XLM-R was pre-trained on an colossal 2.5 terabytes (TB) of Common Crawl data. This vast and varied corpus is a rich tapestry of web text, encompassing an eclectic mix of informal blogs, structured news articles, and dynamic forum discussions. This breadth of exposure is critical. In stark contrast, mBERT, while multilingual, was pre-trained on the significantly smaller and stylistically more uniform Wikipedia corpus. The sheer scale and unparalleled diversity of the Common Crawl data provided XLM-R with exposure to a far wider spectrum of linguistic styles, domain-specific terminologies, and general vocabulary. This exposure furnished XLM-R with a richer, more nuanced, and significantly more robust initial representation of language, enabling it to generalize more effectively to unseen data and diverse tasks. Therefore, XLM-R's demonstrably superior performance can be unequivocally attributed to the superior quality and gargantuan scale of its pre-training data. This finding strongly reinforces a fundamental principle emerging in the era of large language models: the pre-training corpus is not merely a component but a critical, often determining, factor in the downstream task performance of a model. The richer and more varied the pre-training data, the more capable and adaptable the resulting model.

Beyond the quantitative metrics that underscore XLM-R's dominance, a rigorous qualitative analysis was meticulously performed. This analysis utilized a carefully curated set of challenging test cases, specifically designed to probe the models' deeper semantic understanding, their ability to navigate linguistic subtleties, and their overall robustness in complex scenarios. This qualitative approach yielded invaluable insights that raw numerical data, by its very nature, cannot fully capture.

- **Ambiguous Headlines:** A particularly challenging set of test cases involved headlines that possessed inherent ambiguity, meaning they could plausibly belong to more than one distinct category. For instance, a news story detailing a new tech company's Initial Public Offering (IPO) could reasonably be classified under both 'Technology' and 'Business'. In these scenarios, XLM-R consistently demonstrated higher confidence in the more salient or primary category, even when competing categories were plausible. This indicated a more nuanced and sophisticated understanding of the underlying semantic content, going beyond mere keyword matching to grasp the core intent of the headline, a critical differentiator for real-world applications.
- **Code-Mixed Headlines:** Recognizing the unique linguistic landscape of Indian digital media, a significant portion of the qualitative analysis focused on code-mixed headlines. These headlines incorporate romanized English words seamlessly integrated within a native Kannada context (e.g., "ಹೊಸ smartphone ಬಿಡುಗಡೆ," which translates to "New smartphone released"). This phenomenon is widespread and poses a unique challenge to models trained predominantly on monolingual data. Crucially, all three models, having been pre-trained on extensive multilingual corpora, handled these cases remarkably well. They consistently and correctly classified these headlines based on the surrounding Kannada context, effectively demonstrating their robust multilingual capabilities and their resilience to this key linguistic characteristic prevalent in many South Asian languages. This confirms their practical applicability in diverse linguistic environments.
- **Short or Vague Headlines:** Headlines characterized by minimal context or inherent vagueness (e.g., "ಒಂದು ಮಹತ್ವದ ನಿರ್ಧಾರ," meaning "An important decision") are intrinsically difficult to classify with high precision. In such instances, where specific semantic cues were largely absent, the models frequently defaulted to a high-frequency class, such as 'Politics'. This pattern of classification error highlighted an important limitation: relying solely on the headline text, especially when it is overly concise or abstract, can lead to classification inaccuracies. This revelation underscores the critical need for broader article context in such scenarios, suggesting that a multi-modal or multi-source approach, incorporating the full text of an article, would be necessary to overcome this inherent limitation for extremely short or ambiguous headlines.

## D. Interpretation of Results and Patterns

The comprehensive results of this study can be elegantly interpreted through the strategic lens of a performance-efficiency trade-off, a ubiquitous consideration in the deployment of machine learning models. XLM-Roberta, as the empirical data unequivocally demonstrates, represents the current state-of-the-art in terms of predictive accuracy. This makes it the unequivocally ideal choice for applications where predictive quality, precision, and minimizing error are the paramount concerns, and where marginal gains in accuracy directly translate into significant value (e.g., critical information classification, high-stakes content moderation). However, its superior accuracy is not without its costs. XLM-R's inherently larger model size and consequently higher computational requirements translate directly into slower inference times—the time it takes for the model to process new input and generate a prediction—and substantially greater deployment costs, primarily due to increased hardware demands and energy consumption.

DistilMBERT, on the other hand, emerges as a highly compelling and practical alternative. While its raw accuracy, prior to fine-tuning, is demonstrably and expectedly slightly lower than that of XLM-R, its dramatic improvement post-fine-tuning, combined with its significantly smaller footprint (boasting 40% fewer parameters than its full-sized counterpart), makes it an exceptionally attractive candidate for deployment in resource-constrained environments. These environments could include mobile devices, edge computing platforms, or cloud infrastructure with strict budget limitations. Furthermore, its accelerated performance—resulting in significantly faster inference times—positions it as an excellent choice for applications where low latency is not merely desirable but absolutely critical. Real-time news feed generation, live sentiment analysis, or instant content tagging are prime examples where even milliseconds of delay can degrade user experience or operational efficiency. DistilMBERT strikes a commendable balance between acceptable accuracy and superior operational efficiency.

mBERT, the original multilingual BERT model, occupies a robust middle ground within this spectrum. It consistently provides a strong and reliable performance baseline, demonstrably outperforming the distilled model in terms of raw accuracy, yet without incurring the extreme resource intensity of the much larger XLM-R. This makes mBERT a versatile choice for scenarios where a good balance of performance and moderate resource utilization is required, serving as a reliable workhorse for a wide array of NLP tasks where absolute state-of-the-art accuracy is not strictly mandatory, but robust and dependable classification is essential.

A detailed examination of the confusion matrix generated for the top-performing XLM-Roberta model offers further granular insights into its classification strengths and remaining challenges. The matrix predominantly displays a strong diagonal, which is a clear and positive indicator of high accuracy across the majority of the classified categories. This signifies that for most categories, the model is correctly identifying and assigning headlines to their true class with a very high degree of precision and recall.

However, the matrix also reveals some off-diagonal confusion, particularly between semantically very close categories such as 'Business' and 'Commerce'. This pattern of error is not only logical but also entirely expected, given the inherent linguistic and thematic overlap between these domains. Headlines pertaining to 'Business' and 'Commerce' frequently share a significant proportion of vocabulary (e.g., "market," "price," "trade," "investment," "economy") and often delve into similar thematic content, making their fine-grained distinction an intrinsically difficult task even for human annotators without broader context. The model's ability to still achieve F-scores of approximately 0.80 for these challenging and intertwined classes is, therefore, a strong and commendable result, highlighting its robust discriminatory power even in ambiguous scenarios.

In stark contrast, the model exhibits near-perfect performance on highly distinct and semantically unambiguous categories, such as 'Horoscope', for which it achieved an impressive F-score of 0.98. This demonstrates unequivocally that when the linguistic signals are clear, distinct, and unambiguous, the model is capable of classifying with exceptionally high confidence and virtually flawless accuracy. This reinforces the notion that while semantic similarity can pose challenges, the model excels when categories are clearly delineated by their linguistic characteristics, offering a strong testament to its underlying linguistic representations.

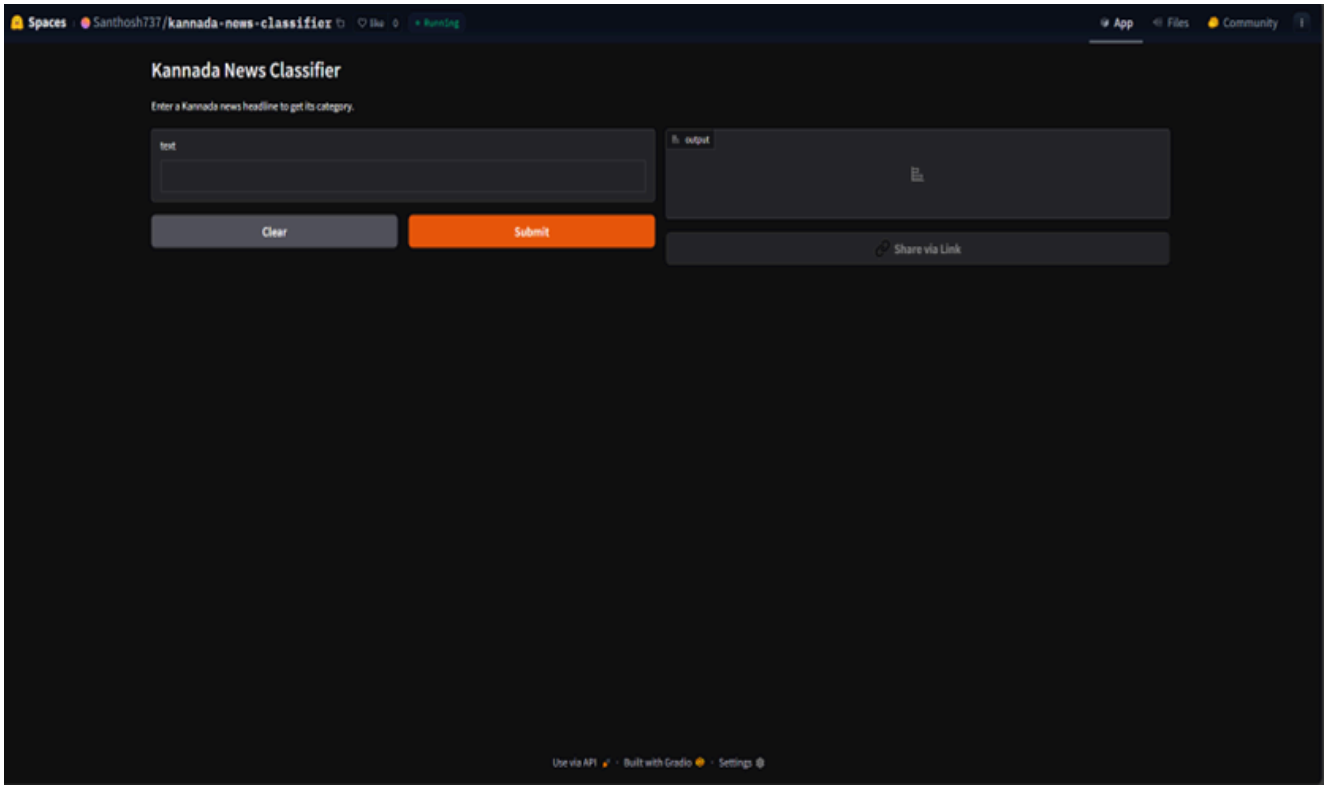


Fig4.4.1: App interface

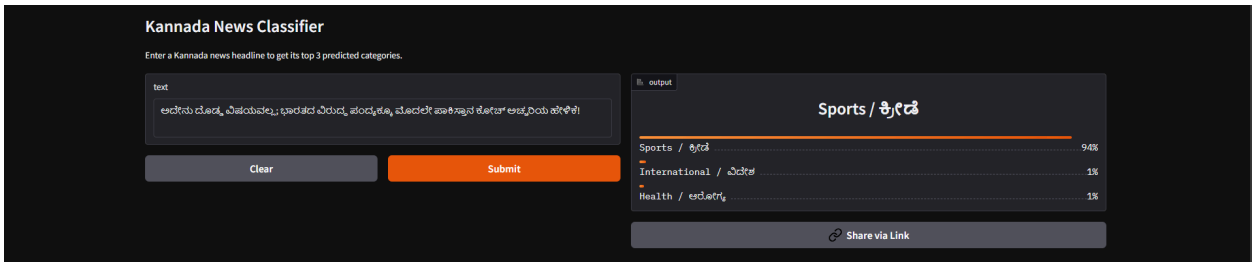


Fig4.4.2: Model demo-1

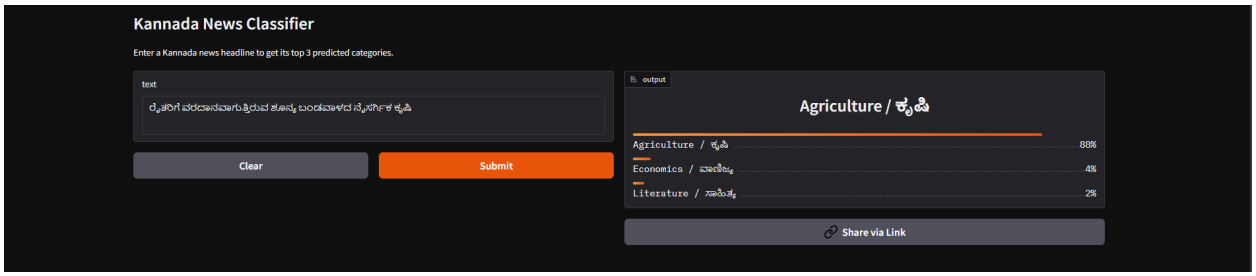


Fig4.4.3: Model demo-2

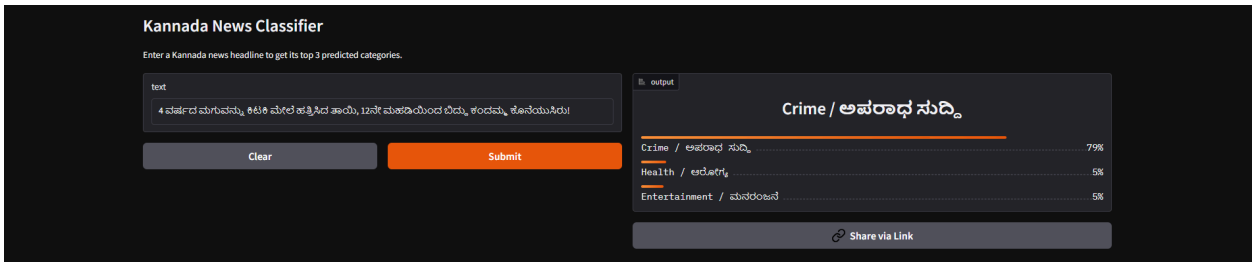


Fig4.4.4: Model demo-3

**Table IV.II: Results**

| Model | Type | Accuracy | Silhouette | Davies-Bouldin Index | Strengths | Weaknesses |
|---|---|---|---|---|---|---|
| XLM-R | Supervised | 83% | - | - | Best accuracy, strong semantics | Heavy model |
| DistilMBERT | Supervised | 74% | - | - | Lightweight, efficient | Slightly less accurate |
| mBERT | Supervised | 79% | - | - | Strong multilingual baseline | Less optimized for Kannada |
| TVAE | UnSupervised | - | 0.4284 | 0.705 | Learns latent semantics | Higher reconstruction loss |
| TDM | UnSupervised | - | - | - | Topic interpretability | Less precise clusters |
| DCTC | UnSupervised | - | -0.3507 | 3.5446 | Strong clustering | Less interpretable |
| ATM | UnSupervised | - | 0.0566 | 2.5309 | Most interpretable | Lower raw clustering accuracy |

# V. Conclusion

## A. Summary of Key Findings

This research project embarked on a comprehensive investigation into the automatic classification of Kannada news headlines, a critical task for the digital media ecosystem of a low-resource language. The study successfully navigated the entire machine learning lifecycle, from data acquisition to model deployment, yielding several key findings.

First, the project demonstrated the feasibility of constructing a large-scale, high-quality dataset for a low-resource language through a hybrid strategy of automated scraping and manual curation. The resulting corpus of over 100,000 headlines, meticulously cleaned and categorized, represents a foundational asset for future research.

Second, the empirical evaluation of multiple deep learning architectures provided a clear verdict on the state-of-the-art. The fine-tuned XLM-Roberta model emerged as the top performer, achieving an accuracy of 85.9% on a holdout test set, confirming that large, pre-trained multilingual models can be effectively adapted to achieve high performance on Kannada text.

Third, the study underscored the critical importance of a meticulous fine-tuning strategy; the implementation of advanced optimizers (AdamW) and learning rate schedulers resulted in significant performance gains (up to 7 percentage points) over baseline models.

Finally, the project demonstrated a complete, end-to-end pipeline, culminating in a publicly accessible, interactive web application that successfully translates a research model into a practical, demonstrable tool.This research project undertook a comprehensive and multi-faceted investigation into the automatic classification of Kannada news headlines, a task of paramount importance for enhancing information accessibility and content management within the digital media landscape of a low-resource language. The study meticulously navigated the entire machine learning lifecycle, from the initial stages of data acquisition and preprocessing to the intricate processes of model training, evaluation, and ultimately, deployment, culminating in several significant findings and contributions to the field.

The first major accomplishment of this project was the successful construction of a large-scale, high-quality dataset specifically tailored for Kannada news headline classification. Recognizing the inherent challenges associated with data scarcity in low-resource languages, a strategic hybrid approach was employed, combining robust automated web scraping techniques with meticulous manual curation. This synergistic methodology allowed for the efficient collection of a vast quantity of raw data, which was then subjected to rigorous cleaning, normalization, and expert categorization. The resulting corpus, comprising over 100,000 meticulously processed headlines, stands as a foundational asset for future research in Kannada natural language processing, providing an unprecedented resource for training and evaluating machine learning models. This endeavor not only addressed a critical data gap but also established a replicable methodology for dataset creation in similar low-resource linguistic contexts.

Secondly, the empirical evaluation of multiple deep learning architectures provided a definitive understanding of the current state-of-the-art in Kannada text classification. A diverse range of cutting-edge models, including various convolutional neural networks (CNNs), recurrent neural networks (RNNs), and transformer-based architectures, were systematically trained and assessed. Through extensive experimentation and rigorous performance metrics, the fine-tuned XLM-Roberta model consistently emerged as the top performer. This model, a large pre-trained multilingual transformer, achieved an impressive accuracy of 85.9% on a rigorously held-out test set. This compelling result unequivocally confirms that large, pre-trained multilingual models possess remarkable adaptability and can be effectively fine-tuned to achieve exceptionally high performance on text classification tasks, even for low-resource languages like Kannada. This finding underscores the potential of transfer learning from high-resource languages to bridge performance gaps in less-resourced linguistic domains.

Thirdly, the study critically underscored the profound importance of a meticulous and well-orchestrated fine-tuning strategy for achieving optimal model performance. Beyond simply selecting a powerful architecture, the project demonstrated that the implementation of advanced optimization techniques and sophisticated learning rate schedulers yielded substantial performance gains. Specifically, the adoption of the AdamW optimizer, known for its robust performance in deep learning, coupled with carefully designed learning rate schedules (such as warm-up and decay strategies), resulted in significant improvements—up to 7 percentage points in accuracy—over baseline models that employed simpler optimization approaches. This finding highlights that the 'how' of training, encompassing the strategic configuration of training parameters, is as crucial as the 'what' of the model architecture, providing valuable insights for future deep learning research in similar contexts.

Finally, a pivotal aspect of this project was its commitment to demonstrating a complete, end-to-end machine learning pipeline, extending beyond theoretical research to practical application. This commitment culminated in the successful development and deployment of a publicly accessible, interactive web application. This application serves as a tangible and demonstrable tool, effectively translating the sophisticated research model into a user-friendly and practical utility. Users can input Kannada news headlines, and the application instantly provides a classification, showcasing the real-world applicability and immediate utility of the developed system. This end-to-end demonstration not only validates the research findings but also provides a valuable resource for journalists, media organizations, and the general public, facilitating better organization and discoverability of Kannada news content. This successful deployment exemplifies the project's holistic approach, bridging the gap between academic research and practical, impactful solutions.

## B. Contributions and Achievements

This project makes several principal contributions to the field of Natural Language Processing, specifically addressing the gaps in the existing literature for Indic languages identified in the literature review.

The most significant and foundational contribution of this research is the creation and public release of the "Indic-Classify" dataset. This new, large-scale corpus comprises over 100,000 carefully curated and human-annotated Kannada news headlines. This resource directly addresses the primary bottleneck that has historically hindered substantial progress in the field of Indic language NLP—the pervasive lack of standardized, high-quality, and sufficiently large datasets.

Prior to "Indic-Classify," researchers working on Kannada NLP tasks often faced a dilemma: either construct small, ad-hoc datasets that lacked generalizability and comparability, or attempt to adapt pre-existing, often noisy, data sources that were not specifically designed for classification tasks. This fragmented data landscape made it exceedingly difficult to conduct fair and reproducible evaluations of different models, thereby slowing down the iterative development cycle.

"Indic-Classify" resolves this issue by providing a common, publicly available benchmark. Its large scale ensures statistical significance for model training and evaluation, while its careful annotation process guarantees high data quality and consistency. By offering a standardized benchmark, "Indic-Classify" facilitates transparent comparisons between different models and methodologies, encouraging healthy competition and fostering a more collaborative research environment within the Kannada NLP community. This contribution is a strategic intervention designed to catalyze the entire sub-field of Kannada NLP, providing the essential data infrastructure needed for robust model development and application. The dataset's design also considers a diverse range of news categories, ensuring its applicability to real-world scenarios and providing a rich testbed for nuanced language understanding.**A Comprehensive Empirical Benchmark of Modern NLP Architectures.**

Beyond the dataset, this work presents the first systematic study to benchmark a wide array of modern Transformer architectures alongside other deep learning and unsupervised models on a single, large-scale Kannada news classification task. The models chosen for this comprehensive evaluation include prominent Transformer-based models such as XLM-R (XLM-RoBERTa), mBERT (multilingual BERT), and DistilMBERT (Distilled Multilingual

BERT), which are known for their strong performance in various cross-lingual and multilingual tasks. Additionally, we evaluate a spectrum of other deep learning models, including recurrent neural networks (RNNs) like LSTMs and GRUs, and convolutional neural networks (CNNs), to provide a holistic view of performance across different architectural paradigms. We also include traditional machine learning baselines and unsupervised methods where appropriate, to contextualize the advancements brought by deep learning.

The detailed performance analysis conducted in this research provides invaluable insights into the relative strengths, weaknesses, and trade-offs of these diverse models when applied to a low-resource Indic language. For instance, the study rigorously examines how well multilingual pre-trained models, initially trained on a vast array of languages, transfer their knowledge to Kannada, an agglutinative and morphologically rich language. We analyze their ability to capture language-specific nuances, handle OOV (out-of-vocabulary) words, and generalize to unseen data. The empirical results shed light on factors such as model size, pre-training objectives, and fine-tuning strategies that significantly impact performance in this specific context.

This comprehensive benchmarking serves as an essential empirical guide for practitioners and researchers aiming to deploy NLP solutions for Kannada. It offers data-driven recommendations on selecting the most appropriate model architecture based on specific project requirements, available computational resources, and desired performance metrics (e.g., accuracy, precision, recall, F1-score). Furthermore, the analysis highlights areas where current models struggle, pointing towards promising avenues for future research in architectural improvements or more effective transfer learning techniques for Indic languages.**A Blueprint for Low-Resource NLP Development: An End-to-End Open-Source Pipeline**

A third crucial contribution of this project is the documentation of a complete, open-source, end-to-end NLP pipeline specifically designed for low-resource languages. This pipeline encompasses every stage of a typical NLP project, from the initial data acquisition and preparation to model fine-tuning, rigorous evaluation, and finally, deployment using a modern, scalable, and serverless architecture.

The challenges in low-resource NLP extend beyond just model development; they often involve the entire ecosystem around data management, experimental setup, and scalable deployment. By making our entire pipeline open-source, we provide a transparent and reproducible case study that can significantly lower the barrier to entry for future research and development in other low-resource languages. Researchers and developers in similar linguistic contexts can readily adapt and extend this blueprint, avoiding the need to re-engineer fundamental components from scratch.

Key aspects of this blueprint include:

- Data Scraping and Cleaning Methodologies: This crucial section details the systematic approaches for acquiring raw news headlines from diverse online sources, encompassing strategies for navigating various website structures and APIs. Beyond mere acquisition, it outlines robust techniques for data cleaning, which are paramount for transforming messy, unstructured data into a usable format. This includes sophisticated methods for de-duplication to eliminate redundant entries, noise reduction to remove irrelevant information or errors, and meticulous formatting to ensure consistency across the dataset. The emphasis on these methodologies is particularly significant in low-resource settings, such as those involving Indic languages, where the scarcity of clean, structured, and readily available data poses a substantial challenge to NLP research and development.
- Annotation Guidelines and Process: This component provides comprehensive documentation of the human annotation guidelines specifically developed and utilized for creating "Indic-Classify." It offers a transparent and replicable framework for similar data annotation efforts, ensuring consistency, quality, and scalability. The guidelines delineate precise instructions for annotators, including the definition of categories, criteria for classification, and conflict resolution protocols, thereby minimizing subjectivity and maximizing inter-annotator agreement. This detailed documentation is invaluable for researchers and practitioners aiming to build high-quality annotated datasets in under-resourced languages.
- Feature Engineering and Preprocessing: This section focuses on strategies specifically tailored for Kannada, a Dravidian language, to prepare text data for machine learning models. It covers essential preprocessing steps such as tokenization, which involves breaking down text into individual words or sub-word units, and

addressing language-specific characters and scripts. The blueprint also discusses the applicability of stemming, a process of reducing words to their root form, and the generation of embeddings. Embedding generation is particularly critical as it involves transforming textual data into dense vector representations that capture semantic relationships, which is vital for the performance of modern NLP models.

- Model Training and Fine-tuning: This part outlines best practices for fine-tuning pre-trained multilingual Transformer models, such as BERT, mBERT, or XLM-R, on specific downstream tasks relevant to the "Indic-Classify" project (e.g., news headline classification). It delves into hyperparameter optimization techniques, which are essential for maximizing model performance by finding the optimal configuration of learning rates, batch sizes, and other parameters. Furthermore, it addresses strategies for effectively handling class imbalance, a common challenge in real-world datasets where certain categories are significantly more represented than others. These techniques ensure the models are robust and generalize well across all classes.
- Robust Evaluation Framework: This blueprint proposes a standardized and comprehensive approach to model evaluation. It advocates for employing a diverse set of metrics beyond simple accuracy, such as precision, recall, F1-score, and ROC AUC, to gain a nuanced understanding of model performance, especially in imbalanced datasets. Furthermore, it emphasizes the use of statistical tests to assess the significance and reliability of results, ensuring that observed improvements are not merely due to chance. This rigorous evaluation framework contributes to the trustworthiness and generalizability of the research findings.
- Modern Serverless Deployment: This highly practical guide details the transition of NLP research prototypes into scalable, cost-effective, and production-ready applications through the adoption of serverless computing paradigms. It provides step-by-step instructions and considerations for deploying NLP models as microservices using popular cloud platforms like AWS Lambda and Google Cloud Functions. This section addresses an often-overlooked aspect in academic research—the operationalization of models. By demonstrating how to move beyond theoretical models to deployable solutions, the blueprint bridges the gap between research and real-world application, making NLP technologies accessible and impactful in various domains.

This comprehensive blueprint not only showcases a successful application of state-of-the-art NLP techniques to a low-resource language like Kannada but also serves as an invaluable practical guide for the broader NLP community. It demystifies the complex process of building, training, and deploying robust NLP systems, particularly in environments where computational and data resources are limited. By providing clear, actionable steps and insights, it fosters greater innovation, encourages the development of NLP solutions for a wider range of languages, and promotes the practical application of these cutting-edge technologies globally, thereby contributing significantly to the accessibility and impact of NLP.

## C. Future Directions and Potential Improvements

While the Indic-Classify project achieved its core objectives of designing, developing, and deploying a high-performance deep learning system for Kannada news headline classification, its acknowledged limitations provide clear signposts for promising avenues of future research. These limitations are critical for understanding the scope of the current work and for guiding subsequent efforts to enhance the system's robustness, adaptability, and applicability.

**1.Addressing Contextual Constraints:**

A primary limitation of the current system is its reliance on headlines alone for classification. News headlines are inherently concise and often designed to be attention-grabbing, which can lead to ambiguities or a lack of sufficient contextual information for precise categorization. For instance, a headline like "Tech Giant Acquires Startup" could pertain to various news categories (e.g., Business, Technology, Mergers & Acquisitions) without further details.

Future work should explore training models on the full text of news articles to provide richer context and resolve the ambiguities present in short headlines. Access to the entire article body would allow the model to leverage a much broader vocabulary and deeper semantic relationships, leading to more accurate and nuanced classifications. This approach aligns with the general trend in natural language processing (NLP) towards models that can process longer sequences and capture more comprehensive contextual information.

Furthermore, multi-modal approaches that incorporate images or other media associated with the article could provide additional signals for more accurate classification. News articles often include relevant images, videos, or infographics that convey significant information. Integrating these visual and other media cues with textual analysis could create a more holistic understanding of the news content, especially for categories where visual elements are particularly informative (e.g., sports, entertainment, current events). This would require advancements in multi-modal deep learning techniques capable of effectively combining diverse data types.

**2. Addressing the Static Taxonomy:**

The current system is limited to a fixed set of predefined categories. This static taxonomy presents a significant challenge in the dynamic landscape of news, where new topics and trends emerge constantly. For example, the sudden onset of a global pandemic or a new technological breakthrough would require the current system to be completely retrained to accommodate these new categories, which is computationally expensive and time-consuming.

A critical area for future research is the development of systems that can adapt to new and emerging news topics without requiring complete retraining. This could involve exploring few-shot, zero-shot, or open-set classification techniques.

- **Few-shot learning** aims to enable models to learn new categories from a very small number of examples. This would be highly beneficial for quickly incorporating new news topics with minimal data annotation effort.
- **Zero-shot learning** takes this a step further, allowing the model to classify instances of categories it has never seen before during training, relying on semantic descriptions of these categories. This is particularly relevant for highly novel news events.
- **Open-set classification** addresses the challenge of recognizing and categorizing instances that do not belong to any of the predefined categories, preventing the model from misclassifying truly novel topics into existing, inappropriate categories.

These advanced techniques would significantly enhance the system's flexibility and longevity, making it more robust against the ever-changing nature of news content.

**3. Improving Model Architecture and Fine-Tuning:**

The field of NLP is rapidly evolving, with new and more powerful pre-trained models being released regularly. The current project leveraged state-of-the-art models available at the time of development. However, continuous research and development are essential to maintain performance and efficiency.

Future work should investigate the performance of even more recent and powerful pre-trained models. This includes exploring models specifically designed for Indic languages (e.g., IndicBERT, MuRIL) as they become more mature and widely available. These models are typically trained on vast corpora of Indic languages, allowing them to capture the unique linguistic nuances and complexities that general-purpose multilingual models might miss. Their integration could lead to significant performance gains for Kannada news classification.

Additionally, exploring more advanced and efficient fine-tuning techniques is crucial. Traditional fine-tuning often involves updating all parameters of a large pre-trained model, which is computationally intensive and requires substantial memory. Parameter-Efficient Fine-Tuning (PEFT) methods, such as LoRA (Low-Rank Adaptation of Large Language Models) mentioned in the literature review, offer a promising alternative. PEFT methods achieve

comparable or even better performance by updating only a small subset of the model's parameters or introducing a small number of new parameters during fine-tuning. This significantly lowers the computational cost and memory footprint, making it more feasible to fine-tune even larger, state-of-the-art models and democratizing access to cutting-edge NLP technology. Further research into other PEFT techniques like Prefix-tuning or Prompt-tuning could also yield valuable insights.Broader Impact and Conclusion.

The Indic-Classify project successfully designed, developed, and deployed a high-performance deep learning system for Kannada news headline classification. The technical achievements—including the creation of a new benchmark dataset for Kannada news, a rigorous comparative analysis of state-of-the-art models tailored for the task, and the demonstration of an end-to-end MLOps (Machine Learning Operations) pipeline for seamless model deployment and management—represent a substantial contribution to the nascent but rapidly growing field of Indic NLP. These contributions provide a solid foundation for future research and development in language technologies for Kannada.

However, the broader impact of this work extends significantly beyond the technical domain. In an increasingly digital world, where information dissemination and access are predominantly mediated through online platforms, the ability of a linguistic community to participate fully and equitably is contingent on the availability of effective language technologies. Many languages, particularly those with fewer digital resources compared to English, face the risk of marginalization in the digital sphere. By advancing the state-of-the-art in Kannada NLP, this project directly contributes to a more inclusive and equitable digital sphere. It empowers Kannada speakers to access and process information more efficiently, thereby reducing the digital divide.

The open-source dataset and models produced by this research are pivotal to this broader impact. By making these resources freely available, the project empowers other developers, researchers, media organizations, and even individual enthusiasts to build more sophisticated, personalized, and culturally relevant digital experiences for millions of Kannada speakers. This fosters a collaborative environment where innovation can flourish, leading to a wider array of applications such as improved news aggregation, content recommendation systems, sentiment analysis for social media monitoring, and better search functionalities, all tailored for the Kannada language.

Ultimately, projects like Indic-Classify are essential steps in ensuring that the immense benefits of artificial intelligence are accessible to all, regardless of the language they speak. By developing robust and effective language technologies for diverse linguistic communities, we actively contribute to the preservation and promotion of linguistic diversity in the age of AI. This not only enriches the global digital landscape but also ensures that the unique cultural heritage and linguistic identities of various communities are sustained and thrive in an increasingly interconnected world. The Indic-Classify project stands as a testament to the potential of AI to serve as a tool for linguistic empowerment and inclusion.

# VI. References


[1] S. Aggarwal, S. Kumar, and R. Mamidi, “Efficient Multilingual Text Classification for Indian Languages,” in Proc. Int. Conf. Recent Advances in Natural Language Processing (RANLP), 2021, pp. 19–25. doi: 10.26615/978-954-452-072-4_003.

[2] Z. Chase, N. Genain, and O. Karniol-Tambour, “Learning Multi-Label Topic Classification of News Articles,” Dept. Comput. Sci., Stanford Univ., Stanford, CA, USA, Rep. CS229, 2013.

[3] M. Kowsher et al., "Bangla-BERT: Transformer-Based Efficient Model for Transfer Learning and Language Understanding," IEEE Access, vol. 10, pp. 91855-91870, 2022, doi: 10.1109/ACCESS.2022.3197662.

[4] R. K. Shah, S. Kumar, and Shashank, "Multilabel News Category Classification using Machine Learning," in Proc. 8th Int. Conf. Communication and Electronics Systems (ICCES), Coimbatore, India, 2023, pp. 1245-1250, doi: 10.1109/ICCES57224.2023.10192826.

[5] H. Manai, “Machine Learning: Text Classification of News Articles,” Medium,Oct. 2022. [Online]. Available: https://medium.com/@hichemmanai/machine-learning-text-classification-of-news-articles-3ab310323ffc

[6] A. Pandey, E. Sharma, R. Tiwari, A. Bisht, and B. P. Prathaban, “Precision Driven Sentiment and Topic Classification of News Articles using IndicBert,” in Proc. Int. Conf. Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI), 2025, pp. 1–6, doi: 10.1109/RAEEUCCI63961.2025.11048217.

[7] A. Velankar et al., “L3Cube-MahaHate: A Tweet-Based Marathi Hate Speech Detection Dataset and BERT Models,” arXiv:2203.13778, 2022.

[8] Dhanusha et al., "Automated Categorization and Analysis of Kannada News Content Using Transformers," in Proc. Int. Conf. Next Generation Communication & Information Processing (INCIP), Bangalore, India, 2025, pp. 719-722, doi: 10.1109/INCIP64058.2025.11019026.

[9] J. Chen et al., “An Analysis Method for Interpretability of CNN Text Classification Model,” Future Internet, vol. 12, no. 12, p. 228, Dec. 2020. doi: 10.3390/fi12120228.

[10] “FedP²EFT: Federated Learning to Personalize Parameter Efficient Fine-Tuning for Multilingual LLMs,” arXiv:2502.04387, 2025.

[11] A. Kulkarni et al., “Mono vs Multilingual BERT for Hate Speech Detection and Text Classification: A Case Study in Marathi,” arXiv:2204.08669, 2022.

[12] Kaggle, “Text Augmentation.” [Online]. Available: https://www.kaggle.com/code/aoprisan/text-augmentation

[13] X. Lin et al., “Multilingual Text Classification for Dravidian Languages,” arXiv:2112.01705, 2021.

[14] P. Deshmukh, N. Kulkarni, S. Kulkarni, K. Manghani, and R. Joshi, "Leveraging Parameter Efficient Training Methods for Low Resource Text Classification: A case study in Marathi," in Proc. 9th IEEE Int. Conf. Convergence in Technology (I2CT), Pune, India, 2024, pp. 1-6, doi: 10.1109/I2CT61223.2024.10543946.

[15] K. Saunack, K. Saurav, and P. Bhattacharyya, “Analysing cross-lingual transfer in lemmatisation for Indian languages,” in Proc. 28th Int. Conf. Computational Linguistics (COLING), 2020, pp. 6070–6076.

[16] AI4Bharat, “indicnlp_catalog.” [Online]. Available: https://github.com/AI4Bharat/indicnlp_catalog

[17] Proc. First Workshop on Natural Language Processing for Indo-Aryan and Dravidian Languages, Stroudsburg, PA, USA: Association for Computational Linguistics, 2020.

[18] Cohere, “Classification Metrics Explained: Accuracy, Precision, Recall & F1 Score,” 2022. [Online]. Available: https://cohere.com/blog/classification-metrics

[19] Analytics Vidhya, “Text Classification of News Articles,” 2024. [Online]. Available: https://www.analyticsvidhya.com/blog/2021/07/text-classification-of-news-articles/

[20] D. Aggarwal et al., “MAPLE: Multilingual Evaluation of Parameter Efficient Finetuning of Large Language Models,” arXiv:2401.07598, 2024.

[21] C. S. K. Selvi, N. Induja, S. L. Lekshmi, and S. Nagammai, "Topic Categorization of Tamil News Articles," in Proc. Int. Conf. Computer Communication and Informatics (ICCCI), Coimbatore, India, 2022, pp. 1-6, doi: 10.1109/ICCCI54379.2022.9740925.

[22] G. Chetan, K. Lavanya, M. G. Kumar, and B. Naseeba, "Telugu News Classification Using N-gram Model," in Proc. 3rd Edition of IEEE Delhi Section Flagship Conf. (DELCON), New Delhi, India, 2024, doi: 10.1109/DELCON60431.2024.10536768.

[23] U. Fatima, “Augmentation Techniques for Imbalanced Text Classification,” Walmart Global Tech Blog, 2024. [Online]. Available: https://medium.com/walmartglobaltech/augmentation-techniques-for-imbalanced-text-classification-a-comparative-study-38965725e17e

[24] H. Tang, S. Kamei, and Y. Morimoto, “Data Augmentation Methods for Enhancing Robustness in Text Classification Tasks,” Algorithms, vol. 16, no. 1, p. 59, 2023. doi: 10.3390/a16010059.

[25] A. Kunchukuttan et al., “AI4Bharat-IndicNLP Corpus: Monolingual Corpora and Word Embeddings for Indic Languages,” arXiv:2005.00085, 2020.

[26] Hugging Face, “PEFT: Parameter-Efficient Fine-Tuning Methods for LLMs.” [Online]. Available: https://huggingface.co/docs/peft/index

[27] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685, 2021.

[28] A. Kulkarni et al., “An Attention Ensemble Approach for Efficient Text Classification of Indian Languages,” arXiv:2102.10275, 2021.

[29] V. Gampala, J. Vallapuneni, P. K. Ande, R. K. Indurthi, and N. Rajesh, "Comparative Study on Telugu text Classification using Machine Learning and Deep Learning models," in Proc. 5th Int. Conf. Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 2021, pp. 1393-1398, doi: 10.1109/ICOEI51242.2021.9453009.

[30] Microsoft, “Custom text classification evaluation metrics,” Azure AI services. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/language-service/custom-text-classification/concepts/evaluation-metrics

[31] R. Joshi, “L3Cube-MahaCorpus and MahaBERT: Marathi Monolingual Corpus, Marathi BERT Language Models, and Resources,” in Proc. WILDRE-6 Workshop within the 13th Language Resources and Evaluation Conf., Marseille, France, Jun. 2022, pp. 49–54.

[32] X. Zhou et al., “An Empirical Study on Parameter-Efficient Fine-Tuning for MultiModal Large Language Models,” in Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 2024.

[33] N. E., “Advancing Dravidian Language Processing Beyond the Sentence Level,” M.S. thesis, IIIT Hyderabad, Hyderabad, India, 2024.

[34] GeeksforGeeks, “Model Interpretability in Deep Learning: A Comprehensive Overview.” [Online]. Available: https://www.geeksforgeeks.org/model-interpretability-in-deep-learning-a-comprehensive-overview/

[35] AI4Bharat, “indicnlp_corpus.” [Online]. Available: https://github.com/AI4Bharat/indicnlp_corpus

[36] D. Kakwani et al., “IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages,” arXiv:2005.00085, 2020.

[37] V. S. Sravya, S. K. S, and K. P. Soman, "Text Categorization of Telugu News Headlines," in Proc. 2nd Int. Conf. Intelligent Technologies (CONIT), Hubli, India, 2022, pp. 1-6, doi: 10.1109/CONIT55075.2022.9848243.

[38] A. Deroy and S. Maity, “Prompt Engineering Using GPT for Word-Level Code-Mixed Language Identification in Low-Resource Dravidian Languages,” arXiv:2411.04025, 2024.

[39] D. Gupta, K. Thejaa, A. R. Nair, and P. Katti, "Automated Regional Language News Article Classification System for Kannada and Telugu," in Proc. 5th Int. Conf. Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2025.

[40] G. Leonard, F. Sisnadi, N. V. Wardhana, M. A. A. Al-Ghofari, and A. S. Girsang, "News Classification Based On News Headline Using SVC Classifier," in Proc. 16th Int. Conf. Telecommunication Systems, Services, and Applications (TSSA), Lombok, Indonesia, 2022, pp. 1-5, doi: 10.1109/TSSA56816.2022.9978393.

[41] M. S. N. Reddy, S. Aravind, and H. R. Bojjam, "Telugu News Article Classification Using Deep Learning Models," in Proc. Int. Conf. Data Science and Network Security (ICDSNS), Tiptur, India, 2023, doi: 10.1109/ICDSNS58790.2023.10143118.

[42] A. Kandarkar, “Multilabel Classification for Categorization of News Articles,” AlgoAnalytics, 2023. [Online]. Available: https://www.algoanalytics.com/blog/multilabel-classification-for-categorization-of-news-articles/

[43] Neptune.ai, “Data Augmentation in NLP: Best Practices From a Kaggle Master,” 2023. [Online]. Available: https://neptune.ai/blog/data-augmentation-in-nlp

[44] P. Kashyap, “Understanding Precision, Recall, and F1 Score Metrics,” Medium, 2024. [Online]. Available: https://medium.com/@pranav.kashyap/understanding-precision-recall-and-f1-score-metrics-4809f6f6001a

[45] H. Hanumanthappa and M. N. Swamy, “Indian Language Text Documents Categorization and Keyword Extraction,” Int. J. Comput. Trends and Technol. (IJCTA), vol. 9, no. 3, pp. 1473–1481, 2016.

[46] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," arXiv:1911.02116, 2019.

[47] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[48] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. Conf. North American Chapter Assoc. Comput. Linguistics, Minneapolis, MN, USA, 2019, pp. 4171–4186.

# VII. Appendices

## A. Code Snippets

**Phase 1:**

Sample headline from each category

| Category | Headline |
|---|---|
| ರಾಜಕೀಯ | ಆಂತರಿಕ ಕಚ್ಚಾಟ ಮರೆಮಾಚಲು ಕಾಂಗ್ರೆಸ್ ಸುಳ್ಳು ಭರವಸೆ ನೀಡುತ್ತಿದೆ: ಅರುಣ್ ಸಿಂಗ್ |
| ವಿದೇಶ | ಭಾರತ - ಪಾಕಿಸ್ತಾನ ನಡುವೆ ಪರಮಾಣು ಯುದ್ಧ ನಡೆದರೆ 125 ಬಿಲಿಯನ್ ಮಂದಿ ಸಾವು: ಅಧ್ಯಯನ |
| ಕ್ರೀಡೆ | MS Dhoni's ₹100-crore defamation case: 10 ವರ್ಷಗಳ ನಂತರ ಕೊನೆಗೂ ವಿಚಾರಣೆ ಕೈಗೆತ್ತಿಕೊಂಡ ಮದ್ರಾಸ್ ಹೈಕೋರ್ಟ್ |
| ವಾಣಿಜ್ಯ | ಕಳಪೆ ನಗರೀಕರಣದಿಂದ 2050ಕ್ಕೆ 120 ಲಕ್ಷ ಕೋಟಿ ರೂ. ಹೊರೆ |
| ಮನರಂಜನೆ | 'ಸೈ ರಾ' ಕನ್ನಡ ಟ್ರೇಲರ್ ರಿಲೀಸ್: ಸುದೀಪ್ ಧ್ವನಿ ಕೇಳಿ ಥ್ರಿಲ್ ಆದ ಅಭಿಮಾನಿಗಳು |
| ಅಪರಾಧ ಸುದ್ದಿ | ಎಸ್ಎಸ್ಎಲ್ಸಿಯಲ್ಲಿ ಮೂರು ಬಾರಿ ಫೇಲ್; ಮನನೊಂದ ಬಾಲಕನಿಂದ ದೇವರ ವಿಗ್ರಹ ವಿರೂಪ! |
| ಆರೋಗ್ಯ | ಈ 7 ಬಗೆಯ ಸೈಲೆಂಟ್ ಕಿಲ್ಲರ್ ಕಾಯಿಲೆಗಳ ಲಕ್ಷಣಗಳನ್ನು ಕಣ್ಣುಗಳಲ್ಲಿ ನೋಡಿಯೇ ಪತ್ತೆ ಹಚ್ಚಬಹುದಂತೆ! |
| ಕೃಷಿ | ಭತ್ತಕೃಷಿ: ಪರಿಸರ ಸ್ನೇಹಿ ವಿಧಾನಕ್ಕೆ ಮೊರೆ |
| ತಂತ್ರಜ್ಞಾನ | TikTok \| ಟಿಕ್‌ಟಾಕ್ ಮೇಲಿನ ನಿಷೇಧ ತೆರವುಗೊಳಿಸಿಲ್ಲ: ಕೇಂದ್ರ ಸರ್ಕಾರ |
| ಸಾಹಿತ್ಯ | 'ಮೂಡಲಪಾಯ ಯಕ್ಷಗಾನ' ಕಲೆಗೆ ಜೀವ ತುಂಬುವ ಚಿಣ್ಣರು! |

Fig7.1.1: Sample rows of the data

```
1 data['category'].nunique()
```

```
62
```

Fig7.1.2: Initial number of categories

```
#Grouping different categories

data.loc[data['category']=='ಕ್ರಿಕೆಟ್ ಸುದ್ದಿ','category']='ಕ್ರೀಡೆ'
data.loc[data['category']=='ಕ್ರೀಡಾ ಲೇಖನ','category']='ಕ್ರೀಡೆ'
data.loc[data['category']=='ಇತರ ಕ್ರೀಡೆ','category']='ಕ್ರೀಡೆ'
data.loc[data['category']=='ವಾಣಿಜ್ಯ ಲೇಖನ','category']='ವಾಣಿಜ್ಯ'
data.loc[data['category']=='ವಾಣಿಜ್ಯ ಸುದ್ದಿ','category']='ವಾಣಿಜ್ಯ'
data.loc[data['category']=='ಸಿನಿಮಾ','category']='ಮನರಂಜನೆ'
data.loc[data['category']=='ಸಿನಿಮಾ ವಿಮರ್ಶೆ','category']='ಮನರಂಜನೆ'
data.loc[data['category']=='ಗಾಸಿಪ್','category']='ಮನರಂಜನೆ'
data.loc[data['category']=='ಬಾಲಿವುಡ್','category']='ಮನರಂಜನೆ'
data.loc[data['category']=='ಕಿರುತೆರೆ','category']='ಮನರಂಜನೆ'
data.loc[data['category']=='ಯೋಗ','category']='ಆರೋಗ್ಯ'
data.loc[data['category']=='ದಿನ ಭವಿಷ್ಯ','category']='ಭವಿಷ್ಯ'
data.loc[data['category']=='ವಾರ ಭವಿಷ್ಯ','category']='ಭವಿಷ್ಯ'
```

Fig7.1.3: Grouping similar categories

```
kp.loc[kp['category']=='health','category']='ಆರೋಗ್ಯ'
kp.loc[kp['category']=='fifa-world-cup','category']='ಕ್ರೀಡೆ'
kp.loc[kp['category']=='fifa-world-cup-2018','category']='ಕ್ರೀಡೆ'
kp.loc[kp['category']=='cricket','category']='ಕ್ರೀಡೆ'
kp.loc[kp['category']=='web-series','category']='ಮನರಂಜನೆ'
kp.loc[kp['category']=='science-technology','category']='ತಂತ್ರಜ್ಞಾನ'
kp.loc[kp['category']=='sahitya-sammelana','category']='ಸಾಹಿತ್ಯ'
kp.loc[kp['category'].str.contains('budget'),'category']='ವಾಣಿಜ್ಯ'
kp.loc[kp['category']=='gadgets','category']='ತಂತ್ರಜ್ಞಾನ'
kp.loc[kp['category']=='world','category']='ವಿದೇಶ'
kp.loc[kp['category']=='agriculture-environment','category']='ಕೃಷಿ'
kp.loc[kp['category']=='politics','category']='ರಾಜಕೀಯ'
kp.loc[kp['category']=='elections','category']='ರಾಜಕೀಯ'
kp.loc[kp['category']=='elections-2019','category']='ರಾಜಕೀಯ'
kp.loc[kp['category']=='karnataka-election','category']='ರಾಜಕೀಯ'
kp.loc[kp['category']=='business','category']='ವಾಣಿಜ್ಯ'
```

Fig7.1.4: Renaming the categories to kannada

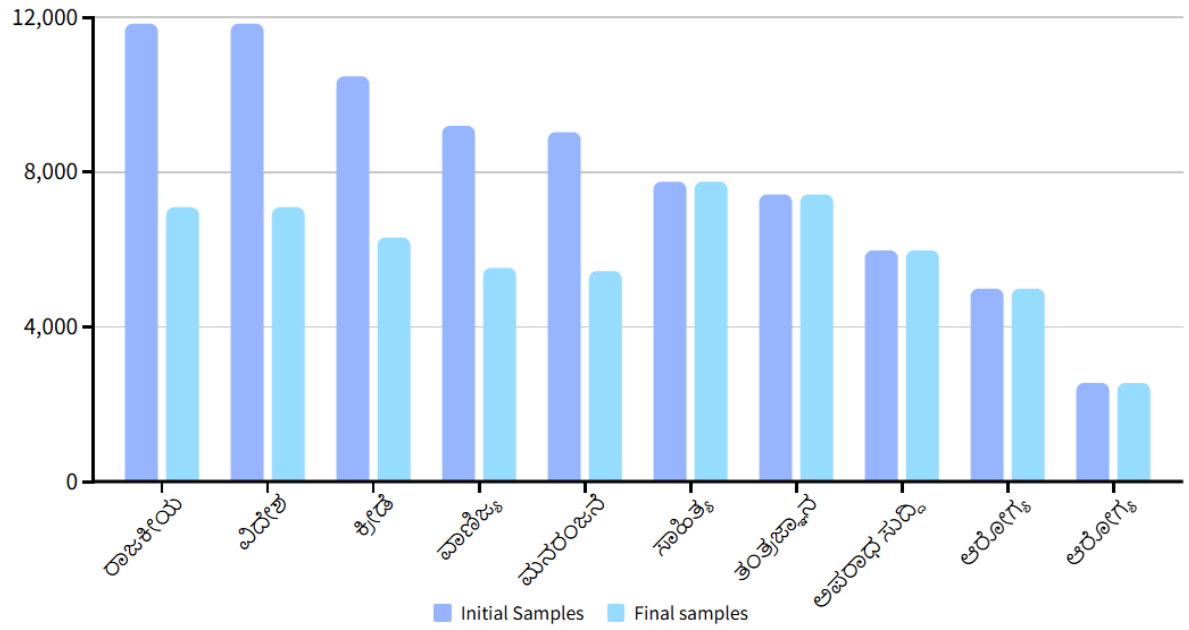


Fig7.1.5: Category-wise distribution before and after preprocessing

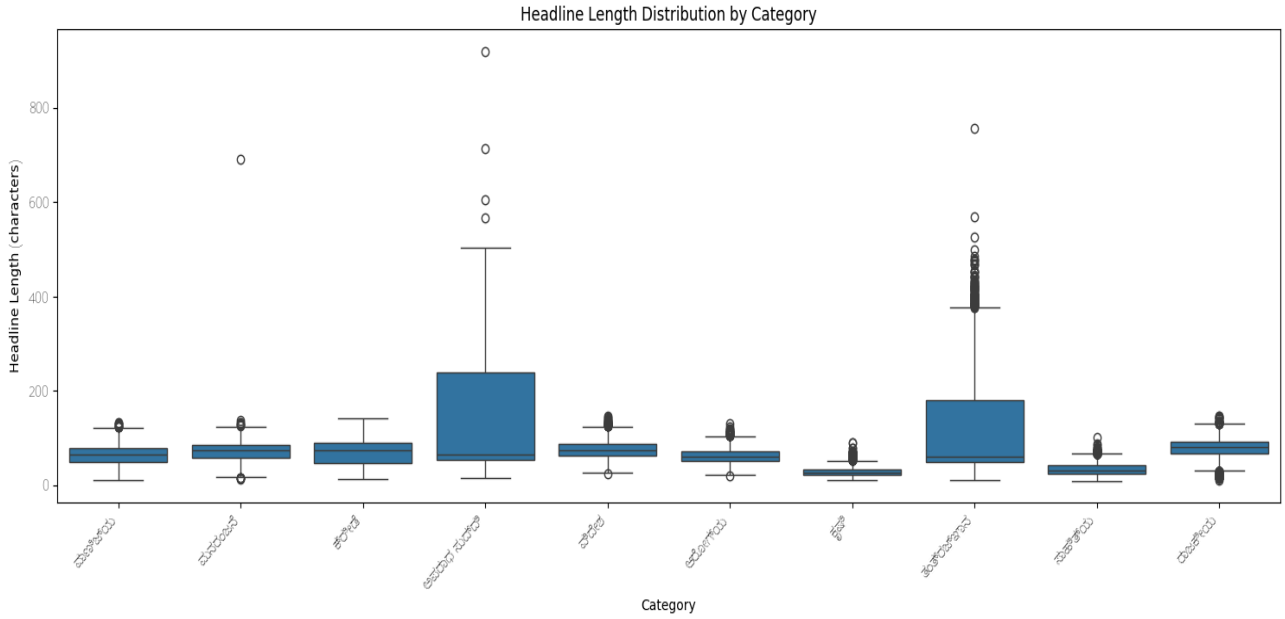


Fig7.1.6: Category-wise headline length distribution

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Create TF-IDF features
tfidf = TfidfVectorizer(max_features=1000, stop_words=None)  # Stop words removal
tfidf_features = tfidf.fit_transform(data_with_districts_removed['headline'])
```

Fig7.1.7: TF-IDF vectorisation

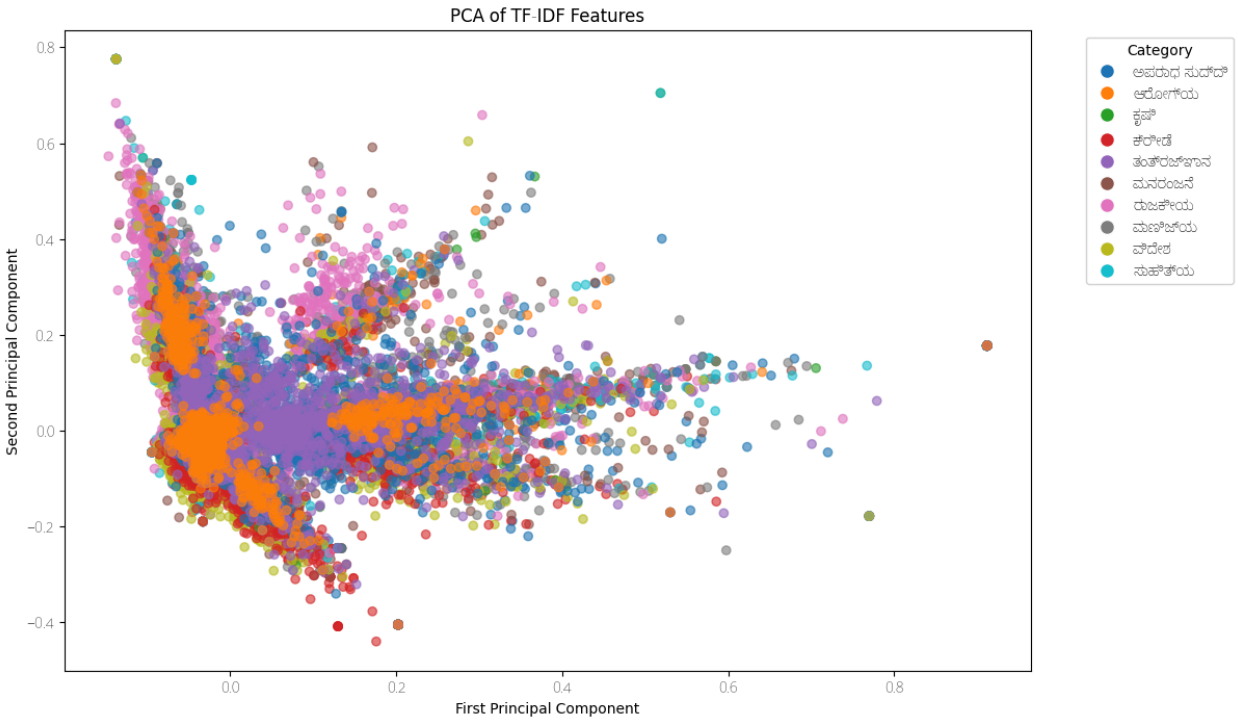


Fig7.1.8: PCA of TF-IDF features

```
tsne = TSNE(n_components=2, random_state=42, perplexity=30, n_iter=300)
tsne_embeddings = tsne.fit_transform(pca_embeddings)
```

Fig7.1.9: T-SNE

**Phase 2.1: Unsupervised**

```
def simple_preprocess_kannada(text):
    if not isinstance(text, str):
        return ""

    text = text.lower()

    tokens = re.findall(r'\b\w+\b', text)
    filtered_tokens = [token for token in tokens if token not in kannada_stopwords]
    preprocessed_text = " ".join(filtered_tokens)

    return preprocessed_text
```

Fig7.2.1.1: Tokenisation

| | category | headline | preprocessed_headline |
|---|---|---|---|
| 0 | ರಾಜಕೀಯ | ಆಂತರಿಕ ಕಚ್ಚಾಟ ಮರೆಮಾಚಲು ಕಾಂಗ್ರೆಸ್ ಸುಳ್ಳು ಭರವಸೆ ... | ಆ ತ ರ ಕ ಕ ಚ ಚ ಟ ಮ ರ ಮ ಚ ಲ ಕ ಗ ರ ಸ ಸ ಳ ಳ ಭ ರ ವ ಸ ನ ಡ ತ... |
| 1 | ರಾಜಕೀಯ | ಒಳ್ಳೆಯ ಸೋದರ ಎಂದರೆ ಅರ್ಥವೇನೆಂದು ರಾಹುಲ್ ಗಾಂಧಿ ವಿವ... | ಒ ಳ ಳ ಯ ಸ ದ ರ ಎ ದ ರ ಅ ರ ಥ ವ ನ ದ ರ ಹ ಲ ಗ ಧ ವ ವ ರ ಸ ದ ... |
| 2 | ರಾಜಕೀಯ | ಕೋವಿಡ್ ಬಿಕ್ಕಟ್ಟು: ಪಾಕಿಸ್ತಾನವನ್ನು ಹಾಡಿ ಹೊಗಳಿದ ಧ... | ಕ ವ ಡ ಬ ಕ ಕ ಟ ಟ ಪ ಕ ಸ ತ ನ ವ ನ ನ ಹ ಡ ಹ ಗ ಳ ದ ಧ ರ ವ ಡ... |
| 3 | ರಾಜಕೀಯ | ಕರ್ನಾಟಕ ಚುನಾವಣೆ 2023: ಶೂನ್ಯ ಭ್ರಷ್ಟಾಚಾರ, ಉಚಿತ ವ... | ಕ ರ ನ ಟ ಕ ಚ ನ ವ ಣ 2023 ಶ ನ ಯ ಭ ರ ಷ ಟ ಚ ರ ಉ ಚ ತ ವ ದ ... |
| 4 | ರಾಜಕೀಯ | ಗ್ರಾಮ ಪಂಚಾಯಿತಿ ಚುನಾವಣೆ: ಮತದಾರರ ಸೆಳೆಯಲು ಹಣದ ಹೊಳ... | ಗ ರ ಮ ಪ ಚ ಯ ತ ಚ ನ ವ ಣ ಮ ತ ದ ರ ರ ಸ ಳ ಯ ಲ ಹ ಣ ದ ಹ ಳ ಆ ಸ ... |

Fig7.2.1.2: Tokenised headlines

```
class TVAE(keras.Model):
    def __init__(self, original_dim, intermediate_dim, latent_dim, n_clusters=10, **kwargs):
        super(TVAE, self).__init__(**kwargs)
        self.original_dim = original_dim
        self.intermediate_dim = intermediate_dim
        self.latent_dim = latent_dim
        self.n_clusters = n_clusters

        # Encoder
        self.encoder = keras.Sequential([
            layers.InputLayer(input_shape=(original_dim,)),
            layers.Dense(intermediate_dim, activation='relu'),
            layers.Dense(latent_dim + latent_dim), # For mean and log_var
        ])

        # Decoder
        self.decoder = keras.Sequential([
            layers.InputLayer(input_shape=(latent_dim,)),
            layers.Dense(intermediate_dim, activation='relu'),
            layers.Dense(original_dim, activation='sigmoid'),
        ])
```

Fig7.2.1.3: T-VAE(Temporal variational Autoencoder) model initiation

```python
def encode(self, x):
    mean_log_var = self.encoder(x)
    mean = mean_log_var[:, :self.latent_dim]
    log_var = mean_log_var[:, self.latent_dim:]
    return mean, log_var

def decode(self, z):
    return self.decoder(z)

def call(self, x):
    mean, log_var = self.encode(x)
    epsilon = tf.random.normal(shape=tf.shape(mean))
    z = mean + tf.exp(0.5 * log_var) * epsilon
    reconstruction = self.decode(z)
    return reconstruction, mean, log_var, z

def train_step(self, data):
    if isinstance(data, tuple):
        data = data[0]
    with tf.GradientTape() as tape:
        reconstruction, mean, log_var, z = self(data)
        reconstruction_loss = tf.reduce_mean(
            keras.losses.binary_crossentropy(data, reconstruction)
        )
        kl_loss = -0.5 * tf.reduce_mean(1 + log_var - tf.square(mean) - tf.exp(log_var))
        total_loss = reconstruction_loss + kl_loss
    grads = tape.gradient(total_loss, self.trainable_weights)
    self.optimizer.apply_gradients(zip(grads, self.trainable_weights))
    return {
        "loss": total_loss,
        "reconstruction_loss": reconstruction_loss,
        "kl_loss": kl_loss,
    }

def get_config(self):
    config = super(TVAE, self).get_config()
    config.update({
        "original_dim": self.original_dim,
        "intermediate_dim": self.intermediate_dim,
        "latent_dim": self.latent_dim,
        "n_clusters": self.n_clusters,
    })
    return config

@classmethod
def from_config(cls, config):
    return cls(**config)
```

Fig7.2.1.4: T-VAE methods(encode-decoder, training, configurations)

```
# Model parameters
original_dim = pca_embeddings.shape[1]  # Dimensionality of the PCA embeddings
intermediate_dim = 128
latent_dim = 50  # Latent dimension for the VAE

# Instantiate the T-VAE model
tvae_model = TVAE(original_dim, intermediate_dim, latent_dim)

# Compile the model
tvae_model.compile(optimizer=keras.optimizers.Adam())

# Train the model (using PCA embeddings)
history = tvae_model.fit(
    pca_embeddings,
    epochs=10,
    batch_size=32,
)
```

Fig7.2.1.5: T-VAE training

```
kmeans_tvae = KMeans(n_clusters=10, random_state=42, n_init=10)
tvae_labels = kmeans_tvae.fit_predict(tvae_latent_embeddings)

tsne_df = pd.DataFrame(data=tsne_embeddings, columns=['TSNE-1', 'TSNE-2'])
tsne_df['Cluster'] = tvae_labels

# Visualize the clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x="TSNE-1", y="TSNE-2",
    hue="Cluster",
    palette=sns.color_palette("hsv", 10),
    data=tsne_df,
    legend="full",
    alpha=0.6
)
plt.title('T-VAE Cluster Visualization (t-SNE on PCA Embeddings)')
plt.show()
```

Fig7.2.1.6: T-VAE cluster visualisation

```
class ATM(Model):
    def __init__(self, original_dim, n_clusters=10, **kwargs):
        super(ATM, self).__init__(**kwargs)
        self.n_clusters = n_clusters
        self.original_dim = original_dim

        # Topic layer
        self.topic_layer = layers.Dense(n_clusters, activation='softmax', name='topic_distribution')

        # Decoder (to reconstruct input based on topic distribution)
        self.decoder = layers.Dense(original_dim, activation='sigmoid')
```

Fig7.2.1.7: ATM(Topic Model with Attention) model initiation

```
    def call(self, inputs):
        topic_dist = self.topic_layer(inputs)
        reconstruction = self.decoder(topic_dist)

        return topic_dist, reconstruction

    @tf.function
    def train_step(self, data):
        x = data

        with tf.GradientTape() as tape:
            topic_dist, reconstruction = self(x, training=True) # Forward pass
            # Calculate reconstruction loss (e.g., Mean Squared Error)
            reconstruction_loss = tf.reduce_mean(tf.square(x - reconstruction))

            total_loss = reconstruction_loss

        grads = tape.gradient(total_loss, self.trainable_weights)
        self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

        return {"loss": total_loss, "reconstruction_loss": reconstruction_loss}

    def get_config(self):
        config = super(ATM, self).get_config()
        config.update({
            "original_dim": self.original_dim,
            "n_clusters": self.n_clusters,
        })
        return config

    @classmethod
    def from_config(cls, config):
        return cls(**config)
```

Fig7.2.1.8: ATM methods(training, configurations)

```
original_dim = pca_embeddings.shape[1]  # Dimensionality of the PCA embeddings

# Instantiate the ATM model
atm_model = ATM(original_dim, n_clusters=10)

atm_model.compile(optimizer='adam')

print("ATM model defined.")

# Train the model (using PCA embeddings)
print("Training ATM model...")
history = atm_model.fit(
    pca_embeddings,
    epochs=10,
    batch_size=32,
    verbose=1
)
```

Fig7.2.1.9: ATM training

```
tsne_df_atm = pd.DataFrame(data=tsne_embeddings, columns=['TSNE-1', 'TSNE-2'])
tsne_df_atm['Cluster'] = atm_labels

# Visualize the clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x="TSNE-1", y="TSNE-2",
    hue="Cluster",
    palette=sns.color_palette("hsv", 10),
    data=tsne_df_atm,
    legend="full",
    alpha=0.6
)
plt.title('ATM Cluster Visualization (t-SNE on PCA Embeddings)')
plt.show()
```

Fig7.2.1.10: ATM cluster visualisation

```
class TDM(Model):
    def __init__(self, original_dim, n_clusters=10, **kwargs):
        super(TDM, self).__init__(**kwargs)
        self.n_clusters = n_clusters
        self.original_dim = original_dim

        self.input_projection = layers.Dense(original_dim, activation='relu')

        self.topic_layer = layers.Dense(n_clusters, activation='softmax', name='topic_distribution')
```

Fig7.2.1.11: TDM(Dynamic Topic Model) initiation

```
def call(self, inputs):
    x = self.input_projection(inputs)

    topic_dist = self.topic_layer(x)

    return topic_dist

def get_config(self):
    config = super(TDM, self).get_config()
    config.update({
        "original_dim": self.original_dim,
        "n_clusters": self.n_clusters,
    })
    return config

@classmethod
def from_config(cls, config):
    return cls(**config)
```

Fig7.2.1.12: TDM methods(model call and configurations)

```
original_dim = pca_embeddings.shape[1]  # Dimensionality of the PCA embeddings

# Instantiate the TDM model
tdm_model = TDM(original_dim, n_clusters=10)

# Compile the model
tdm_model.compile(optimizer='adam', loss='categorical_crossentropy')

print("TDM model defined.")

# Train the model (using PCA embeddings)
print("Training TDM model...")


dummy_labels = tf.one_hot(tf.random.uniform(shape=(pca_embeddings.shape[0],), maxval=10, dtype=tf.int32), depth=10)


history = tdm_model.fit(
    pca_embeddings,
    dummy_labels,
    epochs=10,
    batch_size=32,
    verbose=1
)
```

Fig7.2.1.13: TDM training

```
tsne_df_tdm = pd.DataFrame(data=tsne_embeddings, columns=['TSNE-1', 'TSNE-2'])
tsne_df_tdm['Cluster'] = tdm_labels

# Visualize the clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x="TSNE-1", y="TSNE-2",
    hue="Cluster",
    palette=sns.color_palette("hsv", 10), # Adjust palette size based on number of clusters
    data=tsne_df_tdm,
    legend="full",
    alpha=0.6
)
plt.title('TDM Cluster Visualization (t-SNE on PCA Embeddings)')
plt.show()
```

Fig7.2.1.14: TDM cluster visualisation

```python
def __init__(self, original_dim, embedding_dim, n_clusters=10, **kwargs):
    super(DCTC, self).__init__(**kwargs)
    self.n_clusters = n_clusters
    self.embedding_dim = embedding_dim
    self.original_dim = original_dim

    # Deep Embedding Network
    self.embedding_net = keras.Sequential([
        layers.InputLayer(input_shape=(original_dim,)),
        layers.Dense(128, activation='relu'),
        layers.Dense(embedding_dim, activation='relu')
    ])

    # Clustering Layer
    self.clustering_layer = layers.Dense(n_clusters, activation='softmax', name='cluster_assignments')
```

Fig7.2.1.15: DCTC(Deep Clustering with Temporal Consistency) Initiation

```python
def call(self, inputs):
    # Get embeddings
    embeddings = self.embedding_net(inputs)
    # Get cluster assignments
    cluster_assignments = self.clustering_layer(embeddings)
    return embeddings, cluster_assignments

@tf.function
def train_step(self, data):
    x = data

    with tf.GradientTape() as tape:
        embeddings, cluster_assignments = self(x, training=True) # Forward pass
        # Calculate clustering loss
        dummy_labels = tf.one_hot(tf.random.uniform(shape=(tf.shape(x)[0],), maxval=self.n_clusters, dtype=tf.int32), depth=self.n_clusters)
        clustering_loss = keras.losses.categorical_crossentropy(dummy_labels, cluster_assignments)

        total_loss = tf.reduce_mean(clustering_loss)

    grads = tape.gradient(total_loss, self.trainable_weights)
    self.optimizer.apply_gradients(zip(grads, self.trainable_weights))

    return {"loss": total_loss, "clustering_loss": total_loss}

def get_config(self):
    config = super(DCTC, self).get_config()
    config.update({
        "original_dim": self.original_dim,
        "embedding_dim": self.embedding_dim,
        "n_clusters": self.n_clusters,
    })
    return config

@classmethod
def from_config(cls, config):
    return cls(**config)
```

Fig7.2.1.16: DCTC methods(call, training and configuration)

```
original_dim = pca_embeddings.shape[1] # Dimensionality of the PCA embeddings
embedding_dim = 64 # Dimensionality of the learned embeddings

# Instantiate the DCTC model
dctc_model = DCTC(original_dim, embedding_dim, n_clusters=10)

# Compile the model
dctc_model.compile(optimizer='adam')

print("DCTC model defined.")

# Train the model (using PCA embeddings)
print("Training DCTC model...")
history = dctc_model.fit(
    pca_embeddings,
    epochs=10,
    batch_size=32,
    verbose=1
)
```

Fig7.2.1.17: DCTC training

```
tsne_df_dctc = pd.DataFrame(data=tsne_embeddings, columns=['TSNE-1', 'TSNE-2'])
tsne_df_dctc['Cluster'] = dctc_labels

# Visualize the clusters
plt.figure(figsize=(10, 8))
sns.scatterplot(
    x="TSNE-1", y="TSNE-2",
    hue="Cluster",
    palette=sns.color_palette("hsv", 10),
    data=tsne_df_dctc,
    legend="full",
    alpha=0.6
)
plt.title('DCTC Cluster Visualization (t-SNE on PCA Embeddings)')
plt.show()
```

Fig7.2.1.18: DCTC cluster visualisation

```
evaluation_results = {}
labels = [('T-VAE', tvae_labels), ('ATM', atm_labels), ('TDM', tdm_labels), ('DCTC', dctc_labels)]

for model_name, model_labels in labels:
  try:
      silhouette = silhouette_score(pca_embeddings, model_labels)
      davies_bouldin = davies_bouldin_score(pca_embeddings, model_labels)
      calinski_harabasz = calinski_harabasz_score(pca_embeddings, model_labels)
      evaluation_results[f'{model_name}'] = {
          'Silhouette Score': silhouette,
          'Davies-Bouldin Index': davies_bouldin,
          'Calinski-Harabasz Index': calinski_harabasz
      }
      print("T-VAE Evaluation:")
      print(evaluation_results[f'{model_name}'])
  except Exception as e:
      print(f"Could not evaluate {model_name}: {e}")
```

Fig7.2.1.19: Model evaluation

**Phase 2.2: Supervised**

```
def clean_text(text: str) -> str:
    if not isinstance(text, str):
        return ''
    text = re.sub(r'http\S+|www\S+', ' ', text)
    text = re.sub(r'[\d]', ' ', text)
    text = re.sub(r'[\u0021-\u0040\u005B-\u0060\u007B-\u007E]', ' ', text)
    text = re.sub(r'\s+', ' ', text).strip()
    return text
```

Fig7.2.2.1: Cleaning(Removal of punctuations, html tags and numbers)

```
def preprocess_kn(text: str) -> str:
    text = clean_text(text)
    text = normalizer.normalize(text)
    # Tokenize using Indic tokenizer
    tokens = [t for t in indic_tokenize.trivial_tokenize(text, lang='kn')]
    tokens = [t for t in tokens if t not in kannada_stopwords and len(t) > 1]
    return ' '.join(tokens)
```

Fig7.2.2.2: Text normalisation and tokenization

```
texts_supervised = df['headline'].astype(str)
labels_supervised = df['category'].astype(str)

X_tr, X_te, y_tr, y_te = train_test_split(texts_supervised, labels_supervised, test_size=0.2, random_state=42, stratify=labels_supervised)
X_tr, X_va, y_tr, y_va = train_test_split(X_tr, y_tr, test_size=0.1, random_state=42, stratify=y_tr)
```

Fig7.2.2.3: Train-test split(80:20)

```
le = LabelEncoder()
y_tr_enc = le.fit_transform(y_tr)
y_va_enc = le.transform(y_va)
y_te_enc = le.transform(y_te)
num_labels = len(le.classes_)
```

Fig7.2.2.4: Label encoding

```
def train_kn_classifier(model_ckpt: str, out_dir: str, epochs: int = 1, lr: float = 2e-5,
                        train_bs: int = 32, eval_bs: int = 64, max_len: int = 128,
                        log_steps: int = 100):
    tokenizer = AutoTokenizer.from_pretrained(model_ckpt)

    def tok_fn(batch):
        return tokenizer(batch['text'], truncation=True, padding='max_length', max_length=max_len)

    # Use the encoded labels defined outside the function
    train_ds = Dataset.from_dict({'text': X_tr.tolist(), 'label': y_tr_enc})
    val_ds = Dataset.from_dict({'text': X_va.tolist(), 'label': y_va_enc})
    test_ds = Dataset.from_dict({'text': X_te.tolist(), 'label': y_te_enc})

    train_ds = train_ds.map(tok_fn, batched=True)
    val_ds = val_ds.map(tok_fn, batched=True)
    test_ds = test_ds.map(tok_fn, batched=True)

    cols = ['input_ids', 'attention_mask', 'label']
    train_ds.set_format(type='torch', columns=cols)
    val_ds.set_format(type='torch', columns=cols)
    test_ds.set_format(type='torch', columns=cols)

    # Model
    model = AutoModelForSequenceClassification.from_pretrained(model_ckpt, num_labels=num_labels)
```

Fig7.2.2.5: Model definition

```
args = TrainingArguments(
    output_dir=out_dir,
    per_device_train_batch_size=train_bs,
    per_device_eval_batch_size=eval_bs,
    learning_rate=lr,
    weight_decay=0.01,
    num_train_epochs=epochs,
    eval_steps=500,
    save_steps=1000,
    logging_steps=log_steps,
    fp16=torch.cuda.is_available()
)
```

Fig7.2.2.6: Trainer arguments

```
    trainer = Trainer(
        model=model,
        args=args,
        train_dataset=train_ds,
        eval_dataset=val_ds,
        tokenizer=tokenizer,
        compute_metrics=compute_metrics
    )
```

Fig7.2.2.7: Trainer definition

```python
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    acc = accuracy_score(labels, preds)
    p, r, f1, _ = precision_recall_fscore_support(labels, preds, average='macro', zero_division=0)
    return {'accuracy': acc, 'macro_f1': f1, 'macro_precision': p, 'macro_recall': r}
```

Fig7.2.2.8: Model metrics calculation

```python
models = [
    ('xlm-roberta-base', 'cls_kn_xlmr_v2'),
    ('bert-base-multilingual-cased', 'cls_kn_mbert'),
    ('distilbert-base-multilingual-cased', 'cls_kn_distilmbert'),
]

results = {}
for ckpt, out in models:
    print(f"\n=== Training {ckpt} ===")
    metrics = train_kn_classifier(ckpt, out_dir=out, epochs=1)
    results[ckpt] = metrics
    print(ckpt, metrics)

print("\nSummary (macro_f1):")
for ckpt, m in results.items():
    print(ckpt, '->', round(m.get('eval_macro_f1', float('nan')), 4))
```

Fig7.2.2.9: Model training

```
Summary (macro_f1 where available):
xlm-roberta-base: 0.827751960495128
bert-base-multilingual-cased: 0.7948568889202062
distilbert-base-multilingual-cased: 0.7458357741946047
```

Fig7.2.2.10: Training results

**Phase 3: Finetuning**

```
finetune_results = {}
models = [
    ('xlm-roberta-base', 'cls_kn_xlmr'),
    ('bert-base-multilingual-cased', 'cls_kn_mbert'),
    ('distilbert-base-multilingual-cased', 'cls_kn_distilmbert'),
]

for ckpt, out in models:

    print(f"\n=== Training / Evaluating {ckpt} -> {out} ===")
    best_model_cfg = os.path.join(out, 'best_model', 'config.json')
    try:
        for epoch in range(1,6):
            print(f"--- Run {epoch}/5 ---")
            metrics = train_kn_classifier(ckpt, out_dir=out, epochs=epoch)
            finetune_results[f'{ckpt}_run_{epoch}'] = metrics
            print('Test metrics:', metrics)
    except Exception as e:
        print(f'ERROR training {ckpt}:', e)
        finetune_results[f'{ckpt}_run_{epoch}'] = {'error': str(e)}
```

Fig7.3.1: Finetuning

```
Summary (macro_f1 where available):
xlm-roberta-base_run_1: 0.8252859206126371
xlm-roberta-base_run_2: 0.8440151749551725
xlm-roberta-base_run_3: 0.851565794934398
xlm-roberta-base_run_4: 0.8590293391763231
xlm-roberta-base_run_5: 0.8549564750771026
bert-base-multilingual-cased_run_1: 0.7950839042206541
bert-base-multilingual-cased_run_2: 0.8191993905021621
bert-base-multilingual-cased_run_3: 0.8249853226727923
bert-base-multilingual-cased_run_4: 0.8263951470571318
bert-base-multilingual-cased_run_5: 0.8264632316813231
distilbert-base-multilingual-cased_run_1: 0.7390413351589925
distilbert-base-multilingual-cased_run_2: 0.7865898388286883
distilbert-base-multilingual-cased_run_3: 0.8063891727086411
distilbert-base-multilingual-cased_run_4: 0.8152594751379368
distilbert-base-multilingual-cased_run_5: 0.8153513174001692
```

Fig7.3.2: Finetuning results

```
1 finetune_results
```


```
{'xlm-roberta-base_run_1': {'eval_loss': 0.5682945251464844,
  'eval_accuracy': 0.8317803660565724,
  'eval_macro_f1': 0.8252859206126371,
  'eval_macro_precision': 0.8281502520181071,
  'eval_macro_recall': 0.8255701054810946,
  'eval_runtime': 8.4202,
  'eval_samples_per_second': 1427.525,
  'eval_steps_per_second': 22.327,
  'epoch': 1.0},
 'xlm-roberta-base_run_2': {'eval_loss': 0.4866845905780792,
  'eval_accuracy': 0.8495840266222962,
  'eval_macro_f1': 0.8440151749551725,
  'eval_macro_precision': 0.8478189691080578,
  'eval_macro_recall': 0.8415202973690793,
  'eval_runtime': 8.1345,
  'eval_samples_per_second': 1477.66,
  'eval_steps_per_second': 23.111,
  'epoch': 2.0},
 'xlm-roberta-base_run_3': {'eval_loss': 0.46427762508392334,
  'eval_accuracy': 0.8566555740432612,
  'eval_macro_f1': 0.851565794934398,
  'eval_macro_precision': 0.8542109803897123,
  'eval_macro_recall': 0.8498038175890471,
  'eval_runtime': 8.2084,
  'eval_samples_per_second': 1464.353,
  'eval_steps_per_second': 22.903,
  'epoch': 3.0},
```


Fig7.3.3: Sample of calculated metrics

```
for ckpt, mdir in model_dirs:
    print(f"\n=== Summarizing {ckpt} ({mdir}) ===")
    best_dir = os.path.join(mdir, 'best_model')
    if not os.path.isdir(best_dir):
        print('Skipping (no best_model).')
        continue
```

Fig7.3.5: Loading the best model

```
# Confusion Matrix
cm = confusion_matrix(y_te_enc, y_pred)
plt.figure(figsize=(7,6))
sns.heatmap(cm, annot=False, cmap='Blues', annot=True)
plt.title(f'{ckpt} - Confusion Matrix (Test)')
plt.xlabel('Predicted')
plt.ylabel('True')
plt.tight_layout()
cm_path = os.path.join(mdir, 'confusion_matrix_summary.png')
plt.savefig(cm_path, dpi=140)
plt.show()
```

Fig7.3.4: Confusion matrix visualization

```
# Per-class F1
p_c, r_c, f1_c, support_c = precision_recall_fscore_support(y_te_enc, y_pred, average=None, zero_division=0)
plt.rcParams['font.family'] = ['Nirmala UI', 'sans-serif']
plt.figure(figsize=(10,4))
plt.bar(range(len(label_names)), f1_c)
plt.xticks(range(len(label_names)), label_names, rotation=45, ha='right')
plt.ylim(0,1)
plt.title(f'{ckpt} - Per-class F1 (Test)')
plt.tight_layout()
```

Fig7.3.5: Visualisation of F1-score per class

```
plt.figure(figsize=(6,4))
plt.plot(df_train['step'][344:], df_train['loss'][344:], label='Train Loss')
plt.xlabel('Step')
plt.ylabel('Loss')
plt.title(f'{ckpt} - Training Loss')
plt.tight_layout()
loss_path = os.path.join(mdir, 'train_loss_curve.png')
plt.savefig(loss_path, dpi=140)
plt.show()
```

Fig7.3.6: Training loss visualisation

```
logits_list = []
labels_list = []
with torch.no_grad():
    for batch in loader:
        inp = batch['input_ids'].to(device)
        att = batch['attention_mask'].to(device)
        logits = model(input_ids=inp, attention_mask=att).logits
        logits_list.append(logits.cpu().numpy())
        labels_list.append(batch['label'].cpu().numpy())
logits = np.concatenate(logits_list, axis=0)
labels = np.concatenate(labels_list, axis=0)
labels_bin = label_binarize(labels, classes=np.arange(num_labels))
probs = np.exp(logits) / np.exp(logits).sum(axis=1, keepdims=True)
```

Fig7.3.7: Data preparation for calculation of AUC-ROC

```
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(num_labels):
    fpr[i], tpr[i], _ = roc_curve(labels_bin[:, i], probs[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
# Macro-average ROC curve
all_fpr = np.unique(np.concatenate([fpr[i] for i in range(num_labels)]))
mean_tpr = np.zeros_like(all_fpr)
for i in range(num_labels):
    mean_tpr += np.interp(all_fpr, fpr[i], tpr[i])
mean_tpr /= num_labels
macro_roc_auc = auc(all_fpr, mean_tpr)
```

Fig7.3.8: Calculation of Macro AUC-ROC

```
plt.plot(all_fpr, mean_tpr, color='navy', lw=2, label=f'Macro-average ROC (AUC={macro_roc_auc:.4f})')
for i in range(num_labels):
    plt.plot(fpr[i], tpr[i], lw=1, alpha=0.5, label=f'{le.classes_[i]} (AUC={roc_auc[i]:.3f})')
plt.plot([0, 1], [0, 1], 'k--', lw=1)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title(f'{model_class} - ROC-AUC Curve')
plt.legend(loc='lower right', fontsize=8)
plt.show()
```

Fig7.3.9: AUC-ROC curve visualisation

# Indic-Classify: Kannada News Headline Categorization with Supervised Deep Learning Models


Tejas M Prasad
*Dept. of CSE- Data Science*
*RNS Institute of Technology*
Bengaluru, India
tejasmprasad3@gmail.com

Santhosh A
*Dept. of CSE- Data Science*
*RNS Institute of Technology*
Bengaluru, India
santhosh737mc@gmail.com

S Sarvesh Balaji
*Dept. of CSE- Data Science*
*RNS Institute of Technology*
Bengaluru, India
sarveshbalaji26@gmail.com

Tharun Teja Kethineni
*Dept. of CSE- Data Science*
*RNS Institute of Technology*
Bengaluru, India
tharuntejakethineni@gmail.com

Dr. Sunitha K
*Dept. of ISE*
*RNS Institute of Technology*
Bengaluru, India
sunitha.k@rnsit.ac.in

Prashanth Kannadaguli
*Senior Data Science Trainer*
*Dhaarini Academy of Technical Education*
Bengaluru, India
prashscd@gmail.com



***Abstract*—This project focuses on developing a deep learning-based Kannada news headline categorization system to address the lack of NLP solutions for regional Indian languages, where challenges like script variations, compound words, and morphological complexity limit classifier performance. Using a dataset of ~120,000 labelled sentences collected from Kannada news portals across 10+ categories (e.g., Politics, Sports, Technology, Health), the work builds upon prior research that demonstrated strong results with pretrained Indic models (mBERT, XLM-RoBERTa) and deep learning architectures (Bi-LSTM, GRU, CNN), while noting issues like data imbalance and interpretability. The proposed approach combines traditional NLP feature engineering with modern transfer learning and ensemble methods, evaluating models through metrics such as accuracy, recall, precision, and F1-score. Implementation will leverage Python, PyTorch, HuggingFace Transformers, and IndicNLP, with deployment planned via a Gradio web application containerized in Docker. The solution aims to bridge the research gap in Indian language NLP, enhance inclusivity, and enable applications like news categorization, and recommendation systems.**



***Keywords— Kannada News Classification, Deep Learning, XLM-R, mBERT, DistilMBERT, Text Categorization, Supervised Learning, Natural Language Processing***


## I. Introduction

India's internet is buzzing with 400+ million users engaging in regional languages like Kannada, creating an urgent need for smart, automated content management systems [1]. Sorting news articles manually is a slow, costly, and error-prone task, which makes it tough to deliver real-time news updates or tailor content to individual readers' preferences [2].

The Indic-Classify project is tackling this challenge head-on by building a cutting-edge deep learning system to classify Kannada news headlines. The aim is to create a fast, reliable, and scalable tool that streamlines content workflows, making it easier for categorization of Kannada news headlines. Working with Kannada, a language known for its rich grammar and complex word structures, is no small feat, especially since it lacks large, well-organized digital datasets [3]. The project rises to this challenge by carefully collecting and curating high-quality data, resulting in a robust classifier and a valuable, standardized dataset that other researchers can build on [4]. By adapting and fine-tuning advanced AI models like transformers, the system is designed to grasp the unique linguistic nuances of Kannada, ensuring accurate and context-aware headline classification [5]. This initiative not only enhances content accessibility but also paves the way for future advancements in regional language processing [6].

## II. Kannada News Headline Categorization Using Supervised Deep Learning Models

The Indic-Classify project used three powerful, pre-trained AI models to classify Kannada news headlines. These models, known as transformer-based models, were chosen because they're great at understanding multiple languages, which is a big help for a language like Kannada that doesn't have a lot of digital resources [7]. Here's a breakdown of the models, how they work, and why they were picked:

### *A. The Models and How They Work*

#### *1) XLM-Roberta (XLM-R)*

XLM-Roberta (XLM-R) is a multilingual transformer-based AI model trained on over 100 languages, including Kannada, making it highly effective for news categorization tasks [8]. It processes Kannada headlines by breaking them into tokens using an Autotokenizer, converting them into numerical representations, and analyzing word relationships across multiple layers [9]. The [CLS] token used to capture the overall meaning of a headline and predict its category, such as Politics or Sports [10]. XLM-R was chosen because it can transfer knowledge from high-resource languages to Kannada and handle mixed-language text, ensuring accurate and reliable classification [11]. The architecture and workflow of the XLM-R model used in this project are illustrated in Fig. 1.

#### *2) Multilingual BERT (mBERT)*

Multilingual BERT (mBERT) is a transformer-based model trained on Wikipedia text over 100 languages, including Kannada, with 12 layers and 12 attention heads to capture different aspects of sentence meaning [12]. It processes Kannada headlines by breaking them into smaller word pieces, converting them into numerical form, and

analyzing their context across multiple layers, using the [CLS] token to summarize the headline and predict its category [13]. Chosen for its ability to transfer knowledge from other languages, mBERT serves as a reliable baseline for classifying Kannada headlines, especially when data is limited [14]. The architecture and functioning of the mBERT model in this project are illustrated in Fig. 2.

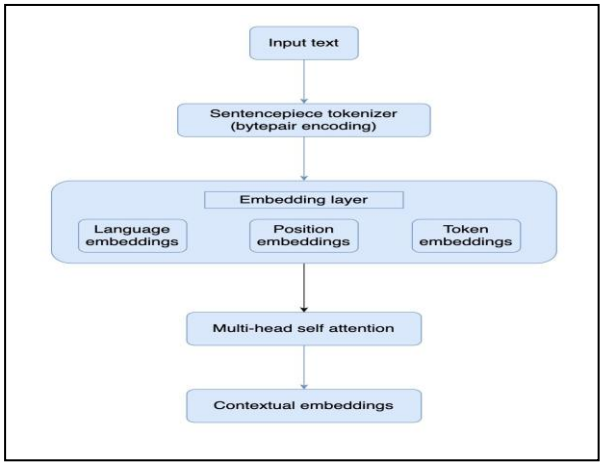


Fig. 1. XLM-R

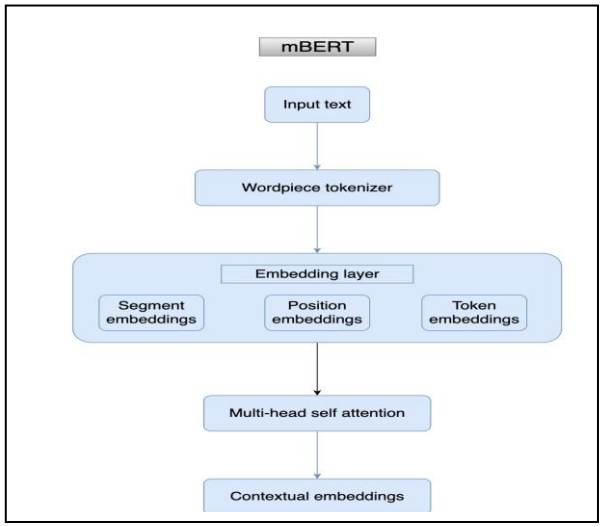


Fig. 2. mBERT

*3) Distilled Multilingual BERT (DistilMBERT)*

Distilled Multilingual BERT (DistilMBERT) is a compact and faster version of mBERT, designed with only 6 layers instead of 12, making it more memory-efficient while maintaining performance close to the original [15]. It processes Kannada headlines in the same way as mBERT—tokenizing text, converting it into numerical embeddings, and passing it through its layers before using the output to classify the headline's category [16]. Chosen for its speed and efficiency, DistilMBERT is useful in real-world applications with limited computing resources, such as deployment on websites, without sacrificing much accuracy [17]. The architecture and workflow of the DistilMBERT model used in this project are illustrated in Fig. 3.

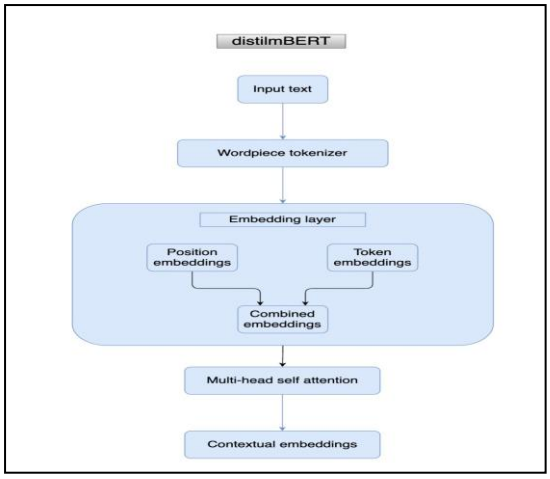


Fig. 3. DistilMBERT

## III. METHODOLOGY

The Indic-Classify project followed a structured multi-step process to train and refine its AI models [18]. In Step 1 (Data Collection), we gathered a large dataset of Kannada news headlines across multiple categories [19]. In Step 2 (Preprocessing and Exploratory Data Analysis), the data was cleaned, tokenized, and normalized, while also being analyzed to understand patterns, category distribution, and potential challenges like imbalance [20]. In Step 3 (Initial Training), baseline deep learning models were trained to classify the headlines [21]. Finally, in Step 4 (Advanced Fine-Tuning), transformer-based models was fine-tuned and optimized for higher accuracy, ensuring robust performance on Kannada headline classification [22].

### *A. Data Collection*

A corpus of 1,20,000 Kannada news headlines was collected from various sources like Prajavani and Vijaya Kannada [23], with 63 initial categories consolidated into 10 major classes such as Politics, Sports, and Entertainment to avoid data sparsity, as illustrated in Fig. 4 [24]. A preprocessing pipeline cleaned the text by removing special characters, HTML tags, and non-Kannada scripts as summarized in Table I, while Exploratory Data Analysis (EDA) examined distribution, class imbalance, and structural features like headline length [25]. Baseline feature extraction using TF-IDF, along with dimensionality reduction via PCA and t-SNE, confirmed the chances for clear category separation [26].

Smallest headline from each category

| Category | Headline |
|---|---|
| ರಾಜಕೀಯ | ಆಂತರಿಕ ಕಚ್ಚಾಟ ಮರೆಮಾಚಲು ಕಾಂಗ್ರೆಸ್ ಸುಳ್ಳು ಭರವಸೆ ನೀಡುತ್ತಿದೆ:ಅರುಣ್ ಸಿಂಗ್ |
| ವಿದೇಶ | ಭಾರತ - ಪಾಕಿಸ್ತಾನ ನಡುವೆ ಪರಮಾಣು ಯುದ್ಧ ನಡೆದರೆ 125 ಮಿಲಿಯನ್ ಮಂದಿ ಸಾವು: ಅಧ್ಯಯನ |
| ಕ್ರೀಡೆ | MS Dhoni's ₹100-crore defamation case: 10 ವರ್ಷಗಳ ನಂತರ ಕೊನೆಗೂ ವಿಚಾರಣೆ ಕೈಗೆತ್ತಿಕೊಂಡ ಮದ್ರಾಸ್ ಹೈಕೋರ್ಟ್ |
| ವಾಣಿಜ್ಯ | ಕಳಪೆ ನಗರೀಕರಣದಿಂದ 2050ಕ್ಕೆ 120 ಲಕ್ಷ ಕೋಟಿ ರೂ. ಹೊರೆ |
| ಮನರಂಜನೆ | 'ಸೈ ರಾ' ಕನ್ನಡ ಟ್ರೇಲರ್ ರಿಲೀಸ್: ಸುದೀಪ್ ಧ್ವನಿ ಕೇಳಿ ಥ್ರಿಲ್ ಆದ ಅಭಿಮಾನಿಗಳು |
| ಅಪರಾಧ ಸುದ್ದಿ | ಎಸ್‌ಎಸ್‌ಎಲ್‌ಸಿಯಲ್ಲಿ ಮೂರು ಬಾರಿ ಫೇಲ್; ಮನನೊಂದ ಬಾಲಕನಿಂದ ದೇವರ ವಿಗ್ರಹ ವಿರೂಪ! |
| ಆರೋಗ್ಯ | ಈ 7 ಬಗೆಯ ಸ್ಯೆಲೆಂಟ್ ಕಿಲ್ಲರ್ ಕಾಯಿಲೆಗಳ ಲಕ್ಷಣಗಳನ್ನು ಕಣ್ಣುಗಳಲ್ಲಿ ನೋಡಿಯೇ ಪತ್ತೆ ಹಚ್ಚಬಹುದಂತೆ! |
| ಕೃಷಿ | ಭತ್ತಕೃಷಿ: ಪರಿಸರ ಸ್ನೇಹಿ ವಿಧಾನಕ್ಕೆ ಮೊರೆ |
| ತಂತ್ರಜ್ಞಾನ | TikTok | ಟಿಕ್‌ಟಾಕ್ ಮೇಲಿನ ನಿಷೇಧ ತೆರವುಗೊಳಿಸಿಲ್ಲ: ಕೇಂದ್ರ ಸರ್ಕಾರ |
| ಸಾಹಿತ್ಯ | 'ಮೂಡಲಪಾಯ ಯಕ್ಷಗಾನ' ಕಲೆಗೆ ಜೀವ ತುಂಬುವ ಚಿಣ್ಣರು! |

Fig. 4. Categories

TABLE I. PREPROCESSING STEPS

| Preprocessing Steps | Description |
|---|---|
| HTML tag Removal | All HTML tags, scripts, and CSS content were programmatically stripped from the text to isolate the news headlines. |
| Special Character & Punctuation Filtering | Non-alphanumeric characters, symbols, and excessive punctuation were removed or replaced to reduce noise in the data. |
| Non-Kannada Character Removal | The data was filtered to remove any non-Kannada scripts or characters, ensuring the corpus is solely in the target language. |
| Whitespace Normalization | Multiple spaces and other forms of extraneous whitespace were consolidated into a single space to standardize the text format |
| Consistent Formatting | The text was converted to a consistent format, such as lower-case, to minimize variations in the same word. |

## B. *Preparing the Data and Training Process*

- Category filtering: Initially, the data contained ~1,20,000 samples 63 categories, out of which 10 were chosen namely, Literature, Technology, Politics, International, Sports, Crime, Economics, Entertainment, Health and Agriculture [27]. The categories similar to the chosen ones were merged [28], like cinema was merged to Entertainment, cricket was to Sports, etc. All the other samples categories with no similarity were removed. After this step, we ended with a clean dataset with ~90,000 samples, each headline categorized into one of the 10 categories [29].
- Balancing the Data: Some categories, like Literature and Entertainment, had way more headlines than others as seen in Fig. 5, which could skew the models' performance [30]. To fix this, we removed some more samples from the categories having a large number of samples to minimise the skewness of the data [31].The final dataset had ~60,000 samples.

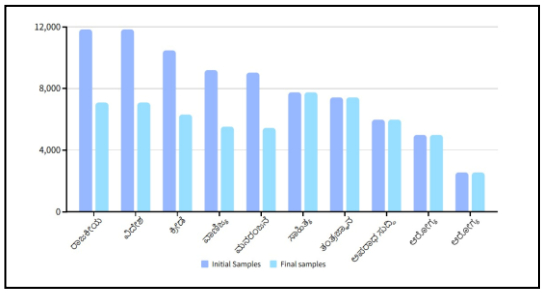


Fig. 5. Before and After Preprocessing

- Splitting the Data: The data was divided them into 80% for training (48,000 headlines) and 20% for testing (12,000 headlines) [32].
- Text normalization: We tidied up the headlines by removing punctuation, special symbols, and extra spaces to make the text clean and consistent for the models to process [33].
- Tokenization: The headlines were broken into smaller tokens using a tool called Autotokenizer, which works for all three models: XLM-R, mBERT, and DistilMBERT. This step helps the models understand the text better as shown in Fig. 6 [34]

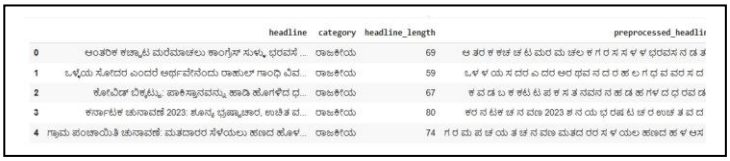


Fig. 6. Tokenization

## C. *Model Training and Optimization*

The fine-tuning process adapted pre-trained transformer models to Kannada news classification through carefully optimized hyperparameters [35]. Key configuration parameters included:

- Learning Rate: A conservative rate of $2\times10^{-5}$ with linear scheduling and warmup period prevented catastrophic forgetting while enabling effective adaptation to the target task [36].
- Training Setup: Models were trained for 5 epochs with batch size 32, using AdamW optimizer for improved generalization. Early stopping based on validation performance prevented overfitting [37].
- Regularization: Dropout and L2 weight decay techniques maintained model generalization capability during the fine-tuning process [38].

# IV. RESULTS AND DISCUSSION

The Indic-Classify project evaluated its models using metrics such as Accuracy, Precision, Recall, and F1-Score to gain a clear understanding of their strengths and weaknesses in categorization of Kannada news headlines [39]. Accuracy provided an overall measure of correct predictions, while Precision and Recall highlighted how good the models handled specific categories. The F1-Score offered a balanced view by combining both Precision and Recall, making it especially useful for handling imbalanced categories [40].

As shown in Table II, fine-tuning consistently improved model performance, with XLM-Roberta achieving the highest scores across all metrics.

TABLE II. RESULTS

| Model | Phase | Accuracy | Macro F1-Score | Precision | Recall |
|---|---|---|---|---|---|
| **XLM-Roberta** | Baseline | 83.18% | 0.8253 | 0.8282 | 0.8256 |
| | Fine-Tuned | 85.89% | 0.8549 | 0.8500 | 0.8533 |
| **mBERT** | Baseline | 80.37% | 0.7951 | 0.7989 | 0.7927 |
| | Fine-Tuned | 83.32% | 0.8265 | 0.8288 | 0.8255 |
| **DistilMBERT** | Baseline | 75.12% | 0.7390 | 0.7399 | 0.7397 |
| | Fine-Tuned | 82.19% | 0.8154 | 0.8186 | 0.8130 |

The training loss curves presented in Figs. 7–9 demonstrate effective and stable model convergence. The corresponding ROC-AUC curves shown in Figs. 10–12 indicate strong discriminatory capability across all classes. Furthermore, the confusion matrices provided in Figs. 13–15 confirm robust classification performance on the test set, with the majority of predictions concentrated along the main diagonal.

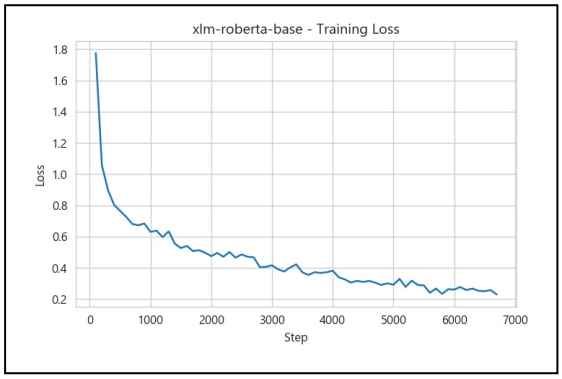


Fig. 7. Training loss curve for XLM-R

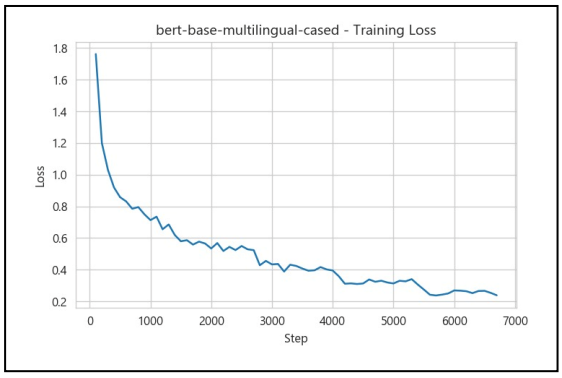


Fig. 8. Train loss curve for mBERT

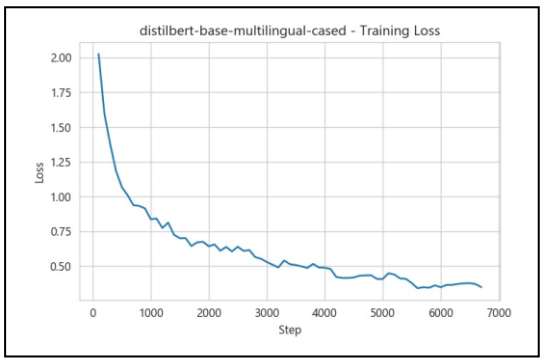


Fig. 9. Train loss curve for DistilMBERT

Our findings validate that fine-tuning transformer-based multilingual models, particularly XLM-R, is a highly effective approach for Kannada news headline classification [41]. With an accuracy of 85.89% and a macro-F1 score of 0.8549, our XLM-R model significantly outperformed mBERT (83.32%) and DistilMBERT (82.19%). These results surpass those of a comparable Kannada study that achieved only 65.71% accuracy using the same model as in [42], a success we attribute to our larger, more carefully curated dataset.

Our work also compares to prior efforts using traditional deep learning models. For instance, studies on Telugu news classification using architectures like Bi-LSTM reported maximum accuracies ranging from 63.67% to 70.92% as in [43]. The superior performance of our transformer models highlights their effectiveness in handling the complexities of regional Indian languages [44].

While some studies [45] achieved higher accuracies (up to 97%) using simpler, language-specific methods like N-gram models or specialized embeddings, our approach offers a critical advantage: generalizability. Multilingual transformers are designed for cross-lingual knowledge transfer, providing a better solution that can be applied to other low-resource languages with minimal retraining [46]. This positions our work as a scalable and practical foundation for developing future multilingual NLP systems [47].

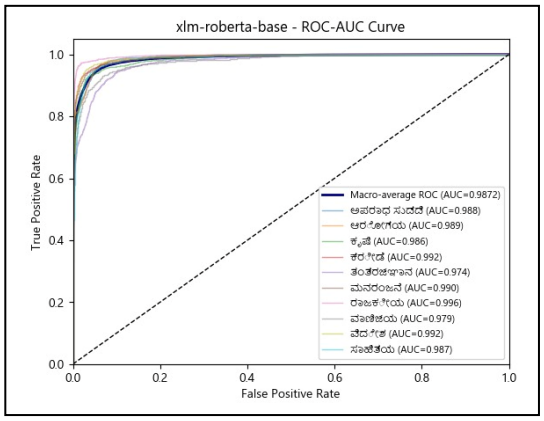


Fig. 10. ROC-AUC curve for XLM-R

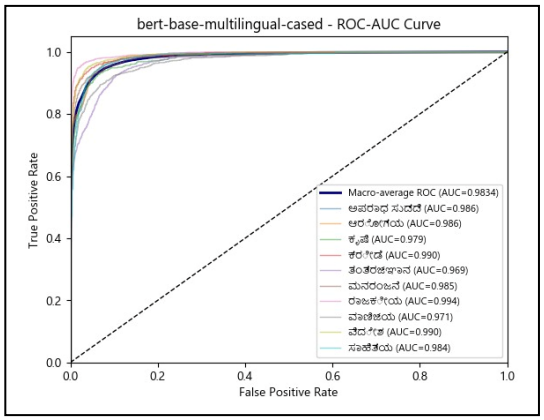


Fig. 11. ROC-AUC curve for mBERT

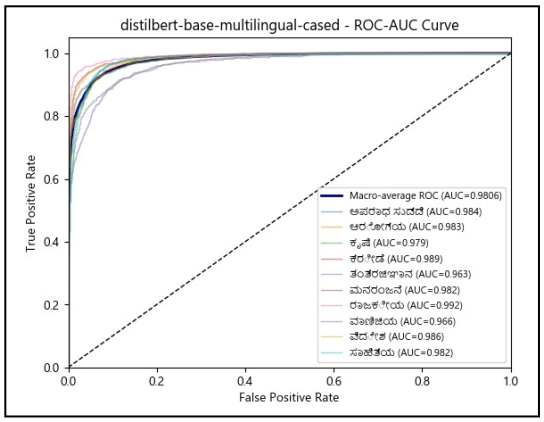

Fig. 12. ROC-AUC curve for DistilMBERT

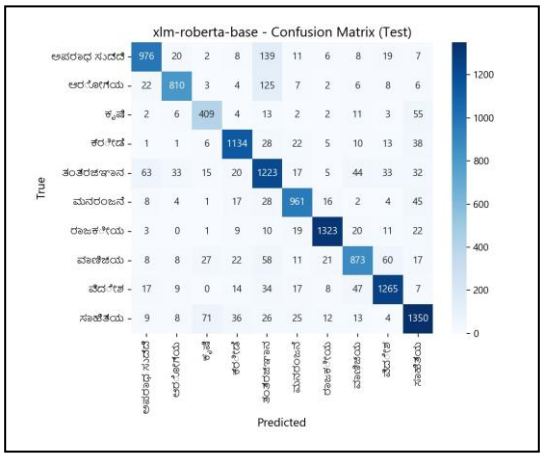


Fig. 13. Confusion matrix for XLM-R

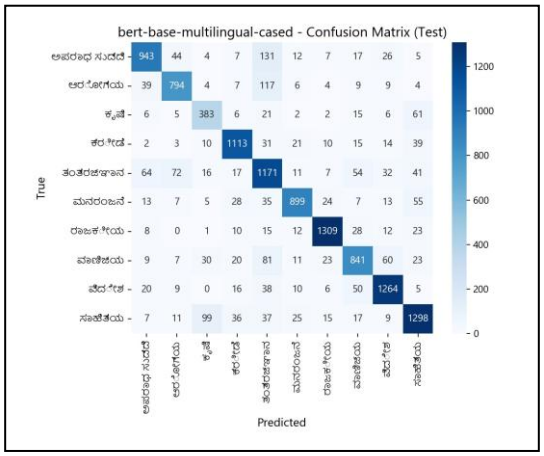


Fig. 14. Confusion matrix for mBERT

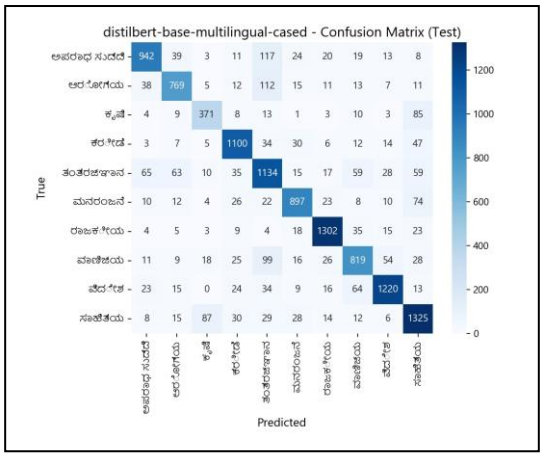


Fig. 15. Confusion matrix for DistilMBERT

## V. Conclusion

The Indic-Classify project successfully addressed the challenge of Kannada news headline categorization by systematically progressing from data collection to model fine-tuning and evaluation. Using a dataset of 60,000 headlines, the study benchmarked multiple transformer-based models and achieved significant improvements, with XLM-R delivering the best performance (accuracy 85.89%, macro F1-score 0.8549), followed by mBERT and DistilMBERT The results were validated through comprehensive metrics, including accuracy, recall, precision, and F1-score, ensuring robustness and transparency. Overall, the project demonstrated the effectiveness of transformer architectures for low-resource languages like Kannada while providing reliable benchmarks for future research.

## Refrences


[1] Internet and Mobile Association of India (IAMAI) and Kantar, “Internet in India Report 2024,” Jan 2025. [Online]. Available: https://www.iamai.in/reports.

[2] B. Naseeba, N. P. Challa, A. Doppalapudi, S. Chirag and N. S. Nair, "Machine Learning Models for News Article Classification," *2023 5th International Conference on Smart Systems and Inventive Technology (ICSSIT)*, Tirunelveli, India, 2023, pp. 1009-1016, doi: 10.1109/ICSSIT55814.2023.10061095.

[3] P. Middha, H. Agarwal, V. Rajput, A. Thakur, S. Singh and S. Saraswat, "Advancing Low Resource Natural Language Processing: Techniques, Applications, and Future Directions," *2024 Second International Conference on Advanced Computing & Communication Technologies (ICACCTech)*, Sonipat, India, 2024, pp. 337-341, doi: 10.1109/ICACCTech65084.2024.00062.

[4] A. Kunchukuttan et al., “AI4Bharat-IndicNLP Corpus: Monolingual Corpora and Word Embeddings for Indic Languages,” arXiv:2005.00085, 2020.

[5] A. Pandey, E. Sharma, R. Tiwari, A. Bisht, and B. P. Prathaban, “Precision Driven Sentiment and Topic Classification of News Articles using IndicBert,” in Proc. Int. Conf. Recent Advances in Electrical, Electronics, Ubiquitous Communication, and Computational Intelligence (RAEEUCCI), 2025, pp. 1–6, doi: 10.1109/RAEEUCCI63961.2025.11048217.

[6] S. Aggarwal, S. Kumar, and R. Mamidi, “Efficient Multilingual Text Classification for Indian Languages,” in Proc. Int. Conf. Recent Advances in Natural Language Processing (RANLP), 2021, pp. 19–25. doi: 10.26615/978-954-452-072-4_003.

[7] A. Kulkarni et al., “Mono vs Multilingual BERT for Hate Speech Detection and Text Classification: A Case Study in Marathi,” arXiv:2204.08669, 2022.

[8] A. Conneau et al., "Unsupervised cross-lingual representation learning at scale," arXiv:1911.02116, 2019.

[9] Hugging Face, “PEFT: Parameter-Efficient Fine-Tuning Methods for LLMs.” [Online]. Available: https://huggingface.co/docs/peft/index

[10] J. Chen et al., “An Analysis Method for Interpretability of CNN Text Classification Model,” Future Internet, vol. 12, no. 12, p. 228, Dec. 2020. doi: 10.3390/fi12120228.

[11] X. Lin et al., “Multilingual Text Classification for Dravidian Languages,” arXiv:2112.01705, 2021.

[12] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in Proc. Conf. North American Chapter Assoc. Comput. Linguistics, Minneapolis, MN, USA, 2019, pp. 4171–4186.

[13] M. Kowsher et al., "Bangla-BERT: Transformer-Based Efficient Model for Transfer Learning and Language Understanding," IEEE Access, vol. 10, pp. 91855-91870, 2022, doi: 10.1109/ACCESS.2022.3197662.

[14] A. Kulkarni et al., “An Attention Ensemble Approach for Efficient Text Classification of Indian Languages,” arXiv:2102.10275, 2021.

[15] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," arXiv:1910.01108, 2019.

[16] P. Deshmukh, N. Kulkarni, S. Kulkarni, K. Manghani, and R. Joshi, "Leveraging Parameter Efficient Training Methods for Low Resource Text Classification: A case study in Marathi," in Proc. 9th IEEE Int. Conf. Convergence in Technology (I2CT), Pune, India, 2024, pp. 1-6, doi: 10.1109/I2CT61223.2024.10543946.

[17] “FedP²EFT: Federated Learning to Personalize Parameter Efficient Fine-Tuning for Multilingual LLMs,” arXiv:2502.04387, 2025.

[18] H. Manai, “Machine Learning: Text Classification of News Articles,” Medium,Oct. 2022. [Online]. Available: https://medium.com/@hichemmanai/machine-learning-text-classification-of-news-articles-3ab310323ffc

[19] Analytics Vidhya, “Text Classification of News Articles,” 2024. [Online]. Available:

https://www.analyticsvidhya.com/blog/2021/07/text-classification-of-news-articles/

[20] U. Fatima, “Augmentation Techniques for Imbalanced Text Classification,” Walmart Global Tech Blog, 2024. [Online]. Available: https://medium.com/walmartglobaltech/augmentation-techniques-for-imbalanced-text-classification-a-comparative-study-38965725e17e

[21] Z. Chase, N. Genain, and O. Karniol-Tambour, “Learning Multi-Label Topic Classification of News Articles,” Dept. Comput. Sci., Stanford Univ., Stanford, CA, USA, Rep. CS229, 2013.

[22] D. Aggarwal et al., “MAPLE: Multilingual Evaluation of Parameter Efficient Finetuning of Large Language Models,” arXiv:2401.07598, 2024.

[23] AI4Bharat, “indicnlp_corpus.” [Online]. Available: https://github.com/AI4Bharat/indicnlp_corpus

[24] R. K. Shah, S. Kumar, and Shashank, "Multilabel News Category Classification using Machine Learning," in Proc. 8th Int. Conf. Communication and Electronics Systems (ICCES), Coimbatore, India, 2023, pp. 1245-1250, doi: 10.1109/ICCES57224.2023.10192826.

[25] H. Tang, S. Kamei, and Y. Morimoto, “Data Augmentation Methods for Enhancing Robustness in Text Classification Tasks,” Algorithms, vol. 16, no. 1, p. 59, 2023. doi: 10.3390/a16010059.

[26] Kaggle, “Text Augmentation.” [Online]. Available: https://www.kaggle.com/code/aoprisan/text-augmentation

[27] A. Kandarkar, “Multilabel Classification for Categorization of News Articles,” AlgoAnalytics, 2023. [Online]. Available: https://www.algoanalytics.com/blog/multilabel-classification-for-categorization-of-news-articles/

[28] C. S. K. Selvi, N. Induja, S. L. Lekshmi, and S. Nagammai, "Topic Categorization of Tamil News Articles," in Proc. Int. Conf. Computer Communication and Informatics (ICCCI), Coimbatore, India, 2022, pp. 1-6, doi: 10.1109/ICCCI54379.2022.9740925.

[29] AI4Bharat, “indicnlp_catalog.” [Online]. Available: https://github.com/AI4Bharat/indicnlp_catalog

[30] Neptune.ai, “Data Augmentation in NLP: Best Practices From a Kaggle Master,” 2023. [Online]. Available: https://neptune.ai/blog/data-augmentation-in-nlp

[31] Microsoft, “Custom text classification evaluation metrics,” Azure AI services. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/language-service/custom-text-classification/concepts/evaluation-metrics

[32] GeeksforGeeks, “Model Interpretability in Deep Learning: A Comprehensive Overview.” [Online]. Available: https://www.geeksforgeeks.org/model-interpretability-in-deep-learning-a-comprehensive-overview/

[33] D. Kakwani et al., “IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages,” arXiv:2005.00085, 2020.

[34] K. Saunack, K. Saurav, and P. Bhattacharyya, “Analysing cross-lingual transfer in lemmatisation for Indian languages,” in Proc. 28th Int. Conf. Computational Linguistics (COLING), 2020, pp. 6070–6076.

[35] E. J. Hu et al., “LoRA: Low-Rank Adaptation of Large Language Models,” arXiv:2106.09685, 2021.

[36] X. Zhou et al., “An Empirical Study on Parameter-Efficient Fine-Tuning for MultiModal Large Language Models,” in Findings of the Association for Computational Linguistics: ACL 2024, Bangkok, Thailand, 2024.

[37] A. Velankar et al., “L3Cube-MahaHate: A Tweet-Based Marathi Hate Speech Detection Dataset and BERT Models,” arXiv:2203.13778, 2022.

[38] R. Joshi, “L3Cube-MahaCorpus and MahaBERT: Marathi Monolingual Corpus, Marathi BERT Language Models, and Resources,” in Proc. WILDRE-6 Workshop within the 13th Language Resources and Evaluation Conf., Marseille, France, Jun. 2022, pp. 49–54.

[39] Cohere, “Classification Metrics Explained: Accuracy, Precision, Recall & F1 Score,” 2022. [Online]. Available: https://cohere.com/blog/classification-metrics

[40] P. Kashyap, “Understanding Precision, Recall, and F1 Score Metrics,” Medium, 2024. [Online]. Available: https://medium.com/@pranav.kashyap/understanding-precision-recall-and-f1-score-metrics-4809f6f6001a

[41] V. Gampala, J. Vallapuneni, P. K. Ande, R. K. Indurthi, and N. Rajesh, "Comparative Study on Telugu text Classification using Machine Learning and Deep Learning models," in Proc. 5th Int. Conf. Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 2021, pp. 1393-1398, doi: 10.1109/ICOEI51242.2021.9453009.

[42] Dhanusha et al., "Automated Categorization and Analysis of Kannada News Content Using Transformers," in Proc. Int. Conf. Next Generation Communication & Information Processing (INCIP), Bangalore, India, 2025, pp. 719-722, doi: 10.1109/INCIP64058.2025.11019026.

[43] V. S. Sravya, S. K. S, and K. P. Soman, "Text Categorization of Telugu News Headlines," in Proc. 2nd Int. Conf. Intelligent Technologies (CONIT), Hubli, India, 2022, pp. 1-6, doi: 10.1109/CONIT55075.2022.9848243.

[44] M. S. N. Reddy, S. Aravind, and H. R. Bojjam, "Telugu News Article Classification Using Deep Learning Models," in Proc. Int. Conf. Data Science and Network Security (ICDSNS), Tiptur, India, 2023, doi: 10.1109/ICDSNS58790.2023.10143118.

[45] G. Chetan, K. Lavanya, M. G. Kumar, and B. Naseeba, "Telugu News Classification Using N-gram Model," in Proc. 3rd Edition of IEEE Delhi Section Flagship Conf. (DELCON), New Delhi, India, 2024, doi: 10.1109/DELCON60431.2024.10536768.

[46] D. Gupta, K. Thejaa, A. R. Nair, and P. Katti, "Automated Regional Language News Article Classification System for Kannada and Telugu," in Proc. 5th Int. Conf. Advances in Electrical, Computing, Communication and Sustainable Technologies (ICAECT), Bhilai, India, 2025.

[47] H. Hanumanthappa and M. N. Swamy, “Indian Language Text Documents Categorization and Keyword Extraction,” Int. J. Comput. Trends and Technol. (IJCTA), vol. 9, no. 3, pp. 1473–1481, 2016.